

Análisis isogeométrico



Carlos Palomera Oliva
Trabajo de fin de grado de Matemáticas
Universidad de Zaragoza

Directores del trabajo: Álvaro Pé de la Riva y
Carmen Rodrigo Cardiel
Noviembre de 2025

Summary

Se creará al finalizar el trabajo.

Índice general

Summary	III
1. Introducción	1
1.1. Contexto y relación con el método de elementos finitos	1
1.2. Evolución del análisis isogeométrico	2
1.3. Objetivos	2
1.4. El problema modelo: la ecuación de Poisson	3
1.4.1. Formulación fuerte (o clásica)	3
1.4.2. Formulación variacional (o débil)	3
1.5. Estructura del documento	3
2. Construcción de estructuras mediante CAD	5
2.1. Bézier	5
2.1.1. Funciones base	5
2.1.2. Curvas	6
2.2. B-Splines	7
2.2.1. Funciones base	7
2.2.2. Curvas	8
2.3. NURBS	8
2.3.1. Funciones base	8
2.3.2. Curvas	8
2.4. Creación de superficies (producto tensorial)	9
2.5. Refinamiento de nudos (h-refinement)	10
2.5.1. Implementación de la inserción de nudos	11
3. Análisis isogeométrico	13
3.1. Espacios funcionales	13
3.2. Formulación discreta de Galerkin	14
3.3. Propiedades del sistema	14
3.4. Caso 1D	14
3.4.1. Elementos de la matriz global y el vector de carga	14
3.4.2. Cálculo de las integrales	15
3.4.3. Imposición de condiciones de contorno.	15
3.5. Caso 2D	16
3.5.1. Elementos de la matriz global y el vector de carga	16
3.5.2. Cálculo de las integrales	16
3.5.3. Imposición de condiciones de contorno	17
3.6. Convergencia en L2 y H1 y tiempo de computación teórico	18
3.6.1. Convergencia	18
3.6.2. Tiempo de computación	18

4. Implementación computacional	21
4.1. Lenguaje y librerías utilizadas	21
4.2. Métodos numéricos empleados	21
4.3. Estructura del programa	21
5. Resultados.	23
5.1. Resultados en 1D	23
5.2. Resultados en 2D	24
6. Conclusiones	27
6.1. Trabajo futuro	27
Bibliografía	29
Anexos	31
A.1. Método de los elementos finitos.	31
A.2. Demostraciones omitidas	34
A.2.1. Paso de Solución Fuerte a Solución Variacional	34
A.2.2. Existencia y unicidad de la solución variacional (lema de Lax-Milgram)	34
A.2.3. Equivalencia de la solución de la formulación variacional y fuerte	36
A.2.4. Equivalencia de normas	37
A.2.5. Condiciones del lema de Céa	38
A.2.6. Continuidad de la forma lineal dual	38
A.2.7. Propiedades de Bezier	39
A.2.8. Propiedades de los B-Splines	40
A.2.9. Igualdad de las nuevas funciones base bajo refinamiento	43
A.2.10. Equivalencia de curvas bajo refinamiento	44
A.2.11. Propiedades del sistema	45
A.3. Base teórica de los algoritmos de condiciones de contorno	46
A.3.1. Tipo Dirichlet	47
A.3.2. Tipo Neumann	47
A.3.3. Tipo Robin o mixtas	47
A.4. Código omitido	47
A.4.1. FEM	47
A.4.2. Figuras de Bézier	49
A.4.3. Código fundamental para el funcionamiento en C++	51
A.4.4. Curvas de NURBS	86
A.4.5. Superficies de NURBS	87
A.4.6. Implementaciones de IGA	88

Capítulo 1

Introducción

1.1. Contexto y relación con el método de elementos finitos

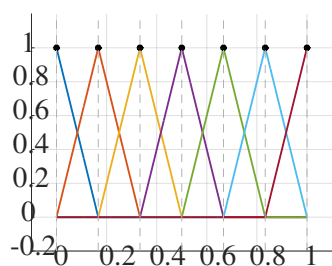
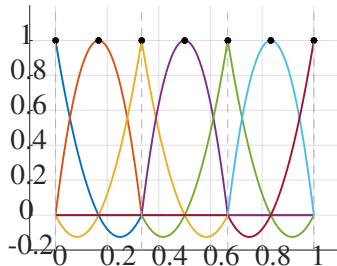
El análisis isogeométrico (IGA) es un método numérico, cuya finalidad se basa en la resolución de problemas de ecuaciones en derivadas parciales.

Se asemeja al método de elementos finitos (FEM), ambos buscan la solución mediante una combinación lineal de funciones base a partir de la formulación variacional; su principal diferencia con él radica en la preservación de la geometría original del espacio en el que se trabaja.

Cuadro 1.1: Comparación técnica entre FEM e IGA.

Criterio	FEM Clásico	Análisis Isogeométrico (IGA)
Geometría	Aproximada. Malla poligonal que introduce error geométrico.	Exacta. Geometría NURBS idéntica al CAD.
Funciones de Base	Interpolatorias (Nodos).	NURBS (Puntos de Control).
Continuidad	Baja (C^0).	Alta (C^{p-1}).
Refinamiento	h y p .	h , p y k -refinamiento.
Coste Ensamblaje	Económico. Cuadratura sencilla.	Elevado. Requiere reglas de cuadratura más complejas.
Mapeo Inverso	Directo. Inversión simple o analítica.	Complejo. Aumento del coste computacional.

Nota: Si se desea conocer más en detalle los distintos elementos del FEM para comparar más meticulosamente ambos métodos refiérase al anexo A.1.

(a) Funciones base FEM, $p=1$.(b) Funciones base FEM, $p=2$.(c) Funciones base NURBS, $p=2$.

Nota: El código fuente de estas figuras se encuentra disponible en los anexos A.4.1 y A.4.4.

1.2. Evolución del análisis isogeométrico

Para comprender el IGA, es necesario conocer la evolución del diseño asistido por ordenador (CAD), en **1967** Ahlberg, Nilson y Walsh publican su obra *Theory of Splines and Their Applications*, pionera en formalizar la teoría de los **splines**.

Con la llegada de la industria de automoción y aeronáutica en los 70 y 80, comienza la necesidad de representar formas complejas de manera eficiente impulsando así el CAD.

Durante esta época se estandarizaron las técnicas y los algoritmos esenciales que posteriormente serían las bases del IGA.

Paralelamente a este proceso, en los años 50 y 60, impulsado por la industria aeroespacial, con el fin de resolver ecuaciones en derivadas parciales en dominios complejos, surgió el FEM, ignorando el CAD y creando una brecha entre ellos que duró décadas.

En una búsqueda de unificar el diseño asistido por ordenador (CAD) y el análisis asistido por ordenador (CAE), en busca de eliminar la necesidad de crear una conversión geométrica que afecta a la precisión, nace el análisis isogeométrico.

Actualmente IGA se usa en diferentes industrias como: **automoción**, simulación de choques, **aeroespacial**, diseño de álabes de turbinas, **biomedicina**, simulación de flujo sanguíneo, **eólica**, Interacción fluido-estructura en las palas.

Finalmente mencionamos los campos de estudio en los que se avanza actualmente en referencia a IGA como son los **T-Splines**, para un mayor refinamiento local, y los métodos **espacio-temporales**, campo actualmente en investigación.

1.3. Objetivos

El objetivo principal del trabajo es entender el marco teórico y las ventajas del IGA frente al FEM, tanto en costo computacional como en orden de convergencia, en problemas de contorno de ecuaciones en derivadas parciales.

Nos centraremos en el estudio de los casos 1D y 2D, pues la generalización a dimensiones superiores es análoga.

Con el fin de cumplir este objetivo, este se segmentará en los siguientes puntos que se seguirán en el orden siguiente:

1. **Marco teórico:** Establecer el marco teórico del IGA, contrastando la formulación variacional basada en las funciones NURBS con la aproximación polinómica del FEM.
2. **Implementación computacional:** Desarrollo e implementación de código propio que pueda resolver problemas de contorno de EDPs usando el IGA.

3. **Validación y comparación:** Validar los resultados numéricos comparándolos con las soluciones analíticas de los problemas y comprobar las diferencias en velocidad de convergencia con el FEM.

1.4. El problema modelo: la ecuación de Poisson

Durante la realización del trabajo nos centraremos en el problema de Poisson como benchmark para mostrar en este las propiedades del IGA.

1.4.1. Formulación fuerte (o clásica)

Definición. Sea $\Omega \subset \mathbb{R}^n$ un conjunto abierto y acotado, donde $n \in \mathbb{N}$, con frontera $\partial\Omega = \Gamma = \overline{\Gamma_D \cup \Gamma_R \cup \Gamma_N}$, que sea suficientemente regular, tal que $\Gamma_D \cap \Gamma_R = \Gamma_D \cap \Gamma_N = \Gamma_R \cap \Gamma_N = \emptyset$.

Buscamos una función solución $u : \Omega \rightarrow \mathbb{R}$, que cumpla $u \in C^2(\Omega)$ y tal que:

$$\begin{cases} -\Delta u = f & \text{en } \Omega, \\ u = g & \text{en } \Gamma_D, \quad (\text{Condición de Dirichlet}), \\ \nabla u \cdot \mathbf{n} = h & \text{en } \Gamma_N, \quad (\text{Condición de Neumann}), \\ \alpha u + \beta \nabla u \cdot \mathbf{n} = r & \text{en } \Gamma_R, \quad (\text{Condición de Robin}). \end{cases}$$

Donde las formas bilineal $a(\cdot, \cdot)$ y lineal $L(\cdot)$ se definen como:

- $f : \Omega \rightarrow \mathbb{R}, f \in L^2(\Omega)$.
- $g : \Gamma_D \rightarrow \mathbb{R}, g \in H^{1/2}(\Gamma_D)$.
- $h : \Gamma_N \rightarrow \mathbb{R}, h \in L^2(\Gamma_N)$.
- $r : \Gamma_R \rightarrow \mathbb{R}, r \in L^2(\Gamma_R)$.
- $\mathbf{n} : \Gamma \rightarrow \mathbb{R}^n$, vector normal exterior unitario a la frontera Γ .
- $\alpha, \beta \in \mathbb{R}$, tal que $\frac{\alpha}{\beta} \geq 0$.

1.4.2. Formulación variacional (o débil)

Definición. Buscamos $u \in V(\Omega) = \{v \in H^1(\Omega) \mid v = g \text{ en } \Gamma_D\}$, tal que para $v \in V_0(\Omega) = \{v \in H^1(\Omega) \mid v = 0 \text{ en } \Gamma_D\}$, se cumple que:

$$a(u, v) = L(v). \quad (1.1)$$

Donde:

- $a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma$.
- $L(v) = \int_{\Omega} f v d\Omega + \int_{\Gamma_N} h v d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma$.

Nota: El desarrollo detallado para obtener (1.1) a partir de la formulación fuerte, la demostración de existencia y unicidad de la solución variacional, así como la prueba de que esta (bajo suficiente regularidad) es también solución del problema fuerte, se detallan en los Anexos A.2.1, A.2.2 y A.2.3.

1.5. Estructura del documento

resumen rapido de los posteriores capitulos asi como explicación de donde aparecieran el codigo o diferentes elementos en el anexo.

Se creará al finalizar el trabajo.

Capítulo 2

Construcción de estructuras mediante CAD

Las curvas y superficies creadas mediante **NURBS** son la base y el estándar del diseño CAD, durante esta sección introduciremos los conceptos necesarios para entender su funcionamiento e implementación siguiendo la progresión natural e histórica de las siguientes curvas:

$$\text{Bézier} \longrightarrow \text{B-Splines} \longrightarrow \text{NURBS}.$$

2.1. Bézier

En los años 60, impulsados por la industria de la automoción francesa y su necesidad de pasar los diseños en arcilla y papel a ordenador. Pierre Bézier y Paul Casteljau desarrollan la manera de crear curvas parametrizadas mediante el uso de **polinomios de Bernstein** como funciones base y **puntos de control**.

2.1.1. Funciones base

Definición. Los polinomios de Bernstein se definen como:

$$B_{i,p}(u) = \binom{p}{i} u^i (1-u)^{p-i}.$$

Proposición 2.1. Algunas de las propiedades fundamentales de los **polinomios de Bernstein** en este contexto son:

- **No negatividad:** $B_{i,p}(u) \geq 0, \forall u \in [0, 1]$.
- **Partición de la unidad:** $\sum_{i=0}^p B_{i,p}(u) = 1, \quad \forall u \in [0, 1]$.
- **Interpolación de los extremos:** $B_{0,p}(0) = 1, \quad B_{p,p}(1) = 1$.
- **Soporte global:** $B_{i,p}(u) > 0, \quad \forall u \in (0, 1)$.

Demostración. La demostración se encuentra en el anexo A.2.7. □

2.1.2. Curvas

Definición. Las curvas de Bézier de grado p son curvas parametrizadas en el espacio $\mathbb{R}^n$ con $n, p \in \mathbb{N}$, de la forma:

$$\mathbf{C}(u) = \sum_{i=0}^p B_{i,p}(u) \mathbf{P}_i, \quad u \in [0, 1].$$

Donde:

- $\{\mathbf{P}_0, \mathbf{P}_1, \dots, \mathbf{P}_p\}^1 \subset \mathbb{R}^n$, es el conjunto formado por los **puntos de control**.

Proposición 2.2. Sea $\mathbf{C}(u) = \sum_{i=0}^p B_{i,p}(u) \mathbf{P}_i$ una curva de Bézier de grado p , con $u \in [0, 1]$. Se satisfacen las siguientes propiedades:

1. **Interpolación en los extremos:** $\mathbf{C}(0) = \mathbf{P}_0$ y $\mathbf{C}(1) = \mathbf{P}_p$.
2. **Tangencia en los extremos:**

$$\mathbf{C}'(0) = p(\mathbf{P}_1 - \mathbf{P}_0), \quad \mathbf{C}'(1) = p(\mathbf{P}_p - \mathbf{P}_{p-1}).$$

Implicación: La continuidad C^1 entre dos curvas de Bézier unidas requiere la colinealidad de $\mathbf{P}_{p-1}, \mathbf{P}_p$ (primera curva) y $\mathbf{P}_1$ (segunda curva).

3. **Invarianza afín:** Sea $\Phi: \mathbb{R}^n \rightarrow \mathbb{R}^n$ una transformación afín ($\Phi(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$). Entonces:

$$\Phi\left(\sum_{i=0}^p B_{i,p}(u) \mathbf{P}_i\right) = \sum_{i=0}^p B_{i,p}(u) \Phi(\mathbf{P}_i).$$

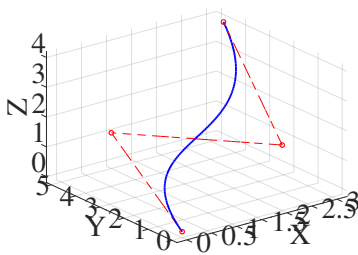
Demostración. La demostración se encuentra en el anexo A.2.7. □

Uno de los principales inconvenientes de las curvas de Bézier es su incapacidad de representar cónicas de forma exacta, para ello se introduce el concepto de racionalizarlas.

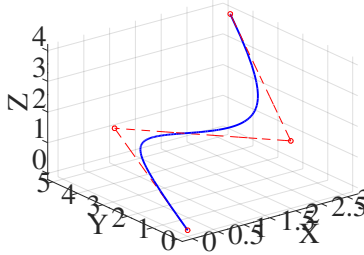
Proposición 2.3. Racionalización de las curvas de Bézier.

Se le asigna un peso $w_i \in \mathbb{R}^+$ a cada punto de control $\mathbf{P}_i$, de manera que elevamos cada uno a coordenadas homogéneas en el **espacio proyectivo** $\mathbb{P}^n$ ² y proyectando de vuelta al espacio euclídeo $\mathbb{R}^n$.

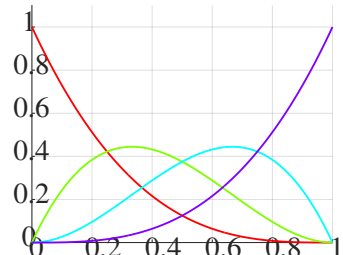
$$\mathbf{C}^R(u) = \frac{\sum_{i=0}^p B_{i,p}(u) w_i \mathbf{P}_i}{\sum_{i=0}^p B_{i,p}(u) w_i}$$



(a) Curva Bézier no racional.



(b) Curva racional, $W = (1, 10, 7, 1)^T$.



(c) Funciones base de Bernstein.

Nota: El código fuente para la generación de estas figuras se encuentra disponible en el anexo A.4.2.

¹El conjunto de puntos de control puede contener repeticiones, es decir, **NO** satisfacen necesariamente $\mathbf{P}_i \neq \mathbf{P}_j$, si $i \neq j$.

²Sea $\sim$ una relación de equivalencia, donde $\mathbf{X} \sim \mathbf{Y}$ si $\mathbf{Y} = \lambda \mathbf{X}$, con $\lambda \in \mathbb{R} \setminus \{0\}$. El **espacio proyectivo** se define como $\mathbb{P}^n = (\mathbb{R}^{n+1} \setminus \{\mathbf{0}\}) / \sim$.

2.2. B-Splines

A pesar de que la racionalización de las curvas de Bézier permitió la representación de cónicas exactas, hubo otros problemas intrínsecos a estas que motivaron la transición en los años 70 debido a la necesidad de representar estructuras más complejas en la industria aeroespacial y de automoción, se transiciona entonces a la implementación de funciones base **B-spline** que permitían, debido a tener un soporte local y un número de elementos no ligado obligatoriamente al grado polinomial de estas, superar las limitaciones de las funciones base de Bernstein.

2.2.1. Funciones base

Definición. El **Vector de nudos** se define como

$$U = \{u_0, u_1, \dots, u_m\},$$

cumplen $u_i \leq u_{i+1}$, $i = 0, \dots, m-1$, el número de nudos $m+1$ está relacionado con p y n tal que cumple: $m = n + p + 1$.

A lo largo del trabajo nos enfocaremos en los vectores de nudos **abiertos**, es decir, aquellos cuyos extremos se repiten $p+1$ veces, $u_0 = u_1 = \dots = u_p$, $u_m = u_{m-1} = \dots = u_{m-p+1}$.

Definición. Las **Funciones base B-Spline**, $N_{i,p}(u)$: se definen recursivamente mediante el **algoritmo de Cox-de Boor**:

- Caso base ($p = 0$):

$$N_{i,0}(u) = \begin{cases} 1 & \text{si } u_i \leq u < u_{i+1}, \\ 0 & \text{en caso contrario.} \end{cases}$$

- Caso genérico ($p \geq 1$):

$$N_{i,p}(u) = \frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u)^3.$$

Proposición 2.4. Algunas de las propiedades importantes de las funciones base B-Splines son:

1. **Soporte Local:** Cada función base $N_{i,p}(u)$ es no nula solo en un número limitado de intervalos de nudo, concretamente en $[u_i, u_{i+p+1})$. En consecuencia la curva adquiere la propiedad de **control local**: mover un punto de control $\mathbf{P}_i$ solo altera la forma de la curva en una región limitada.
2. **No Negatividad:** El valor de las funciones base B-Spline es siempre no negativo en todo el dominio paramétrico, es decir, $\forall u \in [u_0, u_m]$, $N_{i,p}(u) \geq 0$.
3. **Número de Bases Activas:** Para cualquier intervalo de nudos $[u_i, u_{i+1})$, las únicas funciones base $N_{j,p}(u)$ cuyo soporte contiene dicho intervalo son aquellas con el índice:

$$j \in \{i-p, i-p+1, \dots, i\}$$

Por tanto el número de funciones base no nulas en el intervalo es a lo sumo $p+1$.

El número exacto de funciones base no nulas decrece con la multiplicidad k de u_i , reduciéndose a $p+1-k$.

4. **Partición de la Unidad:** $\forall u \in [u_0, u_m]$, se cumple, $\sum_{i=0}^n N_{i,p}(u) = 1$.

Demostración. La demostración puede encontrarse en el anexo A.2.8. □

³Para el caso en el que los coeficientes toman valor $\frac{0}{0}$ se les asignará como valor 1.

2.2.2. Curvas

Definición. Con estas definiciones podemos ahora definir la curva generada por los B-Splines:

$$C(u) = \sum_{i=0}^n N_{i,p}(u) \cdot \mathbf{P}_i.$$

Proposición 2.5. *Algunas de las propiedades importantes de las curvas B-Splines son:*

1. **Invarianza bajo Transformaciones Afines:** *La aplicación de una transformación afín al polígono de control resulta en la misma transformación aplicada directamente a la curva generada.*
2. **Continuidad Controlable:** *La continuidad de la curva en cualquier unión (nudo) se controla localmente mediante la multiplicidad k^4 del nudo, sin depender de la manipulación compleja de los puntos de control. La continuidad es C^r , donde $r = p - k$.*

Demostración. La demostración puede encontrarse en el anexo A.2.8. □

2.3. NURBS

La transición de **B-splines** a **NURBS** no fue instantánea, se tuvo que esperar hasta **1979**, donde Boeing sufría las consecuencias de no poder representar cónicas exactas mediante B-splines, teniendo así su software segmentado, usando un programa basado en B-splines para el fuselaje y otro basado en las **cónicas de Rho** para las partes cónicas, siendo incapaces de conectarlos sin perder precisión.

En este caos Boeing inicia el proyecto **TIGER**, reescribiendo así todo su sistema CAD interno basándose en la tesis de **Verspille**, proponiendo así en **1981** a la IGES que las **NURBS** sean el estándar universal, transicionando así la industria de manera natural entre **1983** y **1990**.

2.3.1. Funciones base

Definición. Las **Funciones base NURBS** se definen como:

$$R_i^p(u) = \frac{N_{i,p}(u)w_i}{\sum_{j=0}^n N_{j,p}(u)w_j}. \quad (2.1)$$

Proposición 2.6. *Con el fin de preservar todas las propiedades de las **funciones base B-spline** dadas en 2.4, nos restringiremos al caso $w_i > 0, \forall i$.*

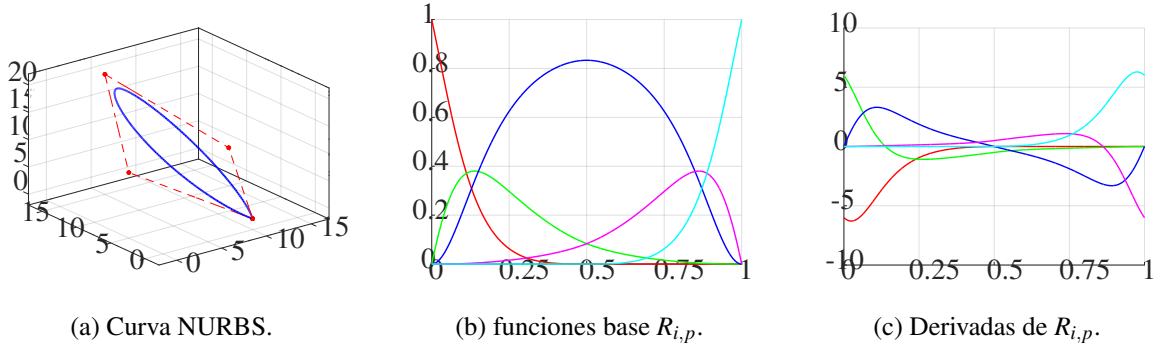
2.3.2. Curvas

Definición. Podemos ahora definir las curvas generadas por NURBS como:

$$C(u) = \sum_{i=0}^n R_i^p(u) \cdot \mathbf{P}_i = \frac{\sum_{i=0}^n N_{i,p}(u)w_i \mathbf{P}_i}{\sum_{j=0}^n N_{j,p}(u)w_j}. \quad (2.2)$$

Proposición 2.7. *Debido a la restricción impuesta anteriormente ($w_i > 0, \forall i$), las **curvas NURBS** también mantienen las propiedades de las **curvas B-spline** dadas en 2.5.*

⁴La multiplicidad k de un nudo u_i es el número de veces que el valor u_i aparece en el vector de nudos U , debe cumplirse que $k \leq p$ en los nudos interiores.



Nota: El código fuente para la generación de estas figuras se encuentra disponible en el anexo A.4.4.

2.4. Creación de superficies (producto tensorial)

El producto tensorial es el estándar de CAD para la creación de superficies, se define como:

$$\mathbf{B}_{i,j}(u, v) = N_{i,p}(u) \cdot M_{j,q}(v).$$

Donde $N_{i,p}(u)$ son las funciones base en la dirección de u y $M_{j,q}(v)$ son las funciones base en la dirección de v .

Mediante este producto definimos la superficie generada por **B-Splines** mediante la fórmula:

$$\mathbf{S}(u, v) = \sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u) M_{j,q}(v) \mathbf{P}_{i,j}.$$

Redefiniendo la estructura en la que se encuentran los puntos de control, pasando a formar ahora una matriz.

Podemos ahora racionalizar estas curvas para obtener superficies generadas mediante **NURBS**, para ello utilizamos una matriz de pesos $w_{i,j}$ y la fórmula:

$$\mathbf{S}(u, v) = \frac{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u) M_{j,q}(v) w_{i,j} \mathbf{P}_{i,j}}{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u) M_{j,q}(v) w_{i,j}} = \sum_{i=0}^n \sum_{j=0}^m R_{i,j}^{p,q}(u, v) \mathbf{P}_{i,j}. \quad (2.3)$$

Donde

$$R_{i,j}^{p,q}(u, v) = \frac{N_{i,p}(u) M_{j,q}(v) w_{i,j}}{\sum_{k=0}^n \sum_{l=0}^m N_{k,p}(u) M_{l,q}(v) w_{k,l}} \quad (2.4)$$

Proposición 2.8. Las superficies generadas mediante el producto tensorial mantienen la suavidad de las curvas que la generan en las direcciones de estas.

Demostración. Tomaremos una dirección arbitraria, v en este caso, y fijaremos $u = u_0$, la demostración es análoga para la dirección u .

$$\mathbf{S}(u_0, v) = \frac{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u_0) M_{j,q}(v) w_{i,j} \mathbf{P}_{i,j}}{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u_0) M_{j,q}(v) w_{i,j}}.$$

Ahora reagrupamos los valores que dependen de v y los que no:

$$\mathbf{S}(u_0, v) = \frac{\sum_{j=0}^m M_{j,q}(v) (\sum_{i=0}^n N_{i,p}(u_0) w_{i,j} \mathbf{P}_{i,j})}{\sum_{j=0}^m M_{j,q}(v) (\sum_{i=0}^n N_{i,p}(u_0) w_{i,j})}.$$

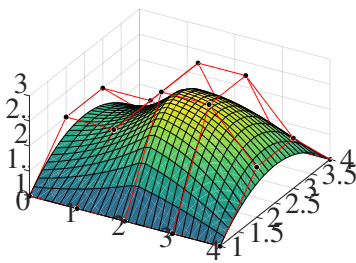
Definimos ahora los siguientes elementos:

$$\mathbf{w}_j^* = \sum_{i=0}^n N_{i,p}(u_0) w_{i,j}, \quad \mathbf{Q}_j = \frac{\sum_{i=0}^n N_{i,p}(u_0) \mathbf{P}_{i,j} w_{i,j}}{\mathbf{w}_j^*}.$$

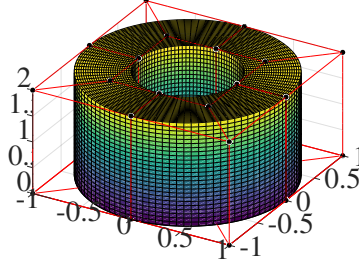
Obteniendo al final una definición

$$C(v) = S(u_0, v) = \frac{\sum_{j=0}^m M_{j,q}(v) \mathbf{w}_j^* \mathbf{Q}_j}{\sum_{j=0}^m M_{j,q}(v) \mathbf{w}_j^*}.$$

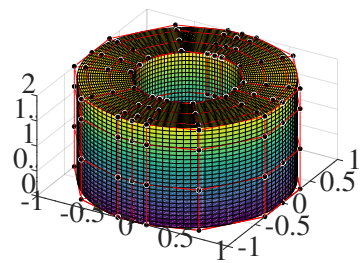
Obteniendo así la curva de NURBS asociada a la dirección v en el punto u_0 . □



(a) Superficie NURBS ejemplo 1.



(b) Superficie NURBS ejemplo 2.



(c) Ejemplo 2 refinada.

Nota: El código fuente para la generación de estas figuras se encuentra disponible en el anexo A.4.5.

2.5. Refinamiento de nudos (h-refinement)

En las figuras anteriores podemos observar un refinamiento de la superficie del ejemplo 2, este procedimiento se basa en insertar nuevos valores en un vector de nudos, introduciendo nuevos puntos de control y pesos para mantener la misma figura geométrica.

El refinamiento de nudos tiene como finalidad aumentar el control local donde se añade el nudo, obteniendo un mayor número de funciones base donde se inserta y decreciendo el tamaño de los elementos sobre los que se trabaja.

Definición. Definiremos los elementos que necesitamos para aplicar el refinamiento:

- **Puntos de control ponderados, $\mathbf{P}_i^w$:** Sea $\mathbf{P}_i$ un punto de control y $\mathbf{w}_i$ su peso asociado, definimos $\mathbf{P}_i^w = (\mathbf{w}_i \mathbf{P}_i, \mathbf{w}_i)^T$.
- **Nuevo nudo, ξ :** aunque ya tenemos el concepto de nudo definido es necesario declarar en que intervalo se encuentra, tomaremos que $\xi \in [u_k, u_{k+1})$.
- **Nuevo vector de nudos, U^* :** $U^* = \{u_0, u_1, \dots, u_k, \xi, u_{k+1}, \dots, u_m\}$.
- **Nuevos puntos de control ponderados, $\mathbf{Q}_i^w$:** estarán definidos mediante $\mathbf{Q}_i^w = \alpha_i \mathbf{P}_i^w + (1 - \alpha_i) \mathbf{P}_{i-1}^w$, donde α_i se define mediante la fórmula:

$$\alpha_i = \begin{cases} 1 & \text{si } i \leq k - p, \\ \frac{\xi - u_i}{u_{i+p} - u_i} & \text{si } k - p + 1 \leq i \leq k, \\ 0 & \text{si } i > k. \end{cases}$$

- **Nuevas funciones base, $N_{i,p}^*(u)$,** asociadas al nuevo vector de nudos U^* , que cumplen la igualdad: $N_{i,p}(u) = \alpha_i N_{i,p}^*(u) + (1 - \alpha_{i+1}) N_{i+1,p}^*(u)$ ⁵.

Proposición 2.9. La curva original ponderada es equivalente a la curva ponderada refinada,

$$\mathbf{C}^w(u) = \sum_{i=0}^m N_{i,p}(u) \mathbf{P}_i^w = \sum_{j=0}^{m+1} N_{j,p}^*(u) \mathbf{Q}_j^w$$

⁵Demostración en el anexo A.2.9

Demostración. La demostración puede encontrarse en el anexo A.2.10. □

La inserción de nudos en superficies se realiza de manera direccional, introduciendo únicamente elementos a uno de los dos vectores de nudos, modificando así una de las dos curvas que generan la superficie, pero al ser invariante bajo la inserción de nudos esta, la superficie generada final es idéntica a la original, los pesos y puntos de control en forma matricial son tratados como en la demostración de la proposición 2.8, trabajando con ellos de forma vectorial, relacionándolos con el vector de nudos que recibe la inserción.

2.5.1. Implementación de la inserción de nudos

Durante el trabajo se utilizarán las siguientes funciones para la inserción de nudos en las estructuras correspondientes:

Caso curvas (1D): La función `CurveKnotsIns` calcula los nuevos puntos de control tras insertar un nudo u con multiplicidad r .

```
iMatrix<double> CurveKnotsIns(
    int np,                // n (num. ptos control)
    int p,                // p (grado)
    std::vector<double> &KnotVector, // Vector de Nudos (U)
    const iMatrix<double> &CtrlPtsW, // Ptos Control Ponderados (Pw)
    double u,             // Valor del nudo a insertar
    int index,            // Span index (k)
    int s,                // Multiplicidad inicial
    int r                 // Veces a insertar
);
```

Listing 2.1: Parámetros para la función de inserción de nudos en curvas.

Caso superficies (2D): Para superficies NURBS, la función se generaliza para operar sobre las direcciones u (índice 1) o v (índice 2) según el parámetro booleano `dir`.

```
std::vector<iMatrix<double>> SurfaceKnotIns(
    int np1, int p1, std::vector<double> &KnotVector1, // Dir U
    int np2, int p2, std::vector<double> &KnotVector2, // Dir V
    const std::vector<iMatrix<double>> &CtrlPtsW, // Red de control
    bool dir, // true = Dir U, false = Dir V
    double uv, // Valor del nudo a insertar
    int index, // Span index
    int r, // Veces a insertar
    int s // Multiplicidad inicial
);
```

Listing 2.2: Parámetros para la función de inserción de nudos en superficies.

La correspondencia matemática de los parámetros es:

- `np`, `np1`, `np2`: Corresponden al número de puntos de control (n).
- `CtrlPtsW`: Representa los puntos de control en coordenadas homogéneas $\mathbf{P}_{i,j}^w = (w_i x_i, w_i y_i, w_i z_i, w_i)$.
- `index`: Corresponde al índice del intervalo nodal o *span* donde se encuentra el nudo a insertar.

El código fuente de estas implementaciones puede encontrarse en el anexo A.4.3.

Capítulo 3

Análisis isogeométrico

Aunque las bases matemáticas del diseño CAD mediante NURBS se consolidaron en los años 80, hasta mediados de los 2000 existió un cuello de botella en el trabajo de ingeniería: los modelos creados en CAD debían ser transformados y aproximados por mallas poligonales para poder aplicar el FEM. Se estima que en industrias complejas, como automoción o naval, este proceso tomaba hasta el 80 % del tiempo total de análisis.

En 2005, Hughes, Cottrell y Bazilevs proponen unificar el diseño CAD y la simulación, empleando las mismas funciones NURBS como base del IGA.

Desde entonces, el IGA ha trascendido el ámbito académico, consolidándose así en industrias como la aeroespacial, de automoción y la naval, lo cual podemos observar mediante las integraciones de módulos isogeométricos en software comercial de simulación como LS-DYNA.

3.1. Espacios funcionales

Definición. Sea $\hat{\Omega}$ el **dominio paramétrico** definido en 1D por el vector de nudos U y en 2D por el producto tensorial de los vectores de nudos $U \times V$.

El **espacio físico** Ω queda parametrizado así mediante el mapeo geométrico $\mathbf{G} : \hat{\Omega} \rightarrow \Omega$ definido en 2.2 en 1D y en 2.3 en 2D.

Definición. Elemento: denotado por $\Omega_e \subset \Omega$, parametrizado mediante la restricción sobre $\hat{\Omega}$ de $\mathbf{G}$, dada por el intervalo no nulo $[u_n, u_{(n+1)})$ en 1D, o por $[u_n, u_{(n+1)}) \times [v_k, v_{(k+1)})$ en 2D,

Definición. Malla física: Sea N_{el} el número de intervalos de nudos no nulos formados por nudos consecutivos, definimos la malla física como $\mathcal{T}_h = \{\Omega_e\}_{e=1}^{N_{el}}$.

Definición. Diámetro de un elemento: $h_e = \text{diam}(\Omega_e) = \sup_{\mathbf{x}, \mathbf{y} \in \Omega_e} \|\mathbf{x} - \mathbf{y}\|_2$.

Definición. Parámetro de discretización global: $h = \max_e h_e$

Definición. Sea $\hat{V}_h$ el **espacio de funciones NURBS** definido en 1D en 2.1 y en 2D en 2.4

Definimos ahora el **espacio de aproximación** V_h como

$$V_h = \left\{ v : \Omega \rightarrow \mathbb{R} \mid v \circ \mathbf{G} \in \hat{V}_h \right\}.$$

Definición. Las **funciones de forma** ϕ_i en 1D y $\phi_{i,j}$ en 2D se definen como:¹

$$\begin{aligned}\phi_i &= \hat{R}_i^p \circ \mathbf{G}^{-1}. \\ \phi_{i,j} &= \hat{R}_{i,j}^{p,q} \circ \mathbf{G}^{-1}.\end{aligned}$$

¹A lo largo de esta sección y las siguientes, todas aquellas funciones que tomen como dominio a $\hat{\Omega}$ serán diferenciadas mediante la imposición del acento circunflejo en estas ($\hat{\cdot}$).

3.2. Formulación discreta de Galerkin

Partiendo del espacio de aproximación $V_h \subset V$ (mismo V que en la sección 1.4.2), aplicamos el principio de Galerkin al problema variacional (1.4.2). Buscamos una solución aproximada $u_h \in V_h$ tal que:

$$a(u_h, v_h) = L(v_h), \quad \forall v_h \in V_h. \quad (3.1)$$

Expresamos la solución discreta u_h como una combinación lineal de las funciones de forma definidas anteriormente:

$$u_h(\mathbf{x}) = \sum_{j=1}^{N_b} d_j \phi_j(\mathbf{x}),$$

donde d_j son los **coeficientes de control** a determinar y N_b es el número total de funciones base globales.

Sustituyendo u_h en la ecuación 3.1 y tomando sucesivamente cada función de base como función de test ($v_h = \phi_i$ para $i = 1, \dots, N_b$), obtenemos el sistema lineal de ecuaciones:

$$\sum_{j=1}^{N_b} a(\phi_j, \phi_i) d_j = L(\phi_i), \quad \forall i = 1, \dots, N_b.$$

Reescribiendo ahora en forma matricial como:

$$\mathbf{K} \mathbf{d} = \mathbf{F},$$

donde $\mathbf{K} \in \mathbb{R}^{N_b \times N_b}$ es la matriz de rigidez global con elementos $K_{i,j} = a(\phi_j, \phi_i)$, $\mathbf{d}$ es el vector de coeficientes de control a determinar y $\mathbf{F}$ es el vector de carga con componentes $F_i = L(\phi_i)$.

3.3. Propiedades del sistema

Proposición 3.1. *Las siguientes propiedades se cumplen para problemas en un abierto $\Omega \subset \mathbb{R}^n$, con N_{el} elementos.*

1. **Dispersión:** *cada fila tiene a lo sumo $(2p+1)^n$, el cual no crece en función del número de elementos del problema, creando la relación asintótica*

$$\lim_{N_{el} \rightarrow \infty} \frac{(2p+1)^n}{N_B} = 0.$$

2. **Simetría:** *los elementos $\mathbf{K}_{(i,j)} = a(\phi_j, \phi_i) = a(\phi_i, \phi_j) = \mathbf{K}_{(j,i)}$, esta propiedad ocurre si y solo la forma bilineal es simétrica, que depende de que el operador diferencial del problema sea autoadjunto.*
3. **Condicionamiento:** $\kappa(\mathbf{K}) \approx O(h^{-2})$

Demostración. La demostración puede encontrarse en el anexo A.2.11. □

3.4. Caso 1D

3.4.1. Elementos de la matriz global y el vector de carga

Transformamos la **forma bilineal** basada en las funciones de forma en la siguiente integral basada en las funciones base NURBS,

$$\begin{aligned} a(\phi_j(\mathbf{x}), \phi_i(\mathbf{x})) &= \int_{\Omega} \nabla \phi_j(\mathbf{x}) \cdot \nabla \phi_i(\mathbf{x}) d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} \phi_j(\mathbf{x}) \phi_i(\mathbf{x}) d\Gamma = \\ &= \int_{\hat{\Omega}} \left(\frac{d\hat{R}_j^p(\xi)}{d\xi} J^{-1}(\xi) \right) \left(\frac{d\hat{R}_i^p(\xi)}{d\xi} J^{-1}(\xi) \right) |\det J(\xi)| d\hat{\Omega} + \left[\frac{\alpha}{\beta} \hat{R}_j^p(\xi) \hat{R}_i^p(\xi) \right]_{\xi \in \hat{\Gamma}_R} = \\ &= \int_{\hat{\Omega}} \frac{d\hat{R}_j^p(\xi)}{d\xi} \frac{d\hat{R}_i^p(\xi)}{d\xi} \frac{1}{|\det J(\xi)|} d\hat{\Omega} + \left[\frac{\alpha}{\beta} \hat{R}_j^p(\xi) \hat{R}_i^p(\xi) \right]_{\xi \in \hat{\Gamma}_R}. \end{aligned}$$

Aplicamos lo mismo a la **forma lineal**,

$$\begin{aligned} L(\phi_i(\mathbf{x})) &= \int_{\Omega} f(\mathbf{x})\phi_i(\mathbf{x}) d\Omega + \int_{\Gamma_N} h(\mathbf{x})\phi_i(\mathbf{x}) d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} \phi_i(\mathbf{x}) d\Gamma = \\ &= \int_{\hat{\Omega}} f(\mathbf{G}(\xi))\hat{R}_i^p(\xi)|\det J(\xi)| d\hat{\Omega} + [h(\mathbf{G}(\xi))\hat{R}_i^p(\xi)]_{\xi \in \hat{\Gamma}_N} + \left[\frac{r}{\beta} \hat{R}_i^p(\xi) \right]_{\xi \in \hat{\Gamma}_R}. \end{aligned}$$

3.4.2. Cálculo de las integrales

El cálculo de las integrales definidas anteriormente se realizará elemento a elemento, entendiendo por elementos los intervalos no nulos, $\hat{\Omega}_e = [u_n, u_{n+1})$.

Dentro de cada elemento se calcularán todas las combinaciones de funciones base activas, que gracias a la teoría de NURBS y por estar las funciones de forma directamente relacionadas con estas, sabemos que hay a lo sumo $p+1$ funciones base activas en cada elemento, obteniendo así una **matriz local $\mathbf{K}^e$** de tamaño $(p+1) \times (p+1)$.

Teniendo ahora los elementos de esta matriz la forma,

$$\mathbf{K}_{i,j}^e = \int_{u_n}^{u_{n+1}} \frac{d\hat{R}_{(j+n-p-1)}^p(\xi)}{d\xi} \cdot \frac{d\hat{R}_{(i+n-p-1)}^p(\xi)}{d\xi} \frac{1}{|\det J|} d\xi.$$

Para formalizar el proceso de ensamblaje, definimos la **matriz de rigidez local extendida**, denotada por $\tilde{\mathbf{K}}^e \in \mathbb{R}^{N_b \times N_b}$, la cual se construye explícitamente mediante la asignación

$$\tilde{\mathbf{K}}_{(i+n-p-1, j+n-p-1)}^e = \mathbf{K}_{i,j}^e, \quad \forall i, j \in \{1, 2, \dots, p+1\},^2$$

siendo cero cualquier otro elemento de $\tilde{\mathbf{K}}^e$ cuyos índices no satisfagan esta correspondencia.

Finalmente la matriz global $\mathbf{K}$ se obtiene al sumar estas matrices extendidas

$$\mathbf{K} = \sum_{e=1}^{N_{el}} \tilde{\mathbf{K}}^e.$$

Aplicamos el mismo procedimiento ahora a la forma lineal, definimos el **vector de carga local $\mathbf{F}^e \in \mathbb{R}^{(p+1)}$** cuyos elementos definimos,

$$\mathbf{F}_i^e = \int_{u_n}^{u_{n+1}} f(\mathbf{G}(\xi))R_{(i+n-p-1)}^p(\xi)|\det J| d\xi.$$

Definimos el **vector de carga local extendido $\tilde{\mathbf{F}}_i^e \in \mathbb{R}^{N_b}$** , construido mediante la asignación

$$\tilde{\mathbf{F}}_{(i+n-p-1)}^e = \mathbf{F}_i^e, \quad \forall i \in \{1, 2, \dots, p+1\},^3$$

siendo cero cualquier otro elemento de $\tilde{\mathbf{F}}^e$ que no satisfaga la correspondencia.

Para finalizar se obtiene el vector global $\mathbf{F}$ al sumar todos los vectores extendidos

$$\mathbf{F} = \sum_{e=1}^{N_{el}} \tilde{\mathbf{F}}^e.$$

3.4.3. Imposición de condiciones de contorno.

Tras el ensamblado de la matriz de rigidez global y del vector de carga se implementan las condiciones de contorno,

Al ser en el caso 1D, solo es posible aplicar condiciones de contorno a dos puntos del dominio, explicaremos como se aplican al punto u_0 que será análogo para el punto u_m .

Nota: La explicación de por qué se realizan así las imposiciones de contorno puede consultarse en el anexo A.3.

²En la práctica estas matrices no existirán, se sumarán directamente los elementos de $\mathbf{K}^e$ en $\mathbf{K}$.

³Al igual que en las matrices extendidas anteriores, estos vectores no existirán en la práctica y los elementos de $\mathbf{F}^e$ serán ensamblados directamente sobre $\mathbf{F}$.

Tipo Dirichlet

Sustituimos la primera (última en el caso u_m) fila de $\mathbf{K}$, el elemento (1,1) será 1, y el resto pasarán a ser 0, $\mathbf{F}_1$ tomará el valor de la condición de contorno $g(u_0)$.

Con el fin de mantener la simetría del sistema se modificarán las p filas siguientes (anteriores), estableceremos a 0 los elementos de la forma (1,j), y modificaremos los elementos $\mathbf{F}_j$ restándoles el valor $\mathbf{K}_{(1,j)}g(u_0)$.

Tipo Neumann

Actualizaremos el valor de $\mathbf{F}_1 \rightarrow \mathbf{F}_1 + h(u_0)$ ($\mathbf{F}_{N_b} \rightarrow \mathbf{F}_{N_b} + h(u_m)$).

Tipo Robin o mixtas

Actualizaremos tanto $\mathbf{K}_{(1,1)} \rightarrow \mathbf{K}_{(1,1)} + \frac{\alpha}{\beta}$, ($\mathbf{K}_{(N_b,N_b)} \rightarrow \mathbf{K}_{(N_b,N_b)} + \frac{\alpha}{\beta}$) y $\mathbf{F}_1 \rightarrow \mathbf{F}_1 + \frac{r}{\beta}$, ($\mathbf{F}_{N_b} \rightarrow \mathbf{F}_{N_b} + \frac{r}{\beta}$).

3.5. Caso 2D

3.5.1. Elementos de la matriz global y el vector de carga

Forma bilineal:

$$\begin{aligned} a(\phi_{(i_2,j_2)}(\mathbf{x}), \phi_{(i_1,j_1)}(\mathbf{x})) &= \int_{\Omega} \nabla \phi_{(i_2,j_2)}(\mathbf{x}) \cdot \nabla \phi_{(i_1,j_1)}(\mathbf{x}) d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} \phi_{(i_2,j_2)}(\mathbf{x}) \phi_{(i_1,j_1)}(\mathbf{x}) d\Gamma = \\ &= \int_{v_0}^{v_{m_v}} \int_{u_0}^{u_{m_u}} \left(\mathbf{J}^{-T}(\xi, \eta) \hat{\nabla} \hat{R}_{(i_2,j_2)}^{(p,q)}(\xi, \eta) \right)^T \cdot \left(\mathbf{J}^{-T}(\xi, \eta) \hat{\nabla} \hat{R}_{(i_1,j_1)}^{(p,q)}(\xi, \eta) \right) |\det \mathbf{J}(\xi, \eta)| d\xi d\eta \\ &\quad + \int_{\hat{\Gamma}_R} \frac{\alpha}{\beta} \hat{R}_{(i_2,j_2)}^{(p,q)}(\xi, \eta) \hat{R}_{(i_1,j_1)}^{(p,q)}(\xi, \eta) J|_{\hat{\Gamma}}(\xi, \eta) d\hat{\Gamma} = \\ &= \int_{v_0}^{v_{m_v}} \int_{u_0}^{u_{m_u}} \left(\hat{\nabla} \hat{R}_{(i_2,j_2)}^{(p,q)}(\xi, \eta) \right)^T \left(\mathbf{J}^{-1}(\xi, \eta) \mathbf{J}^{-T}(\xi, \eta) \right) \left(\hat{\nabla} \hat{R}_{(i_1,j_1)}^{(p,q)}(\xi, \eta) \right) |\det \mathbf{J}(\xi, \eta)| d\xi d\eta \\ &\quad + \int_{\hat{\Gamma}_R} \frac{\alpha}{\beta} \hat{R}_{(i_2,j_2)}^{(p,q)}(\xi, \eta) \hat{R}_{(i_1,j_1)}^{(p,q)}(\xi, \eta) J|_{\hat{\Gamma}}(\xi, \eta) d\hat{\Gamma}. \end{aligned}$$

Forma lineal:

$$\begin{aligned} L(\phi_{(i_1,j_1)}(\mathbf{x})) &= \int_{\Omega} f(\mathbf{x}) \phi_{(i_1,j_1)}(\mathbf{x}) d\Omega + \int_{\Gamma_N} h(\mathbf{x}) \phi_{(i_1,j_1)}(\mathbf{x}) d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} \phi_{(i_1,j_1)}(\mathbf{x}) d\Gamma = \\ &= \int_{v_0}^{v_{m_v}} \int_{u_0}^{u_{m_u}} f(\mathbf{G}(\xi, \eta)) \hat{R}_{(i_1,j_1)}^{(p,q)}(\xi, \eta) |\det \mathbf{J}(\xi, \eta)| d\xi d\eta \\ &\quad + \int_{\hat{\Gamma}_N} h(\mathbf{G}(\xi, \eta)) \hat{R}_{(i_1,j_1)}^{(p,q)}(\xi, \eta) J|_{\hat{\Gamma}}(\xi, \eta) d\hat{\Gamma} + \int_{\hat{\Gamma}_R} \frac{r}{\beta} \hat{R}_{(i_1,j_1)}^{(p,q)}(\xi, \eta) J|_{\hat{\Gamma}}(\xi, \eta) d\hat{\Gamma}. \end{aligned}$$

3.5.2. Cálculo de las integrales

Tomaremos ahora intervalos no nulos de la forma $\hat{\Omega}_e = [u_n, u_{n+1}) \times [v_k, v_{k+1})$.

Para este caso la **matriz local** $\mathbf{K}^e$ será de tamaño $(p+1)(q+1) \times (p+1)(q+1)$, con el fin de facilitar la comprensión de la correlación entre funciones base y elementos de la matriz tomaremos una aplicación que relacione los índices de los elementos $\mathbf{K}_{(i,j)}^e$ con el número de funciones activas en cada dirección del elemento correspondiente:

$$i = (i_1 - 1) \cdot (q + 1) + j_1 = (i_1, j_1), \quad (3.2)$$

$$j = (i_2 - 1) \cdot (q + 1) + j_2 = (i_2, j_2), \quad i_1, i_2 \in \{1, 2, \dots, p+1\}, \quad j_1, j_2 \in \{1, 2, \dots, q+1\}. \quad (3.3)$$

Teniendo así la integral que le corresponde al elemento:

$$\mathbf{K}_{((i_1, j_1), (i_2, j_2))}^e = \int_{v_k}^{v_{k+1}} \int_{u_n}^{u_{n+1}} \left(\hat{\mathbf{V}} \hat{\mathbf{R}}_{(i_2+n-p-1, j_2+k-q-1)}^{(p, q)} \right)^T (\mathbf{J}^{-1} \mathbf{J}^{-T}) \left(\hat{\mathbf{V}} \hat{\mathbf{R}}_{(i_1+n-p-1, j_1+k-q-1)}^{(p, q)} \right) |\det \mathbf{J}| d\xi d\eta.$$

Definimos de la misma manera ahora la **matriz local extendida** $\tilde{\mathbf{K}}^e$,

$$\tilde{\mathbf{K}}_{((i_1+n-p-1, j_1+k-q-1), (i_2+n-p-1, j_2+k-q-1))}^e = \mathbf{K}_{((i_1, j_1), (i_2, j_2))}^e.$$

Formando así la **matriz de rigidez** $\mathbf{K}$ mediante la suma de estas matrices:

$$\mathbf{K} = \sum_{e=1}^{N_{el}} \tilde{\mathbf{K}}^e.$$

Realizamos el mismo procedimiento para el vector de carga:

$$\begin{aligned} \mathbf{F}_{(i_1, j_1)}^e &= \int_{v_k}^{v_{k+1}} \int_{u_n}^{u_{n+1}} f(\mathbf{G}) \hat{\mathbf{R}}_{(i_1, j_1)}^{(p, q)} |\det \mathbf{J}| d\xi d\eta. \\ \tilde{\mathbf{F}}_{(i_1+n-p-1, j_1+k-q-1)}^e &= \mathbf{F}_{(i_1, j_1)}^e. \\ \mathbf{F} &= \sum_{e=1}^{N_{el}} \tilde{\mathbf{F}}^e. \end{aligned}$$

3.5.3. Imposición de condiciones de contorno

Se aplicarán los siguientes algoritmos para los elementos que contengan a la frontera de nuestro espacio físico.

Sea $\hat{\Omega}_e = [u_n, u_{n+1}] \times [v_k, v_{k+1}]$, un elemento paramétrico tal que $\mathbf{G}(\hat{\Omega}_e)$ contiene parte de la frontera de Ω , dado que la frontera de Ω coincide con la frontera paramétrica, suponemos, sin pérdida de generalidad, que la frontera correspondiente a $\mathbf{G}(\hat{\Omega}_e)$ coincide únicamente con la frontera al borde paramétrico inferior, es decir, $\xi \in [u_n, u_{n+1}]$ y $\eta = v_k$ ⁴.

Recorreremos las funciones base asociadas a esta frontera, (en este caso recorreremos el índice i_1 y fijamos $j_1 = 1$), para ello proyectamos de vuelta nuestros índices $I = (i_1, 1)$ para $i_2 = 1, \dots, p+1$, mediante la ecuación 3.2.

Tipo Dirichlet

1. Para cada índice I :

- Sustituimos la fila I -ésima de la matriz global $\mathbf{K}$ por ceros, imponiendo 1 en la diagonal ($\mathbf{K}_{I,I} = 1$).
- Actualizamos el vector de carga: $\mathbf{F}_I = g(\mathbf{x}_I)$.

2. Para mantener la simetría del sistema, eliminamos las columnas I correspondientes:

- Para cada fila distinta de $I_2 \neq I$ donde $\mathbf{K}_{I_2, I} \neq 0$, modificamos el vector de carga restando la contribución conocida: $\mathbf{F}_{I_2} \leftarrow \mathbf{F}_{I_2} - \mathbf{K}_{I_2, I} \cdot g(\mathbf{x}_I)$.
- Establecemos $\mathbf{K}_{I_2, I} = 0$.

Tipo Neumann

Para cada índice I :

$$\mathbf{F}_I \leftarrow \mathbf{F}_I + \int_{u_n}^{u_{n+1}} h(\mathbf{G}(\xi, v_k)) \cdot \hat{\mathbf{R}}_{(i_1, 1)}^{(p, q)}(\xi, v_k) \cdot J_\Gamma(\xi) d\xi.$$

⁴Para los elementos esquina basta aplicar el algoritmo dos veces, una para cada borde.

Tipo Robin o mixtas

Proyectamos también en este caso $J = (i_2, 1)$ para $i_2 = 1, \dots, p+1$.

Para cada índice I :

$$\mathbf{F}_I \leftarrow \mathbf{F}_I + \int_{u_n}^{u_{n+1}} \frac{r}{\beta} \cdot \hat{R}_{(i_1,1)}^{(p,q)}(\xi, v_k) \cdot J_\Gamma(\xi) d\xi.$$

Para cada par de índices (I, J) :

$$\mathbf{K}_{I,J} \leftarrow \mathbf{K}_{I,J} + \int_{u_n}^{u_{n+1}} \frac{\alpha}{\beta} \cdot \hat{R}_{(i_1,1)}^{(p,q)}(\xi, v_k) \cdot \hat{R}_{(i_2,1)}^{(p,q)}(\xi, v_k) \cdot J_\Gamma(\xi) d\xi.$$

3.6. Convergencia en L2 y H1 y tiempo de computación teórico

3.6.1. Convergencia

Para asegurar la convergencia debemos partir de unas condiciones iniciales:

- **Regularidad de la solución:** $u \in H^{p+1}(\Omega)$.
- **Malla no degenerada:** $\det(J) > 0$ y $|\det(J)| < \infty$.

Bajo estas condiciones podemos tomar cotas del error en diferentes espacios:

- **Norma en $H^1(\Omega)$:** $\|u - u_h\|_{H^1(\Omega)} \leq C_1 h^p |u|_{H^{p+1}(\Omega)}$.
- **Norma en $L^2(\Omega)$:** $\|u - u_h\|_{L^2(\Omega)} \leq C_2 h^{p+1} |u|_{H^{p+1}(\Omega)}$.

Donde C_1 y C_2 dependen de la continuidad de las estructuras de NURBS asociadas al problema, a medida que la suavidad crece, estas decrecen.

Demostración. Se demostrará para dimensión 2, siendo análoga la demostración para el caso de dimensión 1.

Buscamos demostrar que $\forall v_h \in V_h, v_h \in H^1(\Omega)$, para ello tomamos la siguiente igualdad:

$$\|\nabla v_h\|_{L^2(\Omega)}^2 = \int_{\Omega} |\nabla v_h(\mathbf{x})|^2 d\mathbf{x} = \int_{\hat{\Omega}} \left| J^{-T} \left(\sum_{i,j} c_{i,j} \nabla \hat{R}_{i,j}^{p,q} \right) \right|^2 |\det(J)| d\hat{\Omega}.$$

Esta norma está acotada en Ω gracias a la regularidad de las funciones base NURBS y las condiciones expuestas anteriormente, gracias a esto la demostración de convergencia es análoga a la del teorema A.1, en el caso de dimensión 2. $\square$

3.6.2. Tiempo de computación

Separaremos el tiempo de computación en sus fases independientes:

Fase de evaluación

Fase previa al ensamblado para dimensiones mayores o iguales que dos, que aprovecha las propiedades del producto tensorial para evitar recalculer evaluaciones de funciones base, su coste computacional:

$$\sum_{i=1}^d Q_i \cdot n_i = \mathcal{O}(d \cdot \max_i \{Q_i \cdot n_i\}),$$

donde Q_i es el número de evaluaciones direccionales y n_i el número de divisiones correspondientes a la dimensión i .

Fase de ensamblado

Fase en la que se realizan las integrales numéricas y se ensamblan, su coste computacional:

$$\prod_{i=1}^d Q_i^2 \cdot n_i \cdot C,$$

Donde C es el coste de la integración numérica, cálculo del jacobiano y manipulación de la memoria.

Fase de resolución

Depende del tamaño final del sistema y del método elegido para resolver el sistema resultante.

Capítulo 4

Implementación computacional

A lo largo del siguiente capítulo mostraremos como se ha implementado numéricamente todo lo referente a IGA.

4.1. Lenguaje y librerías utilizadas

El lenguaje principal utilizado ha sido C++, los cálculos de las curvas y superficies NURBS y el análisis isogeométrico, han sido escritos en este lenguaje, Octave ha sido empleado como visualizador de los ficheros de salida del código en C++.

Se ha implementado la clase **iMatrix** que define una matriz de elementos abstractos, así como todos los métodos y operadores necesarios.

Las librerías necesarias para el funcionamiento e implementación son:

- **Eigen:** matrices *sparse*, optimización en el ensamblado y solvers.
- **Librería estándar:** Para contenedores y algoritmos (vector y algorithm), cálculo del tiempo de ejecución (chrono), funciones matemáticas básica (CMath) y Entrada y salida para creación de ficheros (iostream, fstream, iomanip y string).

4.2. Métodos numéricos empleados

Trataremos ahora los métodos numéricos que no son inherentes al IGA que hemos utilizado:

- **Integración numérica:** Se ha utilizado la cuadratura de Gauss-Legendre, con $p_i + 1$ puntos de cuadratura en la dirección de la dimensión i , dándonos así $\prod_{i=1}^d (p_i + 1)$ puntos de cuadratura totales.
- **Solvers numéricos:** Se ha utilizado el método LU para el caso 1D y el método del gradiente conjugado preconditionado (Cholesky incompleto o Jacobi en su defecto).

4.3. Estructura del programa

Explicaremos el orden de ejecución del programa y que se realiza en cada fase de este.

1. **Estructuras NURBS:** Se crearán el espacio físico y paramétrico referente al problema que se quiere tratar, luego se aplicará el refinado correspondiente para ajustar el tiempo de ejecución y una cota buscada.
2. **Fase de evaluaciones:** Fase opcional que nos permite calcular evaluaciones direccionales de las funciones base pre-racionalizadas (B-Splines), con el fin de luego solo tener que racionalizarlas en cada elemento.

3. **Fase de ensamblado:** Fase en la cual calculamos las integrales numéricas pertinentes a los elementos correspondientes y se ensamblan en los elementos de la matriz global y el vector de carga global, en el caso 2D se ha utilizado *multithreading* con el fin de demostrar su posibilidad de paralelización y reducir el tiempo de esta fase en un escalar.
4. **Imposición de condiciones de contorno:** Se aplicarán los algoritmos dados en las secciones 3.4.3 y 3.5.3.
5. **Resolver el sistema final:** Se implementará el solver adaptado al tamaño de la matriz, aprovechando en la medida de lo posible su estructura simétrica y su propiedad *sparse*.
6. **Construcción y evaluación de la solución:** Partiendo de la solución del sistema, la cual nos aporta el valor de los coeficientes de control, para evaluar la solución en un punto concreto, sumamos las funciones base activas multiplicadas por sus coeficientes correspondientes.
7. **Cálculo de la norma L2 y H1:** Con las evaluaciones correspondientes calcularemos las aproximaciones numéricas a las normas mediante la cuadratura de Gauss-Lengedre en cada elemento.
8. **Representación gráfica en octave:** Tomaremos los ficheros de salida del código en C++ y mediante octave crearemos las visualizaciones.

Puede encontrarse código de ejemplo en el Anexo A.4.6, para el caso 1D y 2D.

Capítulo 5

Resultados.

5.1. Resultados en 1D

Tomaremos el problema en 1D definido mediante $f(x) = 100 \frac{\pi^2}{L^2} \sin(10\pi \frac{x}{L})$, $u(0) = 0$, $u(L) = 0$.

Problema 1

- **Grado polinomial p:** 2.
- **Dominio paramétrico:** $U = \{0, 0, 0, \frac{1}{2}, \frac{1}{2}, 1, 1, 1\}$.
- **Dominio físico:** Semicircunferencia contenida en el plano XY, centrada en el origen de radio 1.
- **Solución y derivada analítica:** $u(x) = \sin(10\pi \frac{x}{\pi})$, $u'(x) = 10 \cos(10\pi \frac{x}{\pi})$.
- **Puntos de control y pesos:**

$$\mathbf{P} = \left[\begin{pmatrix} -1 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} -1 \\ 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \right], \quad \mathbf{w} = \left[1, \frac{\sqrt{2}}{2}, 1, \frac{\sqrt{2}}{2}, 1 \right].$$

Cuadro 5.1: Tabla de convergencia y costes computacionales.

Nº Elementos	Tiempo (s)	Error L^2	Error H^1
16	0,0034002	0,0512509	5,15944
32	0,0074421	0,00415079	0,827538
64	0,0123962	0,000451444	0,182791
128	0,0244885	$5,43459 \cdot 10^{-5}$	0,0442749
256	0,0484195	$6,72829 \cdot 10^{-6}$	0,010981

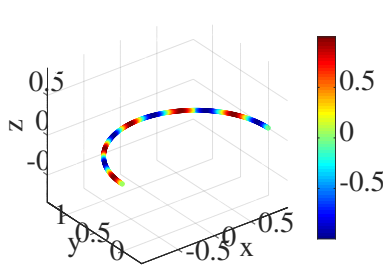
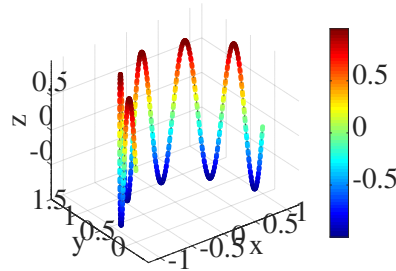
Problema 2

- **Grado polinomial p:** 3.
- **Dominio paramétrico:** $U = \{0, 0, 0, 0, 1, 1, 1, 1\}$.
- **Dominio físico:** Curva Alabeada $x = (\xi, \xi^2, \xi^3)$, $L \approx 1,86302298656134$.
- **Solución y derivada analítica:** $u(x) = \sin(10\pi \frac{x}{L})$, $u'(x) = 10 \frac{\pi}{L} \cos(10\pi \frac{x}{L})$.
- **Puntos de control y pesos:**

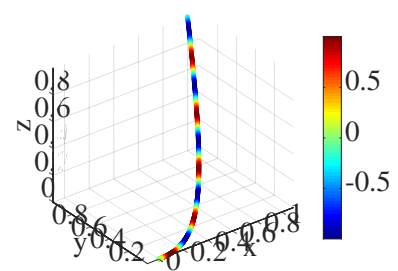
$$\mathbf{P} = \left[\begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} \frac{1}{3} \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} \frac{2}{3} \\ \frac{1}{3} \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \right], \quad \mathbf{w} = [1, 1, 1, 1].$$

Cuadro 5.2: Tabla de convergencia y costes computacionales.

Nº Elementos	Tiempo (s)	Error L^2	Error H^1
16	0,004775	0,19235	7,23543
32	0,0095162	0,00634774	0,742005
64	0,0149659	0,000235597	0,0739008
128	0,0443802	$1,15083 \cdot 10^{-5}$	0,00816382
256	0,0857816	$6,75238 \cdot 10^{-7}$	$1,06953 \cdot 10^{-5}$

(a) Semicircunferencia, $p=2$.

(b) Eje Z tomando valores solución.

(c) Curva Alabeada, $p=3$.

5.2. Resultados en 2D

Problema 1

- **Grados polinomiales:** $q = p = 2$.
- **Dominio paramétrico:** $U = V = \{0, 0, 0, 2, 2, 2\}$.
- **Dominio físico:** $[0, 2] \times [0, 2] \times \{0\}$, parametrizado mediante la identidad sobre los valores X e Y.
- **Ecuaciones del problema:** $f(x, y) = 2\pi^2 \sin(\pi x) \sin(\pi y)$, $u(\Gamma) = 0$.
- **Solución y derivadas analíticas:**
 - $u(x, y) = \sin(\pi x) \sin(\pi y)$.
 - $\frac{\partial u}{\partial x}(x, y) = \pi \cos(\pi x) \sin(\pi y)$.
 - $\frac{\partial u}{\partial y}(x, y) = \pi \sin(\pi x) \cos(\pi y)$.
- **Puntos de control y pesos:**

$$\mathbf{P} = \begin{pmatrix} (0, 0, 0) & (1, 0, 1) & (2, 0, 0) \\ (0, 1, 0) & (1, 1, 0) & (2, 1, 0) \\ (0, 2, 0) & (1, 2, 0) & (2, 2, 0) \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}.$$

Cuadro 5.3: Tabla de convergencia y costes computacionales.

Nº Elementos	T. Ensamblado (s)	T. Resolución (s)	Error L^2	Error H^1
16×16	0,0070	0,0013	$4,36 \cdot 10^{-4}$	$2,60 \cdot 10^{-2}$
32×32	0,0251	0,0013	$5,23 \cdot 10^{-5}$	$6,42 \cdot 10^{-3}$
64×64	0,0604	0,0244	$6,46 \cdot 10^{-6}$	$1,60 \cdot 10^{-3}$
128×128	0,2064	0,1083	$8,06 \cdot 10^{-7}$	$3,99 \cdot 10^{-4}$
256×256	0,6724	0,7157	$1,01 \cdot 10^{-7}$	$9,97 \cdot 10^{-5}$

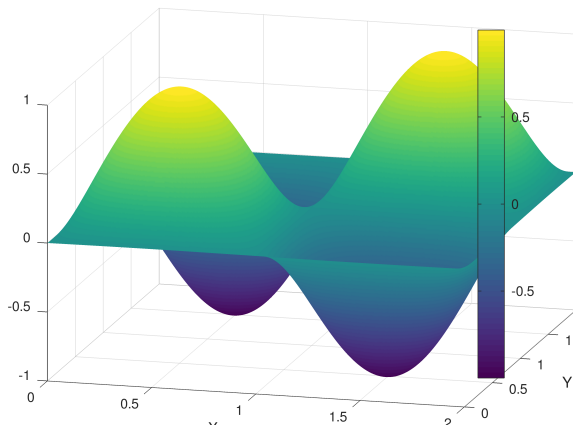
Problema 2

- **Grados polinomiales:** $q = p = 2$.
- **Dominio paramétrico:** $U = V = \{0, 0, 0, 1, 1, 1\}$.
- **Dominio físico:** Círculo contenido en el plano XY de radio 1 y centrado en el origen.
- **Ecuaciones del problema:** $f(x, y) = 1$, $u(\Gamma) = 0$.
- **Solución y derivadas analíticas:**
 - $u(x, y) = \frac{1-x^2-y^2}{4}$.
 - $\frac{\partial u}{\partial x}(x, y) = -\frac{x}{2}$.
 - $\frac{\partial u}{\partial y}(x, y) = -\frac{y}{2}$.
- **Puntos de control y pesos:**

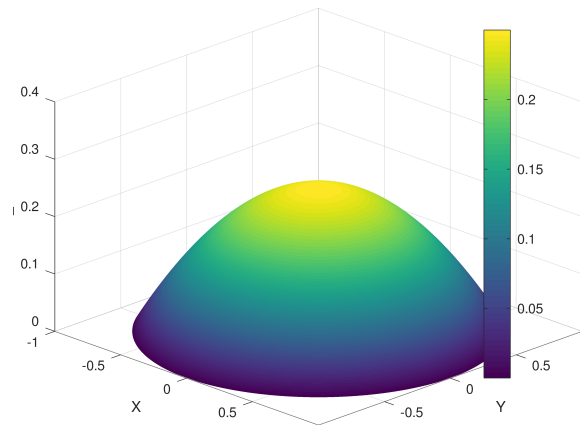
$$\mathbf{P} = \begin{pmatrix} (-1, 0, 0) & (-1, -1, 0) & (0, -1, 0) \\ (-1, 1, 0) & (0, 0, 0) & (1, -1, 0) \\ (0, 1, 0) & (1, 1, 0) & (1, 0, 0) \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} 1 & \frac{\sqrt{2}}{2} & 1 \\ \frac{\sqrt{2}}{2} & 1 & \frac{\sqrt{2}}{2} \\ 1 & \frac{\sqrt{2}}{2} & 1 \end{pmatrix}.$$

Cuadro 5.4: Tabla de convergencia y costes computacionales.

Nº Elementos	T. Ensamblado (s)	T. Resolución (s)	Error L^2	Error H^1
16×16	0,0068	0,0016	$1,42 \cdot 10^{-6}$	$1,21 \cdot 10^{-4}$
32×32	0,0084	0,0198	$1,74 \cdot 10^{-7}$	$3,02 \cdot 10^{-5}$
64×64	0,0663	0,0283	$2,17 \cdot 10^{-8}$	$7,53 \cdot 10^{-6}$
128×128	0,2076	0,1179	$2,71 \cdot 10^{-9}$	$1,88 \cdot 10^{-6}$
256×256	0,6902	0,9705	$3,42 \cdot 10^{-10}$	$4,70 \cdot 10^{-7}$



(a) Problema 1.



(b) Problema 2.

Nota: El eje Z, originalmente constante a 0 en ambas soluciones, se ha modificado para representar el valor de la solución en cada punto.

Capítulo 6

Conclusiones

Durante el trabajo hemos enfocado el IGA como alternativa al FEM, analizando sus bases teóricas, realizando una implementación de este en C++ y validando las soluciones obtenidas y el tiempo de computación.

A la vista del desarrollo teórico y de los resultados numéricos obtenidos en los capítulos previos, extraemos las siguientes conclusiones:

- **Exactitud Geométrica:** principal ventaja del IGA frente al FEM, elimina el error de aproximación geométrico y ahorra el desarrollo de una malla intermedia.
- **Alta Continuidad y Suavidad:** debido a las propiedades de las funciones base NURBS, obtenemos una suavidad de clase $C^{(p-1)}$, en contraposición al FEM, que ofrece suavidad de clase C^0 en las fronteras de los elementos.
- **Convergencia Numérica:** los resultados experimentales presentados en el Capítulo 5 confirman las tasas de convergencia en norma H^1 , decrece con orden $\mathcal{O}(h^p)$, y el error en norma L^2 , con orden $\mathcal{O}(h^{p+1})$.
- **Coste Computacional:** la fase de ensamblado del IGA es computacionalmente más costosa por grado de libertad que del FEM.
- **Implementación:** la implementación de programas que permitan resolver EDPs mediante IGA requieren de una gestión más detallada de la información geométrica.

6.1. Trabajo futuro

Debido a limitaciones de espacio y tiempo, hay temas que no se han tratado en el trabajo que tienen una alta importancia en referencia al IGA.

- **k-refinamiento:** refinamiento elevando el orden y la continuidad simultáneamente, una característica exclusiva de IGA que maximiza la eficiencia computacional.
- **Refinamiento local (T-Splines):** superar las limitaciones intrínsecas del producto tensorial de las NURBS implementando T-Splines, permitiendo así refinar de manera local un elemento concreto sin afectar a vecinos relativamente lejanos que mediante NURBS sería imposible, empleado en biotecnología para representar estructuras orgánicas complejas localmente.
- **Problemas dependientes del tiempo:** implementaciones discretizando el tiempo mediante métodos como Crank-Nicolson al igual que en el FEM, actualmente se encuentra en investigación la unificación de métodos que implementen únicamente IGA para problemas espacio-temporales.
- **Comparación de cotas respecto a FEM:** demostrar que, aun siendo más costoso computacionalmente, el IGA puede compensar esta desventaja al ofrecer una mejor cota del error por grado de libertad que el FEM.

Bibliografía

- [1] M.G. Larson and F. Bengzon. *The Finite Element Method: Theory, Implementation, and Applications*. Texts in Computational Science and Engineering. Springer Berlin Heidelberg, 2013. ISBN 978-3-642-33287-6. URL https://books.google.es/books?id=Vek_AAAAQBAJ.
- [2] H. Brezis. *Functional Analysis, Sobolev Spaces and Partial Differential Equations*. Universitext. Springer New York, 2010. ISBN 978-0-387-70913-0. URL <https://books.google.es/books?id=GAA2Xq0IIGoC>.
- [3] L.C. Evans. *Partial Differential Equations*. Graduate studies in mathematics. American Mathematical Society, 1991. URL <https://books.google.es/books?id=TF8EzQEACAAJ>.
- [4] L. Piegl and W. Tiller. *The NURBS Book*. Monographs in Visual Communication. Springer Berlin Heidelberg, 2012. ISBN 978-3-642-59223-2. URL <https://books.google.es/books?id=58KqCAAAQBAJ>.
- [5] J.A. Cottrell, T.J.R. Hughes, and Y. Bazilevs. *Isogeometric Analysis: Toward Integration of CAD and FEA*. Wiley, 2009. ISBN 978-0-470-74873-2. URL <https://books.google.es/books?id=SfOUdAEACAAJ>.
- [6] G.E. Farin. *Curves and Surfaces for CAGD: A Practical Guide*. Computer graphics and geometric modeling. Elsevier Science, 2002. ISBN 978-1-55860-737-8. URL <https://books.google.es/books?id=5HYTP1dIAp4C>.
- [7] J.H. Ahlberg, E.N. Nilson, J.L. Walsh, and R. Bellman. *The Theory of Splines and Their Applications: Mathematics in Science and Engineering: A Series of Monographs and Textbooks, Vol. 38*. Number v. 38. Academic Press, 2016. ISBN 978-1-4832-2295-0. URL <https://books.google.es/books?id=3bZ1DAAAQBAJ>.
- [8] Ulrich Langer, Stephen E Moore, and Martin Neumüller. Space-time isogeometric analysis of parabolic evolution equations. *Computer Methods in Applied Mechanics and Engineering*, 306: 342–363, 2016. doi: 10.1016/j.cma.2016.03.042. Publisher: Elsevier.
- [9] Y. Bazilevs, V.M. Calo, J.A. Cottrell, J.A. Evans, T.J.R. Hughes, S. Lipton, M.A. Scott, and T.W. Sederberg. Isogeometric analysis using T-splines. *Computer Methods in Applied Mechanics and Engineering*, 199(5-8):229–263, 2010. doi: 10.1016/j.cma.2009.02.036. Publisher: Elsevier.
- [10] David J. Benson, Yuri Bazilevs, Ming-Chen Hsu, and Thomas J.R. Hughes. A large deformation, rotation-free, isogeometric shell. *Computer Methods in Applied Mechanics and Engineering*, 200(13-16):1367–1378, 2011. doi: 10.1016/j.cma.2010.12.003. Publisher: Elsevier.
- [11] Yuri Bazilevs, Jeffrey Gohean, Thomas Hughes, R. Moser, and Yongjie Zhang. Patient-specific isogeometric fluid-structure interaction analysis of thoracic aortic blood flow due to implantation of the Jarvik 2000 left ventricular assist device. *Computer Methods in Applied Mechanics and Engineering*, 198:3534–3550, September 2009. doi: 10.1016/j.cma.2009.04.015.

- [12] Josef Kiendl, Kai-Uwe Bletzinger, J. Linhard, and Roland Wüchner. Isogeometric shell analysis with Kirchhoff-Love elements. *Computer Methods in Applied Mechanics and Engineering - COMPUT METHOD APPL MECH ENG*, 198:3902–3914, November 2009. doi: 10.1016/j.cma.2009.08.013.
- [13] Eric Brechner. Developing and Using CAD/CAM/CAE Systems in Boeing. *IEEE Annals of the History of Computing*, 46(1):38–49, January 2024. ISSN 1058-6180. doi: 10.1109/MAHC.2024.3468340.
- [14] R.A. Liming. *Practical Analytic Geometry with Applications to Aircraft*. Macmillan, 1944. ISBN 978-0-598-45557-4. URL <https://books.google.es/books?id=x99EAAAAAAAJ>.
- [15] K.J. Versprille. *Computer Aided Design Applications of the Rational B-spline Approximation Form*. University Microfilms, 1975. URL <https://books.google.es/books?id=D0wvjwEACAAJ>.
- [16] Dalton Fazanaro, Paulo Amorim, Thiago Franco de Moraes, Jorge Silva, and Helio Pedrini. NURBS Parameterization for Medical Surface Reconstruction. *Applied Mathematics*, 07:137–144, January 2016. doi: 10.4236/am.2016.72013.
- [17] Gerald Farin. A History of Curves and Surfaces in CAGD. *Computer Aided Geometric Design - CAGD*, December 2002. doi: 10.1016/B978-044451104-1/50002-2.
- [18] T. J. R. Hughes, J. A. Cottrell, and Y. Bazilevs. Isogeometric analysis: CAD, finite elements, NURBS, exact geometry and mesh refinement. *Computer Methods in Applied Mechanics and Engineering*, 194(39):4135–4195, 2005. ISSN 0045-7825. doi: <https://doi.org/10.1016/j.cma.2004.10.008>. URL <https://www.sciencedirect.com/science/article/pii/S0045782504005171>.

Anexos

A.1. Método de los elementos finitos.

Partiendo de la formulación variacional, con la garantía de la existencia y unicidad de la solución $u \in V(\Omega) \subset H^1(\Omega)$, en el FEM buscamos una solución aproximada u^h de esta en un espacio de dimensión finita.

Para ello se realizan dos pasos previos:

- **Discretización del dominio (Ω):** este se divide en una colección de subdominios principales denominados **elementos**, estos elementos son politopos cerrados y simples (en 2D normalmente son triángulos, pero pueden ser también otro tipo de polígonos). Los denotaremos como K_j , de manera que se cumple que $\Omega \cup \Gamma = \overline{\Omega} = \bigcup_{j=1}^M K_j$. Esta subdivisión se conoce como malla ($T_h = \{K_1, K_2, \dots, K_M\}$).
- **Discretización del espacio ($V(\Omega)$):** el subespacio de dimensión finita construido a base de funciones polinómicas $V^h = \{v \in H^1(\Omega) \mid v = g \text{ en } \Gamma_D \text{ y } v|_{K_j} \in P_p(K_j), \forall K_j \in T_h\}$, donde $p \in \mathbb{N}$ es el grado polinomial elegido y $h = \max_{K_j \in \mathcal{T}_h} h_j$, $h_j = \text{diam}(K_j) = \sup_{x,y \in K_j} \|x - y\|$, de manera que podemos definir una base de este de la forma $\{\phi_1, \phi_2, \dots, \phi_N\}$.

Definimos adicionalmente los nodos $x_i \in \overline{\Omega}$ y el conjunto nodal $\mathcal{N} = \{x_1, x_2, \dots, x_N\} \subset \overline{\Omega}$, donde N es la dimensión de V^h . El lugar que ocupa cada x_i depende tanto del tipo de elementos utilizados para crear la malla como del grado polinomial p elegido, además, cada uno de estos está asociado a una única función base, de manera que se cumplen las siguientes propiedades:

- $\phi_i(x_j) = \delta_{ij} = \begin{cases} 1 & \text{si } i = j \\ 0 & \text{si } i \neq j \end{cases}$
- $\text{supp}(\phi_i) = \overline{\bigcup \{K_j \in T_h \mid x_i \in K_j\}}$

Tras esto definimos la solución aproximada y su gradiente como:

$$u^h(x) = \sum_{i=1}^N U_i \phi_i(x), \quad U_i \in \mathbb{R}, \quad \forall i = 1, 2, \dots, N.$$

$$\nabla \mathbf{u}^h(\mathbf{x}) = \nabla \left(\sum_{i=1}^N U_i \phi_i(\mathbf{x}) \right) = \sum_{i=1}^N U_i \nabla \phi_i(\mathbf{x}).$$

Luego el problema se resume en encontrar $u \in v^h$ tal que $a(u^h, v^h) - L(v^h) = 0, \forall v^h \in V^h$. sustituyendo y aplicando la **linealidad de la forma bilineal**¹ y de la **forma lineal**² nos queda:

$$\sum_{i=1}^N V_i \left[\sum_{j=1}^N a(\phi_j, \phi_i) U_j - L(\phi_i) \right] = 0, \quad \forall V_i \in \mathbb{R}, \quad \forall i = 1, 2, \dots, N.$$

Finalizando por tanto con el sistema dado por $\sum_{j=1}^N a(\phi_j, \phi_i) U_j = L(\phi_i), \quad \forall i = 1, 2, \dots, N$.

¹ $a(\sum_{j=1}^N U_j \phi_j, \sum_{i=1}^N U_i \phi_i) = \sum_{i=1}^N V_i \sum_{j=1}^N U_j a(\phi_j, \phi_i)$.

² $L(\sum_{i=1}^N V_i \phi_i) = \sum_{i=1}^N V_i L(\phi_i)$.

Definición. Los principales elementos del sistema que se resolverá para obtener la solución son:

- **La matriz de rigidez $\mathbf{A}$:** $A_{ij} = a(\phi_j, \phi_i) = \int_{\Omega} \nabla \phi_j \cdot \nabla \phi_i d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} \phi_j \phi_i d\Gamma$.
- **El vector de carga $\mathbf{F}$:** $F_i = L(\phi_i) = \int_{\Omega} f \phi_i d\Omega + \int_{\Gamma_N} h \phi_i d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} \phi_i d\Gamma$.

³ Finalizando con el sistema $AU = F$, donde $U = (U_1, U_2, \dots, U_N)^T$, adicionalmente $u^h(x_i) = U_i$.

Definición. Definimos ahora la norma inducida por la forma bilineal $a(\cdot, \cdot)$, que denotaremos por $\|\cdot\|_V$:

$$\|v\|_V^2 = a(v, v) = \int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma.$$

Lema A.1.1. Las normas $\|\cdot\|_V$ y $\|\cdot\|_{H^1(\Omega)}$ son equivalentes, es decir, existen $C_1, C_2 \in \mathbb{R}$ tal que:

$$C_1 \|v\|_{H^1(\Omega)}^2 \leq \|v\|_V^2 \leq C_2 \|v\|_{H^1(\Omega)}^2.$$

Demostración omitida. La demostración puede consultarse en el Anexo A.2.4.

Teorema A.1. El error $e = u - u^h$ del FEM converge en las normas $\|\cdot\|_{L^2(\Omega)}$ y $\|\cdot\|_{H^1(\Omega)}$, tal que:

- $\|e\|_{H^1(\Omega)}$, se reduce con orden $\mathcal{O}(h^p)$.
- $\|e\|_{L^2(\Omega)}$, se reduce con orden $\mathcal{O}(h^{p+1})$.

Siempre que $u \in H^{p+1}(\Omega)$.

Demostración. Separaremos la demostración en cada una de las normas:

- **Convergencia en $\|\cdot\|_{H^1(\Omega)}$:**

Asumiendo la coercividad y continuidad de la forma bilineal en $V(\Omega)^4$, nos encontramos en las condiciones del **Lema de Céa** por tanto se cumple:

$$\|u - u_h\|_V \leq \frac{C_{cont}}{C_{coer}} \inf_{v_h \in V_h} \|u - v_h\|_V,$$

donde C_{cont} es la constante de continuidad y C_{coer} es la constante de coercividad.

Aplicando ahora la equivalencia entre normas $\|\cdot\|_V$ y $\|\cdot\|_{H^1(\Omega)}$ obtenemos que $\exists C_{Céa} \in \mathbb{R}^+$ tal que:

$$\|u - u_h\|_{H^1(\Omega)} \leq C_{Céa} \inf_{v_h \in V_h} \|u - v_h\|_{H^1(\Omega)}. \quad (\text{A.1})$$

Por último usaremos la **propiedad de aproximación del espacio de Sobolev**, la cual nos garantiza que $\exists C_{Sob} \in \mathbb{R}^+$ tal que:

$$\inf_{v_h \in V_h} \|u - v_h\|_{H^1(\Omega)} \leq C_{Sob} h^p \|u\|_{H^{p+1}(\Omega)}. \quad (\text{A.2})$$

Finalmente uniendo todo y definiendo $C_{Cota} = C_{Céa} C_{Sob}$ tenemos:

$$\|u - u_h\|_{H^1(\Omega)} \leq C_{Cota} h^p,$$

obteniendo así un error en $H^1(\Omega)$ que converge con orden $\mathcal{O}(h^p)$, siempre que $u \in H^{p+1}(\Omega)$.

³ Si se desea conocer más sobre algoritmos de ensamblado de $\mathbf{A}$ y $\mathbf{F}$, consúltese [1], especialmente los capítulos 4 y 5.

⁴ La demostración de estas puede ser encontrada en el anexo A.2.5

■ **Convergencia en $\|\cdot\|_{L^2(\Omega)}$:**

Para la cota del error $e = u - u_h$ en $L^2(\Omega)$ consideramos el problema dual asociado:

Encontrar $w \in V(\Omega)$ tal que:

$$a(v, w) = (e, v)_{L^2(\Omega)} = \int_{\Omega} e v d\Omega, \quad \forall v \in V(\Omega).$$

Debido a la coercividad y continuidad de la forma bilineal y la continuidad de la forma lineal⁵, debido al **Lema de Lax-Milgram**, la solución w existe y es única, su regularidad viene dada por las condiciones de regularidad elípticas asumidas en el anexo A.2.3, luego $\exists C_{\text{Reg}} \in \mathbb{R}^+$ tal que:

$$\|w\|_{H^2(\Omega)} \leq C_{\text{Reg}} \|e\|_{L^2(\Omega)} \quad (\text{A.3})$$

Tomamos ahora $v = e$, obteniendo así la identidad

$$a(e, w) = (e, e)_{L^2(\Omega)} = \|e\|_{L^2(\Omega)}^2.$$

Como la solución discreta u_h satisface la **propiedad de ortogonalidad de Galerkin**:

$$a(u - u_h, v_h) = a(e, v_h) = 0, \quad \forall v_h \in V_h.$$

Nos permite introducir cualquier función interpolante $w_h \in V_h$ tal que se cumple la siguiente igualdad:

$$\|e\|_{L^2(\Omega)}^2 = a(e, w) - a(e, w_h) = a(e, w - w_h).$$

Aplicando ahora la **continuidad de la forma bilineal**,

$$\|e\|_{L^2(\Omega)}^2 \leq C_{\text{cont}} \|e\|_{H^1(\Omega)} \|w - w_h\|_{H^1(\Omega)}.$$

Usamos ahora el resultado de convergencia para $\|e\|_{H^1(\Omega)}$ dadas por A.1 y A.2

$$\|e\|_{L^2(\Omega)}^2 \leq C_{\text{cont}} C_{\text{Céa}} C_{\text{Sob}} h^p \|u\|_{H^{p+1}(\Omega)} \|w - w_h\|_{H^1(\Omega)}.$$

Terminamos utilizando la **propiedad de aproximación de los espacios de Sobolev**, y tomando que $w \in H^2(\Omega)$ debido a la regularidad elíptica tenemos:

$$\|w - w_h\|_{H^1(\Omega)} \leq C_2 h^{2-1} \|w\|_{H^2(\Omega)} = C_2 h^1 \|w\|_{H^2(\Omega)}.$$

Obteniendo en la desigualdad:

$$\|e\|_{L^2(\Omega)}^2 \leq \left(C_{\text{cont}} C_{\text{Céa}} C_{\text{Sob}} C_2 \|u\|_{H^{p+1}(\Omega)} \|w\|_{H^2(\Omega)} \right) h^{p+1}.$$

Aplicando finalmente A.3, y dividiendo en ambos lados de la igualdad por $\|e\|_{L^2(\Omega)}$ ⁶:

$$\|e\|_{L^2(\Omega)} \leq \left(C_{\text{reg}} C_{\text{cont}} C_{\text{Céa}} C_{\text{Sob}} C_2 \|u\|_{H^{p+1}(\Omega)} \right) h^{p+1}.$$

obteniendo así un error en $L^2(\Omega)$ que converge con orden $\mathcal{O}(h^{p+1})$, siempre que $u \in H^{p+1}(\Omega)$.

□

⁵La demostración de la continuidad de la forma lineal se encuentra en el anexo A.2.6

⁶Notar que si $\|e\|_{L^2(\Omega)} = 0$, $\|e\|_{L^2(\Omega)} \leq C h^{p+1}$, $\forall C, h > 0$, luego asumimos $\|e\|_{L^2(\Omega)} > 0$.

A.2. Demostraciones omitidas

A.2.1. Paso de Solución Fuerte a Solución Variacional

Demostración. Recurriremos al **Principio de Galerkin**, el cual nos garantiza que:

$$\int_{\Omega} (-\Delta u - f)v d\Omega = 0 \quad \text{para todo } v \in V_0(\Omega).$$

Reordenando y utilizando la identidad $\Delta u \cdot v = \nabla \cdot (\nabla u \cdot v) - \nabla u \cdot \nabla v$:

$$\int_{\Omega} (-\Delta u)v d\Omega = \int_{\Omega} -[\nabla \cdot (\nabla u \cdot v) - \nabla u \cdot \nabla v] d\Omega = \int_{\Omega} \nabla u \cdot \nabla v d\Omega - \int_{\Omega} \nabla \cdot (\nabla u \cdot v) d\Omega. \quad (\text{A.4})$$

Al aplicar el **Teorema de Gauss-Ostrogradsky** al último término:

$$\int_{\Omega} \nabla \cdot (\nabla u \cdot v) d\Omega = \int_{\Gamma} (\nabla u \cdot \mathbf{n})v d\Gamma.$$

Sustituyendo esta conversión en la ecuación (A.5) obtenemos la **Formulación variacional preliminar**:

$$\int_{\Omega} \nabla u \cdot \nabla v d\Omega - \int_{\Gamma} (\nabla u \cdot \mathbf{n})v d\Gamma = \int_{\Omega} f v d\Omega.$$

Separando ahora Γ en sus componentes disjuntas:

$$\int_{\Gamma} (\nabla u \cdot \mathbf{n})v d\Gamma = \int_{\Gamma_D} (\nabla u \cdot \mathbf{n})v d\Gamma + \int_{\Gamma_N} (\nabla u \cdot \mathbf{n})v d\Gamma + \int_{\Gamma_R} (\nabla u \cdot \mathbf{n})v d\Gamma.$$

Utilizamos ahora $v \in V_0(\Omega)$ y las condiciones de contorno:

$$\int_{\Gamma} (\nabla u \cdot \mathbf{n})v d\Gamma = 0 + \int_{\Gamma_N} h v d\Gamma + \int_{\Gamma_R} \left(\frac{r - \alpha u}{\beta} \right) v d\Gamma.$$

Por último sustituimos y reordenamos en forma bilineal $a(u, v)$ igualada a la forma lineal $L(v)$, obteniendo así:

$$\begin{aligned} a(u, v) &= \int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} u v d\Gamma = \\ &= \int_{\Omega} f v d\Omega + \int_{\Gamma_N} h v d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma = L(v). \end{aligned}$$

□

A.2.2. Existencia y unicidad de la solución variacional (lema de Lax-Milgram)

Teorema A.2. *La existencia y unicidad de la solución al problema variacional $u \in V(\Omega)$ está garantizada gracias al **Lema de Lax-Milgram**, pues se cumplen las tres siguientes condiciones de este:*

- **Continuidad de la Forma Bilineal:** $|a(u, v)| \leq (1 + C_{Tr}^2 \left| \frac{\alpha}{\beta} \right|) \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$
- **Coercividad de la Forma Bilineal:** $a(v, v) \geq \|\nabla v\|_{L^2(\Omega)}^2 \geq \frac{1}{C_P + 1} \|v\|_{H^1(\Omega)}^2.$
- **Continuidad de la Forma Lineal:** $|L(v)| \leq C \|v\|_{H^1(\Omega)}, \quad C \in \mathbb{R}^+.$

Demostraremos ahora cada una de las propiedades:

- **Continuidad de la Forma Bilineal:**

Demostración. aplicaremos la **desigualdad triangular**,

$$|a(u, v)| = \left| \int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma \right| \leq \left| \int_{\Omega} \nabla u \cdot \nabla v d\Omega \right| + \left| \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma \right|.$$

Continuaremos con la **desigualdad de Cauchy-Schwartz** para el primer término,

$$\left| \int_{\Omega} \nabla u \cdot \nabla v d\Omega \right| \leq \int_{\Omega} |\nabla u| |\nabla v| d\Omega \leq \|\nabla u\|_{L^2(\Omega)} \|\nabla v\|_{L^2(\Omega)} \leq \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$$

Buscamos ahora acotar el segundo término utilizando primero la **desigualdad de Cauchy-Schwartz**:

$$\left| \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma \right| \leq \left| \frac{\alpha}{\beta} \right| \int_{\Gamma_R} |u| |v| d\Gamma \leq \left| \frac{\alpha}{\beta} \right| \|u\|_{L^2(\Gamma_R)} \|v\|_{L^2(\Gamma_R)}.$$

Usamos ahora el **teorema de la traza** que nos garantiza la existencia de una constante C_{Tr} tal que,

$$\left| \frac{\alpha}{\beta} \right| \|u\|_{L^2(\Gamma_R)} \|v\|_{L^2(\Gamma_R)} \leq C_{Tr}^2 \left| \frac{\alpha}{\beta} \right| \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$$

Por lo que obtenemos:

$$|a(u, v)| \leq (1 + C_{Tr}^2 \left| \frac{\alpha}{\beta} \right|) \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$$

□

■ Coercividad de la Forma Bilineal:

Demostración. partimos de la condición impuesta $\frac{\alpha}{\beta} \geq 0$:

$$a(v, v) = \int_{\Omega} \nabla v \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} v^2 d\Gamma = \int_{\Omega} |\nabla v|^2 d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} v^2 d\Gamma \geq \int_{\Omega} |\nabla v|^2 d\Omega = \|\nabla v\|_{L^2(\Omega)}^2.$$

Partiendo de que Ω es acotado y $v \in V_0(\Omega)$ aplicamos la **desigualdad de Poincaré** la cual nos dice:

$$\text{Existe } C_P > 0 \text{ tal que: } \|v\|_{L^2(\Omega)}^2 \leq C_P \|\nabla v\|_{L^2(\Omega)}^2.$$

Recordamos ahora la definición de la norma H^1 y aplicamos la **desigualdad de Poincaré**:

$$\|v\|_{H^1(\Omega)}^2 = \|v\|_{L^2(\Omega)}^2 + \|\nabla v\|_{L^2(\Omega)}^2 \leq (C_P + 1) \|\nabla v\|_{L^2(\Omega)}^2.$$

Dividiendo ambos lados por $(C_P + 1)$ y sustituyendo en la desigualdad de la forma bilinear anterior obtenemos:

$$a(v, v) \geq \|\nabla v\|_{L^2(\Omega)}^2 \geq \frac{1}{C_P + 1} \|v\|_{H^1(\Omega)}^2.$$

□

■ Continuidad de la Forma Lineal:

Demostración. partimos de definición de $L(v)$ y aplicamos la **desigualdad triangular**,

$$|L(v)| \leq \left| \int_{\Omega} f v d\Omega \right| + \left| \int_{\Gamma_N} h v d\Gamma \right| + \left| \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma \right|.$$

Acotaremos ahora término a término,

$$\left| \int_{\Omega} f v d\Omega \right| \leq \|f\|_{L^2(\Omega)} \|v\|_{L^2(\Omega)} \leq \|f\|_{L^2(\Omega)} \|v\|_{H^1(\Omega)}.$$

$$\left| \int_{\Gamma_N} h v d\Gamma \right| \leq \|h\|_{L^2(\Gamma_N)} \|v\|_{L^2(\Gamma_N)} \leq \|h\|_{L^2(\Gamma_N)} C_{Tr} \|v\|_{H^1(\Omega)}, \text{ (Teorema de la traza).}$$

$$\left| \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma \right| \leq \left| \frac{1}{\beta} \right| \|r\|_{L^2(\Gamma_R)} \|v\|_{L^2(\Gamma_R)} \leq \left| \frac{1}{\beta} \right| \|r\|_{L^2(\Gamma_R)} C_{Tr} \|v\|_{H^1(\Omega)}, \text{ (Teorema de la traza).}$$

Finalmente, la continuidad se demuestra al acotar:

$$|L(v)| \leq \left(\|f\|_{L^2(\Omega)} + \|h\|_{L^2(\Gamma_N)} C_{Tr} + \left| \frac{1}{\beta} \right| \|r\|_{L^2(\Gamma_R)} C_{Tr} \right) \|v\|_{H^1(\Omega)}.$$

□

El cumplimiento de estas condiciones para el operador Laplaciano asegura que el problema variacional está **bien planteado**.

A.2.3. Equivalencia de la solución de la formulación variacional y fuerte

Demostración. Asumimos $\Gamma \in C^2$, $f \in L^2(\Omega)$, $g \in H^{3/2}(\Gamma_D)$, $h \in H^{1/2}(\Gamma_N)$, $r \in H^{1/2}(\Gamma_R)$ y $u \in H^2(\Omega)$, recordamos ahora la formulación variacional:

$$\int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} u v d\Gamma = a(u, v) = L(v) = \int_{\Omega} f v d\Omega + \int_{\Gamma_N} h v d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma. \quad (\text{A.5})$$

Como $u \in H^2(\Omega)$ podemos aplicar el teorema de divergencia inverso al primer término de la forma bilineal:

$$\int_{\Omega} \nabla u \cdot \nabla v d\Omega = - \int_{\Omega} (\Delta u) v d\Omega + \int_{\Gamma} (\nabla u \cdot \mathbf{n}) v d\Gamma.$$

Sustituyendo ahora en (A.5) y agrupando todo en un lado de la igualdad obtenemos:

$$- \int_{\Omega} (\Delta u) v d\Omega + \int_{\Gamma} (\nabla u \cdot \mathbf{n}) v d\Gamma + \int_{\Gamma_R} \frac{\alpha}{\beta} u v d\Gamma - \int_{\Omega} f v d\Omega - \int_{\Gamma_N} h v d\Gamma - \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma = 0.$$

Separamos Γ en sus componentes disjuntas Γ_N , Γ_D , Γ_R y como $v \in V_0(\Omega)$, $v = 0$ en Γ_D luego:

$$\int_{\Gamma} (\nabla u \cdot \mathbf{n}) v d\Gamma = \int_{\Gamma_N} (\nabla u \cdot \mathbf{n}) v d\Gamma + \int_{\Gamma_R} (\nabla u \cdot \mathbf{n}) v d\Gamma.$$

Reagrupamos ahora los términos que se integran en las mismas regiones:

$$\int_{\Omega} (-\Delta u - f) v d\Omega + \int_{\Gamma_N} (\nabla u \cdot \mathbf{n} - h) v d\Gamma + \int_{\Gamma_R} (\nabla u \cdot \mathbf{n} + \frac{\alpha}{\beta} u - \frac{r}{\beta}) v d\Gamma = 0.$$

Aplicando ahora el **teorema fundamental del cálculo de variaciones** tenemos que todos los elementos entre paréntesis equivalen a 0 luego:

$$\begin{cases} -\Delta u = f & \text{en } \Omega, \\ u = g & \text{en } \Gamma_D \quad (u \in V(\Omega)), \\ \nabla u \cdot \mathbf{n} = h & \text{en } \Gamma_N, \\ \alpha u + \beta \nabla u \cdot \mathbf{n} = r & \text{en } \Gamma_R. \end{cases}$$

□

A.2.4. Equivalencia de normas

Demostración. Demostraremos ambas cotas por separado:

- **Acotación superior** ($\|v\|_V^2 \leq C_2 \|v\|_{H^1(\Omega)}^2$), iniciamos recordando la definición de cada norma:

- $\|v\|_V^2 = \int_{\Omega} |\nabla v|^2 d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} |v|^2 d\Gamma$
- $\|v\|_{H^1(\Omega)}^2 = \int_{\Omega} |\nabla v|^2 d\Omega + \int_{\Omega} |v|^2 d\Omega$

Aplicaremos ahora la **desigualdad de la traza** obteniendo:

$$\int_{\Gamma_R} |v|^2 d\Gamma \leq C_{Tr} \|v\|_{H^1(\Omega)}^2.$$

Usaremos también la desigualdad trivial:

$$\int_{\Omega} |\nabla v|^2 d\Omega \leq \|v\|_{H^1(\Omega)}^2.$$

Obteniendo finalmente:

$$\|v\|_V^2 \leq \|v\|_{H^1(\Omega)}^2 + C_{Tr} \|v\|_{H^1(\Omega)}^2 = (1 + C_{Tr}) \|v\|_{H^1(\Omega)}^2.$$

Luego tomaremos $C_2 = 1 + C_{Tr}$.

- **Acotación inferior** ($C_1 \|v\|_{H^1(\Omega)}^2 \leq \|v\|_V^2$), partimos de la **desigualdad de Poincaré generalizado**, el cual nos dice que existe $C_P \in \mathbb{R}^+$ tal que:

$$\int_{\Omega} |v|^2 d\Omega \leq C_P \left(\int_{\Omega} |\nabla v|^2 d\Omega + \int_{\Gamma_R} |v|^2 d\Gamma \right)$$

Sustituyendo ahora en $\|v\|_{H^1(\Omega)}$:

$$\|v\|_{H^1(\Omega)}^2 \leq \int_{\Omega} |\nabla v|^2 d\Omega + C_P \left(\int_{\Omega} |\nabla v|^2 d\Omega + \int_{\Gamma_R} |v|^2 d\Gamma \right) = (1 + C_P) \int_{\Omega} |\nabla v|^2 d\Omega + C_P \int_{\Gamma_R} |v|^2 d\Gamma.$$

Buscamos ahora una constante C_R tal que $C_R \cdot \|v\|_V^2 \geq \|v\|_{H^1(\Omega)}^2$ y cumpla las siguientes propiedades:

- $C_R \geq (1 + C_P) \implies C_R \geq 1 + C_P.$
- $C_R \cdot \frac{\alpha}{\beta} \geq C_P \implies C_R \geq C_P \cdot \frac{\beta}{\alpha}$

Luego tomamos:

$$C_R = \max \left(1 + C_P, \frac{\beta C_P}{\alpha} \right).$$

Obteniendo así la siguiente desigualdad:

$$C_R \|v\|_V^2 = C_R \int_{\Omega} |\nabla v|^2 d\Omega + C_R \frac{\alpha}{\beta} \int_{\Gamma_R} |v|^2 d\Gamma \geq (1 + C_P) \int_{\Omega} |\nabla v|^2 d\Omega + C_P \int_{\Gamma_R} |v|^2 d\Gamma.$$

Tomando finalmente $C_1 = \frac{1}{C_R}$:

$$\|v\|_V^2 \geq C_1 \|v\|_{H^1(\Omega)}^2.$$

□

A.2.5. Condiciones del lema de Céa

Demostración. Probaremos la coercividad y la continuidad en $\|\cdot\|_V$:

- **Coercividad:** $\exists C_1 \in \mathbb{R}^+$ tal que $C_1 \|v\|_{H^1(\Omega)}^2 \leq \|v\|_V^2$, Probado en la equivalencia de normas.
- **Continuidad:** $\exists C_2 \in \mathbb{R}^+$ tal que $|a(u, v)| \leq C_2 \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}$: Iniciamos aplicando la **desigualdad triangular** a la definición de $|a(u, v)|$:

$$|a(u, v)| \leq \left| \int_{\Omega} \nabla u \cdot \nabla v d\Omega \right| + \left| \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma \right|.$$

Aplicamos ahora la **desigualdad de Cauchy-Schwartz** en $L^2(\Omega)$ al primer término:

$$\left| \int_{\Omega} \nabla u \cdot \nabla v d\Omega \right| \leq \int_{\Omega} |\nabla u| |\nabla v| d\Omega \leq \|\nabla u\|_{L^2(\Omega)} \|\nabla v\|_{L^2(\Omega)}.$$

Usamos ahora la propiedad trivial $\|v\|_{H^1(\Omega)}^2 \geq \|\nabla v\|_{L^2(\Omega)}^2, \forall v \in H^1(\Omega)$,

$$|a(u, v)| \leq \|\nabla u\|_{L^2(\Omega)} \|\nabla v\|_{L^2(\Omega)} \leq \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$$

Pasamos ahora al segundo término e empezamos aplicando la **desigualdad de Cauchy-Schwartz**:

$$\left| \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma \right| \leq \frac{\alpha}{\beta} \int_{\Gamma_R} |u| |v| d\Gamma \leq \frac{\alpha}{\beta} \|u\|_{L^2(\Gamma_R)} \|v\|_{L^2(\Gamma_R)}.$$

Seguiremos aplicando ahora la **desigualdad de la traza**:

$$\|u\|_{L^2(\Gamma_R)} \|v\|_{L^2(\Gamma_R)} \leq C_{Tr}^2 \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$$

Luego uniendo ambas cotas de los términos individuales obtenemos:

$$|a(u, v)| \leq \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)} + \frac{\alpha}{\beta} C_{Tr}^2 \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)} = (1 + \frac{\alpha}{\beta} C_{Tr}^2) \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$$

Por último tomamos $C_2 = 1 + \frac{\alpha}{\beta} C_{Tr}^2$.

□

A.2.6. Continuidad de la forma lineal dual

Demostración. Recordamos que la forma lineal del problema dual es $L_{\text{dual}}(v) = (e, v)_{L^2(\Omega)}$.

Iniciamos usando la **desigualdad de Cauchy-Schwartz** en $L^2(\Omega)$,

$$|(e, v)_{L^2(\Omega)}| \leq \|e\|_{L^2(\Omega)} \|v\|_{L^2(\Omega)}.$$

Aplicamos ahora la **propiedad trivial** de la acotación superior $\|\cdot\|_{L^2(\Omega)}$ por $\|\cdot\|_{H^1(\Omega)}$,

$$|(e, v)_{L^2(\Omega)}| \leq \|e\|_{L^2(\Omega)} \|v\|_{H^1(\Omega)}.$$

Como $e = u - u^h$ es constante en este contexto, $e \in L^2(\Omega)$, luego tomamos $C_L = \|e\|_{L^2(\Omega)}$.

□

A.2.7. Propiedades de Bezier

Funciones base

Demostración. Se demostrarán las propiedades en el orden en el que han sido establecidas:

1. **No negatividad:** dado que $\binom{p}{i} \geq 0$ y $u \in [0, 1]$ implica que $u^i \geq 0$ y $(1-u)^{p-i} \geq 0$, obtenemos la condición.
2. **Partición de la unidad:** mediante el **Teorema del Binomio de Newton**, que establece que para cualesquiera variables x, y :

$$(x+y)^p = \sum_{i=0}^p \binom{p}{i} x^i y^{p-i}.$$

Si sustituimos $x = u$ y $y = 1 - u$, obtenemos:

$$\sum_{i=0}^p B_{i,p}(u) = \sum_{i=0}^p \binom{p}{i} u^i (1-u)^{p-i} = (u + (1-u))^p = (1)^p = 1.$$

3. **Interpolación de los extremos:** evaluamos en los límites $u = 0$ y $u = 1$. Tomando $0^0 = 1$.

- Para $u = 0$:

$$B_{i,p}(0) = \binom{p}{i} 0^i (1)^{p-i}.$$

Si $i \neq 0$, el término $0^i = 0$, anulando la expresión. El único término no nulo es $i = 0$:

$$B_{0,p}(0) = \binom{p}{0} 0^0 (1)^p = 1 \cdot 1 \cdot 1 = 1.$$

- Para $u = 1$:

$$B_{i,p}(1) = \binom{p}{i} 1^i (0)^{p-i}.$$

Si $i \neq p$, el término $(0)^{p-i} = 0$ (pues $p-i > 0$). El único término no nulo es $i = p$:

$$B_{p,p}(1) = \binom{p}{p} 1^p (0)^0 = 1 \cdot 1 \cdot 1 = 1.$$

4. **Soporte global:** analizamos el intervalo abierto $u \in (0, 1)$. En este intervalo, $u > 0$ y $(1-u) > 0$. Consecuentemente, $u^i > 0$ y $(1-u)^{p-i} > 0$ ($\forall i \geq 0$). Luego obtenemos

$$B_{i,p}(u) > 0, \quad \forall i \geq 0, \quad u \in (0, 1).$$

□

Curvas

Demostración. Se demostrarán las propiedades en el orden en el que han sido establecidas:

1. **Interpolación en los extremos:** de las propiedades de las funciones base de Bezier obtenemos que para $u = 0$ se tiene

$$\mathbf{C}(0) = \sum_{i=0}^p B_{i,p}(0) \mathbf{P}_i = 1 \cdot \mathbf{P}_0.$$

Para el caso $u = 1$ tenemos

$$\mathbf{C}(1) = \sum_{i=0}^p B_{i,p}(1) \mathbf{P}_i = 1 \cdot \mathbf{P}_p.$$

2. **Tangencia en los extremos:** partiremos de la derivada de las funciones base,

$$\frac{d}{du}B_{i,p}(u) = p(B_{i-1,p-1}(u) - B_{i,p-1}(u)).$$

Sustituyendo esto en la derivada de la curva:

$$\mathbf{C}'(u) = \sum_{i=0}^p p(B_{i-1,p-1}(u) - B_{i,p-1}(u))\mathbf{P}_i.$$

Reorganizando el sumatorio obtenemos

$$\mathbf{C}'(u) = p \sum_{i=0}^{p-1} B_{i,p-1}(u)(\mathbf{P}_{i+1} - \mathbf{P}_i).$$

Ahora evaluamos en los extremos usando la propiedad de interpolación de las funciones base:

■ En $u = 0$:

$$\mathbf{C}'(0) = p \cdot B_{0,p-1}(0)(\mathbf{P}_1 - \mathbf{P}_0) = p(\mathbf{P}_1 - \mathbf{P}_0).$$

■ En $u = 1$:

$$\mathbf{C}'(1) = p \cdot B_{p-1,p-1}(1)(\mathbf{P}_p - \mathbf{P}_{p-1}) = p(\mathbf{P}_p - \mathbf{P}_{p-1}).$$

3. **Invarianza afín:** sea $\Phi(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$ una transformación afín. Aplicamos Φ a la curva:

$$\Phi(\mathbf{C}(u)) = \mathbf{A} \left(\sum_{i=0}^p B_{i,p}(u)\mathbf{P}_i \right) + \mathbf{b}.$$

Por la linealidad de la multiplicación matricial $\mathbf{A}$:

$$\Phi(\mathbf{C}(u)) = \sum_{i=0}^p B_{i,p}(u)(\mathbf{A}\mathbf{P}_i) + \mathbf{b}.$$

Aquí utilizamos la propiedad de **Partición de la Unidad** para multiplicar el vector $\mathbf{b}$ por 1 e introducirlo en el sumatorio:

$$\Phi(\mathbf{C}(u)) = \sum_{i=0}^p B_{i,p}(u)(\mathbf{A}\mathbf{P}_i) + \mathbf{b} = \sum_{i=0}^p B_{i,p}(u)(\mathbf{A}\mathbf{P}_i) + \sum_{i=0}^p B_{i,p}(u)\mathbf{b} = \sum_{i=0}^p B_{i,p}(u)(\mathbf{A}\mathbf{P}_i + \mathbf{b}).$$

Reconocemos que $\mathbf{A}\mathbf{P}_i + \mathbf{b} = \Phi(\mathbf{P}_i)$, por lo tanto:

$$\Phi(\mathbf{C}(u)) = \sum_{i=0}^p B_{i,p}(u)\Phi(\mathbf{P}_i).$$

□

A.2.8. Propiedades de los B-Splines

Funciones base

Demostración. Se demostrarán las propiedades en el orden en el que han sido establecidas:

1. **Soporte local:** procederemos mediante inducción sobre p :

■ **Caso base** ($p = 0$): tenemos, $N_{i,0}(u) = 1$ si $u \in [u_i, u_{i+1})$ por definición.

- **Paso de inducción sobre p :** partiendo de la definición mediante el algoritmo de Cox-de Boor:

$$N_{i,p}(u) = \frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u). \quad (\text{A.6})$$

Asumimos que $N_{i,p-1}(u)$ tiene soporte en $[u_i, u_{i+p})$ y $N_{i+1,p-1}(u)$ tiene soporte en $[u_{i+1}, u_{i+p+1})$, luego $N_{i,p}(u)$ tiene soporte $[u_i, u_{i+p+1})$.

2. No negatividad: procederemos mediante inducción sobre p :

- **Caso base ($p = 0$):** tenemos por definición que $N_i = 0$ o 1 , luego $N_i \geq 0$.
- **Paso de inducción sobre p :** asumimos cierta la hipótesis para $p - 1$, tomando $u \in [u_i, u_{i+p+1})$ y aplicando el algoritmo A.6, la demostración se reduce a probar que:
 - $\frac{u - u_i}{u_{i+p} - u_i} \geq 0$, trivial pues por como esta definido el vector de nudos, $u_{i+p} \geq u_i$ y como $u \in [u_i, u_{i+p+1})$, $u \geq u_i$
 - $\frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} \geq 0$, de manera análoga tenemos $u_{i+p+1} \geq u_{i+1}$ y $u_{i+p+1} \geq u$.

3. Número de bases activas: procederemos mediante inducción sobre p :

- **Caso base ($p = 0$):** por definición del caso $p = 0$ de la funciones base y por la propiedad de no decrecimiento del vector de nudos, si $u \in [u_i, u_{i+1})$, existe únicamente una función base $N_{i,0}$ no nula en ese intervalo, luego el único índice j que satisface la condición de tener una función base no nula asociada es $i = j$.
- **Paso de inducción sobre p :** asumimos que para el intervalo $[u_i, u_{i+1})$ existen a lo sumo p funciones base $N_{j,p-1}$ no nulas con índices $j \in \{i - (p - 1), \dots, i\}$, debido al algoritmo de Cox-de Boor A.6 y a la no negatividad de los coeficientes y de las funciones base probada en el apartado anterior, si $u \in [u_i, u_{i+1})$, $N_{j,p}(u)$ será no nula en el intervalo $[u_i, u_{i+1})$ si alguna de las funciones base $N_{j,p-1}(u)$ y $N_{j+1,p-1}(u)$ no lo son, luego:
 - $N_{j,p-1}(u)$ es no nula a lo sumo cuando $j \in \{i - p + 1, i - p + 2, \dots, i\}$
 - $N_{j+1,p-1}(u)$ es no nula a lo sumo cuando $j \in \{i - p, i - p + 1, \dots, i - 1\}$
 Luego $N_{j,p}$ es no nula cuando $j \in [i - p, i]$, probando así las posibles $p + 1$ funciones base no nulas con sus índices posibles correspondientes.

Para demostrar la reducción, consideramos un nudo de valor $\mathbf{u}^*$ con multiplicidad k , tal que la secuencia de nudos coincidentes es $\mathbf{u}_j = \mathbf{u}_{j+1} = \dots = \mathbf{u}_i$, donde $\mathbf{i} = \mathbf{j} + \mathbf{k} - \mathbf{1}$.

Una función base $\mathbf{N}_{\mathbf{m},p}(\mathbf{u})$ es nula en el intervalo $[u_i, u_{i+1})$ si su soporte $(\mathbf{u}_{\mathbf{m}}, \mathbf{u}_{\mathbf{m}+p+1})$ no interseca dicho intervalo, es decir, si $\mathbf{u}_{\mathbf{m}+p+1} \leq \mathbf{u}_i$.

Las $\mathbf{k}$ funciones base con los índices más bajos en el rango máximo, dadas por el conjunto $\mathbf{J}_{\text{nulo}} = \{\mathbf{i} - \mathbf{p}, \mathbf{i} - \mathbf{p} + \mathbf{1}, \dots, \mathbf{i} - \mathbf{p} + \mathbf{k} - \mathbf{1}\}$, se anulan. Analizamos el índice final del soporte para el último elemento de este conjunto, $\mathbf{m} = \mathbf{i} - \mathbf{p} + \mathbf{k} - \mathbf{1}$:

$$\mathbf{u}_{\mathbf{m}+p+1} = u_{(i-p+k-1)+p+1} = \mathbf{u}_{i+k}$$

. Puesto que $\mathbf{u}_i, \mathbf{u}_{i+1}, \dots, \mathbf{u}_{i+k-1}$ son todos iguales a $\mathbf{u}^*$, la secuencia de k nudos coincidentes fuerza a estas $\mathbf{k}$ bases a ser nulas. Por lo tanto, el número de funciones base nulas es $\mathbf{k}$. Consecuentemente, el número de funciones base activas o no nulas es: $\mathbf{p} + \mathbf{1} - \mathbf{k}$.

4. Partición de la unidad:

- **Caso base ($p = 0$):** como hemos probado en el apartado anterior en $p = 0$ por cada intervalo $[u_i, u_{i+1})$ hay una sola función base no nula, luego $\forall u \in [u_0, u_m), \exists j \in R^+$ tal que $u \in [u_j, u_{j+1})$, luego,

$$\sum_{i=0}^n N_{i,0}(u) = N_{j,0}(u) = 1.$$

- **Paso de inducción sobre p :** asumimos ahora que se cumple $\sum_{i=0}^n N_{i,p}(u) = 1$, empezamos entonces con el algoritmo de Cox-de Boor A.6 talque,

$$\begin{aligned} \sum_{i=0}^n N_{i,p}(u) &= \sum_{i=0}^n \left[\frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u) \right] = \\ &= \sum_{i=0}^n \left[\frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) \right] + \sum_{i=0}^n \left[\frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u) \right] = \\ &= \sum_{i=0}^n \left[\frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) \right] + \sum_{i=1}^{n+1} \left[\frac{u_{i+p} - u}{u_{i+p} - u_i} N_{i,p-1}(u) \right]. \end{aligned}$$

Tomando que el soporte de $N_{i,p-1}(u)$ es $[u_i, u_i + p]$ y que U es abierto, luego $u_0 = u_1 = \dots = u_p$, por lo que $N_{0,p-1}(u)$, al tener soporte degenerado, es siempre nula, y $N_{n+1,p-1}$ es nula $\forall u \in [u_0, u_n]$ nos queda:

$$\begin{aligned} \sum_{i=0}^n N_{i,p}(u) &= \sum_{i=0}^n \left[\frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) \right] + \left[\frac{u_{i+p} - u}{u_{i+p} - u_i} N_{i,p-1}(u) \right] = \\ &= \sum_{i=0}^n \left[\frac{u - u_i}{u_{i+p} - u_i} + \frac{u_{i+p} - u}{u_{i+p} - u_i} \right] N_{i,p-1}(u) = \sum_{i=0}^n N_{i,p-1}(u) = 1. \end{aligned}$$

□

Curvas

Demostración. Se demostrarán las propiedades en el orden en el que han sido establecidas:

1. **Invarianza bajo transformaciones afines:** sea una transformación afín $T(\mathbf{P}) = \mathbf{M}\mathbf{P} + \mathbf{t}$, aplicamos esta a la curva obteniendo:

$$\begin{aligned} T(\mathbf{C}(u)) &= \mathbf{M}\mathbf{C}(u) + \mathbf{t} = \mathbf{M} \left(\sum_{i=0}^n N_{i,p}(u) \mathbf{P}_i \right) + \mathbf{t} = \\ &= \sum_{i=0}^n \mathbf{M}(N_{i,p}(u) \mathbf{P}_i) + 1 \cdot \mathbf{t} = \sum_{i=0}^n N_{i,p}(u) (\mathbf{M}\mathbf{P}_i) + \mathbf{t} \left(\sum_{i=0}^n N_{i,p}(u) \right) = \\ &= \sum_{i=0}^n N_{i,p}(u) (\mathbf{M}\mathbf{P}_i + \mathbf{t}). \end{aligned}$$

Siendo esta última serie la curva generada por los puntos de control transformados mediante la transformación afín.

2. **Continuidad controlable:** Suponemos que u_j tiene multiplicidad $k \leq p$, aplicamos ahora la definición de la derivada de la curva:

$$\mathbf{C}'(u) = \frac{d}{du} \left(\sum_{i=0}^n N_{i,p}(u) \mathbf{P}_i \right) = \sum_{i=0}^n N'_{i,p}(u) \mathbf{P}_i = \sum_{i=0}^n \left[\frac{p}{u_{i+p} - u_i} N_{i,p-1}(u) - \frac{p}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u) \right] \mathbf{P}_i.$$

Reordenamos ahora los índices del término de la derecha de la suma, aplicando las mismas propiedades que en el apartado anterior, obteniendo así:

$$\mathbf{C}'(u) = \sum_{i=0}^{n-1} N_{i,p-1}(u) \underbrace{\left[p \frac{\mathbf{P}_{i+1} - \mathbf{P}_i}{u_{i+p+1} - u_{i+1}} \right]}_{\mathbf{P}_i^{(1)}} = \sum_{i=0}^{n-1} N_{i,p-1}(u) \mathbf{P}_i^{(1)}.$$

Así, de manera recursiva definimos la derivada r -ésima de la curva:

$$\mathbf{C}^{(r)}(u) = \sum_i N_{i,p-r}(u) \mathbf{P}_i^{(r)}, \quad \left(\mathbf{P}_i^{(r)} = p \frac{\mathbf{P}_{i+1}^{(r-1)} - \mathbf{P}_i^{(r-1)}}{u_{i+p+1} - u_{i+1}} \right)$$

Obteniendo así que para $r = p - k$, si $u = u_j$, $N_{j-1,k} = 1$, luego:

$$\begin{aligned} \lim_{u \rightarrow u_j^-} \mathbf{C}^{(r)}(u) &= \sum_i \mathbf{P}_i^{(r)} \lim_{u \rightarrow u_j^-} N_{i,k}(u) = \mathbf{P}_{j-1}^{(r)} \cdot \lim_{u \rightarrow u_j^-} N_{j-1,k}(u) = \mathbf{P}_{j-1}^{(r)}. \\ \lim_{u \rightarrow u_j^+} \mathbf{C}^{(r)}(u) &= \sum_i \mathbf{P}_i^{(r)} \lim_{u \rightarrow u_j^+} N_{i,k}(u) = \mathbf{P}_{j-1}^{(r)} \cdot \lim_{u \rightarrow u_j^+} N_{j-1,k}(u) = \mathbf{P}_{j-1}^{(r)}. \end{aligned}$$

Para $r = p - k + 1$, tenemos que:

$$\begin{aligned} \lim_{u \rightarrow u_j^-} N_{j-1,k-1}(u) &= 1, \quad \lim_{u \rightarrow u_j^+} N_{j-1,k-1}(u) = 0. \\ \lim_{u \rightarrow u_j^-} N_{j,k-1}(u) &= 0, \quad \lim_{u \rightarrow u_j^+} N_{j,k-1}(u) = 1. \end{aligned}$$

Por lo que en este caso:

$$\begin{aligned} \lim_{u \rightarrow u_j^-} \mathbf{C}^{(r)}(u) &= \sum_i \mathbf{P}_i^{(r)} \lim_{u \rightarrow u_j^-} N_{i,k-1}(u) = \mathbf{P}_{j-1}^{(r)} \cdot \lim_{u \rightarrow u_j^-} N_{j-1,k-1}(u) = \mathbf{P}_{j-1}^{(r)}. \\ \lim_{u \rightarrow u_j^+} \mathbf{C}^{(r)}(u) &= \sum_i \mathbf{P}_i^{(r)} \lim_{u \rightarrow u_j^+} N_{i,k-1}(u) = \mathbf{P}_j^{(r)} \cdot \lim_{u \rightarrow u_j^+} N_{j,k-1}(u) = \mathbf{P}_j^{(r)}. \end{aligned}$$

Luego como la continuidad de $\mathbf{C}^{(r)}(u)$ depende de los puntos de control, $\mathbf{C}(u) \notin L^{p-k+1}(\Omega)$.

□

A.2.9. Igualdad de las nuevas funciones base bajo refinamiento

Demostración. Procederemos mediante inducción sobre p :

- **Caso base** ($p = 0$): Sea ξ con valor dentro del intervalo de nudos $[u_k, u_{k+1})$, separamos los casos:
 - $i < k$: En este caso $N_{i,0}(u)$ comparte soporte con $N_{i,0}^*(u)$.
 - $i > k$: En este caso $N_{i,0}(u)$ comparte soporte con $N_{i+1,0}^*(u)$.
 - $i = k$: En este caso $N_{k,0}(u)$ tiene como soporte a la unión de ambos soportes de $N_{k,0}^*(u)$ y $N_{k+1,0}^*(u)$, obteniendo la igualdad:

$$\begin{aligned} N_{k,0}(u) &= 1_{[u_k, u_{k+1})}(u) = 1_{[u_k, \xi)}(u) + 1_{[\xi, u_{k+1})}(u) = \\ &= N_{k,0}^*(u) + N_{k+1,0}^*(u) = \alpha_k N_{k,0}^*(u) + (1 - \alpha_{k+1}) N_{k+1,0}^*(u). \end{aligned}$$

- **Paso de inducción sobre p** : iniciamos asumiendo cierta la hipótesis para $p - 1$ y sustituimos en el algoritmo de Cox-de Boor,

$$\begin{aligned} N_{i,p}(u) &= \frac{u - u_i}{u_{i+p} - u_i} [\alpha_{i,p-1} N_{i,p-1}^*(u) + (1 - \alpha_{i+1,p-1}) N_{i+1,p-1}^*(u)] + \\ &+ \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} [\alpha_{i+1,p-1} N_{i+1,p-1}^*(u) + (1 - \alpha_{i+2,p-1}) N_{i+2,p-1}^*(u)]. \end{aligned}$$

Donde

$$\alpha_{i,p} = \begin{cases} 1 & \text{si } i \leq k - p, \\ \frac{\xi - u_i}{u_{i+p} - u_i} & \text{si } k - p < i \leq k, \\ 0 & \text{si } i > k. \end{cases}$$

Reorganizamos ahora la igualdad agrupando los términos asociados a cada función base distinta:

- Término asociado a $N_{i,p-1}^*$:

$$\frac{u - u_i}{u_{i+p} - u_i} \alpha_{i,p-1} = \alpha_{i,p} \frac{u - u_i^*}{u_{i+p}^* - u_i^*}.$$

- Término asociado a $N_{i+1,p-1}^*$:

$$\frac{u - u_i}{u_{i+p} - u_i} (1 - \alpha_{i+1,p-1}) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} \alpha_{i+1,p-1} = \alpha_{i,p} \frac{u_{i+p+1}^* - u}{u_{i+p+1}^* - u_{i+1}^*} + (1 - \alpha_{i+1,p}) \frac{u - u_{i+1}^*}{u_{i+1+p}^* - u_{i+1}^*}.$$

- Término asociado a $N_{i+2,p-1}^*$:

$$\frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} (1 - \alpha_{i+2,p-1}) = (1 - \alpha_{i+1,p}) \frac{u_{i+p+2}^* - u}{u_{i+p+2}^* - u_{i+2}^*}.$$

Sustituyendo ahora en la igualdad original obtenemos:

$$\begin{aligned} N_{i,p}(u) &= \alpha_{i,p} \underbrace{\left[\frac{u - u_i^*}{u_{i+p}^* - u_i^*} N_{i,p-1}^* + \frac{u_{i+p+1}^* - u}{u_{i+p+1}^* - u_{i+1}^*} N_{i+1,p-1}^* \right]}_{N_{i,p}^*(u)} + \\ &+ (1 - \alpha_{i+1,p}) \underbrace{\left[\frac{u - u_{i+1}^*}{u_{i+1+p}^* - u_{i+1}^*} N_{i+1,p-1}^* + \frac{u_{i+p+2}^* - u}{u_{i+p+2}^* - u_{i+2}^*} N_{i+2,p-1}^* \right]}_{N_{i+1,p}^*(u)}. \end{aligned}$$

□

A.2.10. Equivalencia de curvas bajo refinamiento

Demostración. Separaremos entre índices donde α_j toma valores 0 o 1 y en los que no son tan triviales

- Para los índices $j \leq k - p$, $\alpha_j = 1$, $\mathbf{Q}_j^w = (1)\mathbf{P}_j^w + (1-1)\mathbf{P}_{j-1}^w = \mathbf{P}_j^w$, $N_{j,p} = N_{j,p}^*$.
- Ahora para los índices $j > k$, $\alpha_j = 0$, $\mathbf{Q}_j^w = (0)\mathbf{P}_j^w + (1-0)\mathbf{P}_{j-1}^w = \mathbf{P}_{j-1}^w$, $N_{j-1,p} = N_{j,p}^*$.
- Por último para los índices $i \in \{k-p, k-p+1, \dots, k\}$, obtenemos la definición de la influencia de estos índices mediante:

$$\mathbf{C}^w(u) = \sum_{i=k-p}^k N_{i,p}(u) \mathbf{P}_i^w.$$

descomponemos ahora las funciones base en las nuevas funciones base:

$$\mathbf{C}^w(u) = \sum_{i=k-p}^k [\alpha_i N_{i,p}^*(u) + (1 - \alpha_{i+1}) N_{i+1,p}^*(u)] \mathbf{P}_i^w.$$

Separamos en la suma en sus dos términos principales y reindexamos:

$$\begin{aligned} &\sum_{i=k-p}^k \alpha_i N_{i,p}^*(u) \mathbf{P}_i^w. \\ &\sum_{i=k-p}^k (1 - \alpha_{i+1}) N_{i+1,p}^*(u) \mathbf{P}_i^w = \sum_{j=k-p+1}^{k+1} (1 - \alpha_j) N_{j,p}^*(u) \mathbf{P}_{j-1}^w. \end{aligned}$$

Como hemos mostrado anteriormente la equivalencia de los términos de la serie para $j = k - p$, $j = k + 1$ y ambos términos aparecen con su función base y punto de control correspondiente una única vez, podemos eliminarlos de la suma y centrarnos en los índices de los que desconocemos si se cumple.

$$\begin{aligned} \mathbf{C}^w(u) &= \sum_{j=k-p+1}^k [\alpha_j N_{j,p}^*(u) \mathbf{P}_j^w + (1 - \alpha_j) N_{j,p}^*(u) \mathbf{P}_{j-1}^w] \\ &= \sum_{j=k-p+1}^k N_{j,p}^*(u) [\alpha_j \mathbf{P}_j^w + (1 - \alpha_j) \mathbf{P}_{j-1}^w] = \sum_{j=k-p+1}^k N_{j,p}^*(u) \mathbf{Q}_j^w. \end{aligned}$$

□

A.2.11. Propiedades del sistema

Demostración. Se demostrarán las propiedades en el orden en el que han sido establecidas:

1. **Dispersión:** la matriz de rigidez se define como $\mathbf{K}_{i,j} = a(\phi_j, \phi_i)$. La propiedad fundamental de las funciones base NURBS es que tienen **soporte local**.

Para que $\mathbf{K}_{ij} \neq 0$, la intersección de los soportes debe ser no nula:

$$\text{supp}(\phi_i) \cap \text{supp}(\phi_j) \neq \emptyset.$$

Para un cierto grado polinomial p tenemos que cada función base NURBS tiene soporte en, a lo sumo, $p + 1$ intervalos no nulos del vector de nudos, por lo que obtenemos las siguientes propiedades:

- En 1D, una función base comparte soporte con a lo sumo p funciones base con índices menores, p con índices mayores y consigo misma. Total: $2p + 1$.
- En $\mathbb{R}^n$, si tomamos p como el grado polinomial direccional mayor en cada dirección, mediante estructura de producto tensorial, el número de interacciones es a lo sumo el producto de las interacciones en cada dimensión si tuvieran grado máximo: $(2p + 1)^n$.

Por tanto, el número de entradas no nulas por fila de $\mathbf{K}$ está acotado por $C_p = (2p + 1)^n$. Dado que el número total de incógnitas N_B crece con el refinamiento ($N_{el} \rightarrow \infty$), mientras que C_p permanece constante (fijo p o tomando p como el grado polinomial máximo), se cumple:

$$\lim_{N_{el} \rightarrow \infty} \frac{(2p + 1)^n}{N_B} = \lim_{N_B \rightarrow \infty} \frac{C_p}{N_B} = 0.$$

2. **Simetría:** La entrada de la matriz está dada por $\mathbf{K}_{i,j} = a(\phi_j, \phi_i)$. Si el operador diferencial del problema $\mathcal{L}$ es autoadjunto (por ejemplo, $\mathcal{L}u = -\Delta u$), la forma bilineal asociada $a(\cdot, \cdot)$ hereda la propiedad de simetría, tal que:

$$a(u, v) = a(v, u) \quad \forall u, v \in V.$$

Aplicando esto a las funciones de base:

$$\mathbf{K}_{ij} = a(\phi_j, \phi_i) = a(\phi_i, \phi_j) = \mathbf{K}_{ji}.$$

Por consiguiente, la matriz $\mathbf{K}$ es simétrica.

3. **Condicionamiento:** El número de condición espectral se define mediante el cociente de los autovalores extremos $\kappa(\mathbf{K}) = \frac{\lambda_{\max}}{\lambda_{\min}}$. Podemos estimarlos mediante el cociente de Rayleigh asociado a la forma bilineal $a(\cdot, \cdot)$ y la norma euclídea de los coeficientes $\mathbf{c}$:

$$\lambda = \frac{\mathbf{c}^T \mathbf{K} \mathbf{c}}{\mathbf{c}^T \mathbf{c}} = \frac{a(u_h, u_h)}{\|\mathbf{c}\|^2}, \quad \text{donde } u_h = \sum_i c_i \phi_i.$$

Es necesario relacionar la norma L^2 de la solución discreta u_h con la norma euclídea de sus coeficientes de control $\mathbf{c}$. Gracias a la **Estabilidad de Riesz** de la base B-spline y al mapeo geométrico desde el dominio paramétrico, se satisface la equivalencia de normas:

$$C_1 h^n \|\mathbf{c}\|^2 \leq \|u_h\|_{L^2}^2 \leq C_2 h^n \|\mathbf{c}\|^2.$$

Basándonos en esta relación y en las propiedades del espacio de aproximación:

- **Cota superior ($\lambda_{\max}$):** Se utiliza la **desigualdad inversa global** típica en espacios de elementos finitos y splines:

$$\|\nabla u_h\|_{L^2(\Omega)} \leq C_{\text{inv}} h^{-1} \|u_h\|_{L^2(\Omega)}.$$

Elevando al cuadrado para la forma bilineal obtenemos

$$a(u_h, u_h) = \|\nabla u_h\|_{L^2}^2 \lesssim h^{-2} \|u_h\|_{L^2}^2.$$

Finalmente, dividiendo por $\|\mathbf{c}\|^2$:

$$\lambda_{\max} \approx \sup_{\mathbf{c}} \frac{a(u_h, u_h)}{\|\mathbf{c}\|^2} \lesssim h^{n-2}.$$

- **Autovalor mínimo ($\lambda_{\min}$):** Usando la *coercividad* del operador ($a(u_h, u_h) \geq \alpha \|u_h\|_{L^2}^2$):

$$\lambda_{\min} \approx \inf_{u_h} \frac{a(u_h, u_h)}{\|\mathbf{c}\|^2} \gtrsim \frac{\|u_h\|_{L^2}^2}{h^{-n} \|u_h\|_{L^2}^2} \sim O(h^n).$$

Finalmente, el número de condición escala como el cociente de ambas cotas:

$$\kappa(\mathbf{K}) = \frac{\lambda_{\max}}{\lambda_{\min}} \approx C \frac{h^{n-2}}{h^n} = O(h^{-2}).$$

□

A.3. Base teórica de los algoritmos de condiciones de contorno

Partimos de la partición de la frontera $\Gamma = \overline{\Gamma_D} \cup \overline{\Gamma_R} \cup \overline{\Gamma_N}$, con $\Gamma_D \cap \Gamma_R = \Gamma_D \cap \Gamma_N = \Gamma_R \cap \Gamma_N = \emptyset$. Partiendo de la formulación variacional:

$$a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} u v d\Gamma, \quad (\text{A.7})$$

$$L(v) = \int_{\Omega} f v d\Omega + \int_{\Gamma_N} h v d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma. \quad (\text{A.8})$$

Sea el sistema lineal discreto resultante del ensamblado $\mathbf{K} \mathbf{u} = \mathbf{F}$, donde $\mathbf{u} \in \mathbb{R}^n$.

A.3.1. Tipo Dirichlet

Sea el espacio de funciones test en la condición de Dirichlet $V_{h,0} = \{v \in V_h \mid v|_{\Gamma_D} = 0\}$.

Sin embargo tenemos que para cada función de forma ϕ_k cuya intersección de su soporte con Γ_D es no nula, no son necesariamente nulas en la frontera. Esto implica que estas funciones no pertenecen al espacio test admisible.

Como consecuencia las filas k dadas por $\mathbf{K}_{k,i} = a(\phi_i, \phi_k)$, y su correspondiente elemento del vector de carga $L(\phi_k) = F_k$, no son válidas para el sistema, por ende estas filas las "perdemos", obteniendo ahora un sistema singular no resoluble, al que podemos añadir ahora las igualdades en los puntos de la frontera en las que solo una función base es no nula, de manera que obtenemos un número equivalente de ecuaciones a las que hemos eliminado de la forma $\mathbf{K}_{k,i} = 0, \forall i \neq k, \mathbf{K}_{k,k} = 1, F_k = g(x_k)$.

A.3.2. Tipo Neumann

Observamos en A.8, que las condiciones de contorno de tipo Neumann solo suman la integral referente a esta a la forma lineal, obteniendo así un caso trivial en el que solo debemos sumarle al valor del vector de carga su integral correspondiente a cada elemento en la frontera⁷.

A.3.3. Tipo Robin o mixtas

Análogamente a las condiciones de Neumann, en este caso observamos en A.7 y A.8 afectarán a la matriz de rigidez y al vector de carga, debiendo sumar las integrales correspondientes en ambos términos del sistema.

A.4. Código omitido

A.4.1. FEM

Código de las funciones base FEM

⁷Notar que para el caso 1D esta integral se resume a la evaluación en un punto.

```

1  clear; clc; close all;
2
3  % --- 1. PARÁMETROS CONFIGURABLES ---
4  L = 1;
5  num_elementos = 6;
6  p = 1;
7
8  % --- 2. GENERACIÓN DE LA MALLA ---
9  num_nodos = num_elementos * p + 1;
10 x_nodos = linspace(0, L, num_nodos);
11 colores = lines(num_nodos);
12
13 % Configuración de la figura
14 hold on; grid on;
15 axis([-0.1 L+0.1 -0.2 1.2]);
16
17 % --- 3. BUCLE DE FUNCIONES BASE ---
18 for i = 1:num_nodos
19     valores_nodales = zeros(1, num_nodos);
20     valores_nodales(i) = 1;
21     x_plot_total = [];
22     y_plot_total = [];
23     for e = 1:num_elementos
24         idx_ini = (e-1)*p + 1;
25         idx_fin = e*p + 1;
26         indices_locales = idx_ini:idx_fin;
27         x_elem = x_nodos(indices_locales);
28         y_elem = valores_nodales(indices_locales);
29         coeficientes = polyfit(x_elem, y_elem, p);
30         xx_elem = linspace(x_elem(1), x_elem(end), 50);
31         yy_elem = polyval(coeficientes, xx_elem);
32         x_plot_total = [x_plot_total, xx_elem];
33         y_plot_total = [y_plot_total, yy_elem];
34     end
35     % --- DIBUJO ---
36     plot(x_plot_total, y_plot_total, 'Color', colores(i,:), 'LineWidth', 2);
37     %try
38     %fill(x_plot_total, y_plot_total, colores(i,:), 'FaceAlpha', 0.1, 'EdgeColor', 'none');
39     %catch
40     %end
41     plot(x_nodos(i), 1, 'o', 'MarkerFaceColor', colores(i,:), 'MarkerEdgeColor', 'k');
42     if p == 1
43         offset = 0.05;
44     else
45         offset = 0.1 + 0.05*mod(i,2);
46     end
47     text(x_nodos(i), 1 + offset, sprintf('\phi_{%d}', i), ...
48         'Color', colores(i,:), 'HorizontalAlignment', 'center', 'FontWeight', 'bold');
49 end
50 for e = 0:num_elementos
51     x_borde = e * (L/num_elementos);
52     line([x_borde x_borde], [-0.2 1.2], 'Color', [0.7 0.7 0.7], 'LineStyle', '--');
53 end
54
55 hold off;
56
57 nombre_archivo = sprintf('bases_fem_p%d.svg', p);
58 set(gcf, 'PaperPositionMode', 'auto');
59 print(gcf, nombre_archivo, '-dsvg');

```

```

1 function plot_bernstein_bases(p)
2
3     m = 100;
4     u = linspace(0, 1, m+1);
5
6     figure;
7     hold on;
8     grid on;
9
10    colors = hsv(p + 1);
11
12    for i = 0:p
13
14        binom_coeff = nchoosek(p, i);
15        poly_term = u.^i .* (1 - u).^(p - i);
16        B_ip = binom_coeff * poly_term;
17        plot(u, B_ip, 'LineWidth', 2, 'Color', colors(i + 1, :), ...
18             'DisplayName', ['B_{', num2str(i), ',', num2str(p), '}(u)']);
19    end
20
21    xlim([0, 1]);
22    ylim([0, 1.1]);
23    hold off;
24
25    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
26    % EXPORTACIÓN EN FORMATO SVG
27    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
28
29    filename = ['bernstein_bases_p', num2str(p), '.svg'];
30    print(filename, '-dsvg');
31
32    disp(['Figura guardada como: ', filename]);
33 end
34

```

A.4.2. Figuras de Bézier

Código de las bases de Bernstein Código de las curvas racionalizadas de Bernstein

```

1
2 m = 100;
3 u = 0:1/m:1;
4
5 P = [0, 1, 2, 3;
6      0, 5, 0, 4.5;
7      0, 1, 2, 4];
8
9 W = [1.0, 10.0, 7.0, 1.0];
10
11 [aux, n_puntos] = size(P);
12
13 sol = zeros(aux, m+1);
14
15 for k = 1:m+1
16
17     Bold = zeros(n_puntos, 1);
18     Bold(1) = 1;
19
20     for j = 2:n_puntos
21         Bnew = zeros(n_puntos, 1);
22         for i = 1:j-1
23             % B(i,j) = (1-u)*B(i,j-1) + u*B(i-1,j-1)

```



```
88 print('curva_bezier_racional.svg', '-dsvg');
```

A.4.3. Código fundamental para el funcionamiento en C++

Clase iMatrix

```
1  /*****
2  *AUTHOR: Carlos Palomera Oliva
3  *DATE: MARCH 2025
4  *****/
5  #include <iostream>
6  #include <stdexcept>
7  #include <algorithm>
8  #include "C:\Librerias\eigen-3.4.0\Eigen\Dense" //windows version
9  // #include "home/carlo/Librerias/eigen-3.4.0/Eigen/Dense" //linux version
10 // #include "home/carlo/Librerias/eigen-3.4.0/Eigen/Sparse" //linux version
11 #include <iomanip>
12 #include <vector>
13 #include <cmath>
14 #include <numeric>
15 #include <type_traits>
16
17 #ifndef IMATRIX_H
18 #define IMATRIX_H
19
20 // This file is part of the qBLinAlg linear algebra library.
21
22 /*
23 MIT License
24 Copyright (c) 2023 Michael Bennett
25
26 Permission is hereby granted, free of charge, to any person obtaining a copy of this software
27 and associated documentation files (the "Software"), to deal in the Software without restriction,
28 including without limitation the rights to use, copy, modify, merge, publish, distribute,
29 sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is
30 furnished to do so, subject to the following conditions:
31
32 The above copyright notice and this permission notice shall be included in all copies or
33 substantial portions of the Software.
34
35 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING
36 BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
37 NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM,
38 DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
39 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
40 */
41
42
43
44
45 template <class T>
46 class iMatrix{
47 public:
48     // Define the various constructors.
49     iMatrix();
50     iMatrix(int nRows, int nCols);
51     iMatrix(int nRows, int nCols, const T *inputData);
52     iMatrix(int nRows, int nCols, const Eigen::VectorXd &inputData);
53     iMatrix(const iMatrix<T> &inputMatrix);
54
55     // And the destructor.
56     ~iMatrix();
57
58     // Configuration methods.
59     bool Resize(int numRows, int numCols);
60     void SetToIdentity();
61
62     // Element access methods.
63     T GetElement(int row, int col) const;
64     std::vector<T> GetRow(int row) const;
65     std::vector<T> GetCol(int col) const;
66     bool SetElement(int row, int col, T elementValue);
67     bool SetCol(int col, std::vector<T> elements);
68     bool SetRow(int row, std::vector<T> elements);
69     int GetNumRows() const;
70     int GetNumCols() const;
71     iMatrix<T> GetSubMat(int rowIni, int rowEnd, int colIni, int colEnd) const;
72
73
74     // Overload the assignment operator.
75     iMatrix<T> operator= (const iMatrix<T> &rhs);
76
77
78     // Write
79     inline T& operator()(int i, int j) noexcept {
80         return m_matrixData[i * m_nCols + j];
81     };
82
83     // Read
84     inline const T& operator()(int i, int j) const noexcept {
85         return m_matrixData[i * m_nCols + j];
86     };
87
88
89     iMatrix<T> invert() const { //Uses LU factorization with partial pivoting
90         if (m_nRows != m_nCols) {
91             throw std::runtime_error("Matrix must be square for inversion.");
92         }
93     }
94 }
```

```

92     }
93
94     int n = m_nRows;
95     iMatrix<T> A(*this);
96     std::vector<int> piv(n);
97     for (int i = 0; i < n; ++i) piv[i] = i;
98
99     for (int k = 0; k < n; ++k) {
100         T maxVal = std::abs(A(k, k));
101         int pivotRow = k;
102         for (int i = k + 1; i < n; ++i) {
103             T val = std::abs(A(i, k));
104             if (val > maxVal) {
105                 maxVal = val;
106                 pivotRow = i;
107             }
108         }
109         if (maxVal == T(0)) {
110             throw std::runtime_error("Matrix is singular.");
111         }
112
113         if (pivotRow != k) {
114             for (int j = 0; j < n; ++j) {
115                 std::swap(A(k, j), A(pivotRow, j));
116             }
117             std::swap(piv[k], piv[pivotRow]);
118         }
119
120         for (int i = k + 1; i < n; ++i) {
121             A(i, k) /= A(k, k);
122             T factor = A(i, k);
123             for (int j = k + 1; j < n; ++j) {
124                 A(i, j) -= factor * A(k, j);
125             }
126         }
127     }
128
129     iMatrix<T> inv(n, n);
130     for (int col = 0; col < n; ++col) {
131         std::vector<T> b(n, T(0));
132         b[piv[col]] = T(1);
133
134         for (int i = 0; i < n; ++i) {
135             for (int j = 0; j < i; ++j) {
136                 b[i] -= A(i, j) * b[j];
137             }
138         }
139
140         for (int i = n - 1; i >= 0; --i) {
141             for (int j = i + 1; j < n; ++j) {
142                 b[i] -= A(i, j) * b[j];
143             }
144             b[i] /= A(i, i);
145         }
146
147         for (int i = 0; i < n; ++i) {
148             inv(i, col) = b[i];
149         }
150     }
151
152     return inv;
153 }
154 // Overload +, - and * operators (friends).
155 template <class U> friend iMatrix<U> operator+ (const iMatrix<U>& lhs, const iMatrix<U>& rhs);
156 template <class U> friend iMatrix<U> operator+ (const U& lhs, const iMatrix<U>& rhs);
157 template <class U> friend iMatrix<U> operator+ (const iMatrix<U>& lhs, const U& rhs);
158
159 template <class U> friend iMatrix<U> operator- (const iMatrix<U>& lhs, const iMatrix<U>& rhs);
160 template <class U> friend iMatrix<U> operator- (const U& lhs, const iMatrix<U>& rhs);
161 template <class U> friend iMatrix<U> operator- (const iMatrix<U>& lhs, const U& rhs);
162
163 template <class U> friend iMatrix<U> operator* (const iMatrix<U>& lhs, const iMatrix<U>& rhs);
164 template <class U> friend iMatrix<U> operator* (const U& lhs, const iMatrix<U>& rhs);
165 template <class U> friend iMatrix<U> operator* (const iMatrix<U>& lhs, const U& rhs);
166
167 template <class U> friend iMatrix<U> operator* (const std::vector<U>& v1, const std::vector<U>& v2);
168
169 iMatrix<T>& operator+= (const iMatrix<T>& rhs);
170 iMatrix<T>& operator-= (const T& rhs);
171
172 void PrintMatrix();
173 void PrintMatrix(int precision);
174 std::vector<T> Vectorize() const;
175 iMatrix<T> Transpose() const;
176
177
178 bool IsSquare();
179 T Determinant() const;
180
181 // Función estática para calcular el Producto Punto entre dos std::vector<T>.
182 static T dot_product (const std::vector<T>& v1, const std::vector<T>& v2);
183
184 private:
185     int Sub2Ind(int row, int col) const;
186 private:
187     std::vector<T> m_matrixData;
188     int m_nRows, m_nCols;
189 };
190
191
192 /* *****
193 CONSTRUCTOR / DESTRUCTOR FUNCTIONS
194 /* *****

```

```

195 template <class T>
196 iMatrix<T>::iMatrix() : m_nRows(1), m_nCols(1)
197 {
198     // Inicializamos el vector con 1 elemento (por defecto 0 o T())
199     m_matrixData.resize(1);
200 }
201
202 // Construct empty matrix (all elements 0)
203 template <class T>
204 iMatrix<T>::iMatrix(int nRows, int nCols) : m_nRows(nRows), m_nCols(nCols)
205 {
206     int nElements = m_nRows * m_nCols;
207     if constexpr (std::is_arithmetic_v<T>) {
208         // Redimensionar e inicializar a 0.0f
209         m_matrixData.resize(nElements, static_cast<T>(0.0f));
210     } else {
211         // Redimensionar e inicializar con el constructor por defecto T()
212         m_matrixData.resize(nElements);
213     }
214 }
215 // Construct from Eigen::VectorXd
216 template <class T>
217 iMatrix<T>::iMatrix(int nRows, int nCols, const Eigen::VectorXd& inputData)
218 : m_nRows(nRows), m_nCols(nCols)
219 {
220     int nElements = m_nRows * m_nCols;
221     if (inputData.size() != nElements)
222         throw std::invalid_argument("Eigen::VectorXd no tiene el tamaño correcto para llenar la matriz.");
223
224     m_matrixData.resize(nElements);
225     for (int i = 0; i < nElements; ++i)
226         m_matrixData[i] = static_cast<T>(inputData[i]);
227 }
228 // Construct from const linear array.
229 template <class T>
230 iMatrix<T>::iMatrix(int nRows, int nCols, const T *inputData) : m_nRows(nRows), m_nCols(nCols)
231 {
232     int nElements = m_nRows * m_nCols;
233     // Usamos el constructor de rango de std::vector para la copia eficiente.
234     m_matrixData = std::vector<T>(inputData, inputData + nElements);
235 }
236
237 // The copy constructor.
238 template <class T>
239 iMatrix<T>::iMatrix(const iMatrix<T> &inputMatrix)
240 : m_nRows(inputMatrix.m_nRows),
241   m_nCols(inputMatrix.m_nCols),
242   m_matrixData(inputMatrix.m_matrixData)
243 {
244     // Cuerpo del constructor vacío (RAII).
245 }
246
247 template <class T>
248 iMatrix<T>::~iMatrix()
249 {
250     // El destructor de std::vector se llama automáticamente (RAII).
251 }
252 /* *****
253 CONFIGURATION FUNCTIONS
254 /* *****
255 template <class T>
256 bool iMatrix<T>::Resize(int numRows, int numCols)
257 {
258     m_nRows = numRows;
259     m_nCols = numCols;
260     int nElements = (m_nRows * m_nCols);
261
262     if constexpr (std::is_arithmetic_v<T>) {
263         m_matrixData.resize(nElements, static_cast<T>(0.0f));
264     } else {
265         m_matrixData.resize(nElements);
266     }
267
268     return true;
269 }
270
271 template <class T>
272 void iMatrix<T>::SetToIdentity()
273 {
274     if (!IsSquare())
275         throw std::invalid_argument("Cannot form an identity matrix that is not square.");
276
277     for (int row=0; row<m_nRows; ++row)
278     {
279         for (int col=0; col<m_nCols; ++col)
280         {
281             if (col == row)
282                 m_matrixData[Sub2Ind(row,col)] = 1.0;
283             else
284                 m_matrixData[Sub2Ind(row,col)] = 0.0;
285         }
286     }
287 }
288
289 template <class T>
290 int iMatrix<T>::GetNumRows() const
291 {
292     return m_nRows;
293 }
294
295 template <class T>
296 int iMatrix<T>::GetNumCols() const
297 {

```

```

298     return m_nCols;
299 }
300 /* *****
301 ELEMENT FUNCTIONS
302 /* *****
303 template <class T>
304 T iMatrix<T>::GetElement(int row, int col) const
305 {
306     int linearIndex = Sub2Ind(row, col);
307     if (linearIndex >= 0)
308         return m_matrixData[linearIndex];
309     else{
310         if constexpr (std::is_arithmetic_v<T>) {
311             return 0; // Para tipos numéricos (int, float, double)
312         } else {
313             return T(); // Para contenedores (std::vector<float>, etc.)
314         }
315     }
316 }
317
318 template <class T>
319 std::vector<T> iMatrix<T>::GetRow(int row) const
320 {
321     int linearIndex = Sub2Ind(row, 0);
322     std::vector<T> AuxRow;
323     for(unsigned int i=0; i<this->GetNumCols(); i++){
324         AuxRow.push_back(m_matrixData[linearIndex]);
325         linearIndex++;
326     }
327     return AuxRow;
328 }
329
330 template <class T>
331 bool iMatrix<T>::SetElement(int row, int col, T elementValue)
332 {
333     int linearIndex = Sub2Ind(row, col);
334     if (linearIndex >= 0)
335     {
336         m_matrixData[linearIndex] = elementValue;
337         return true;
338     }
339     else
340     {
341         return false;
342     }
343 }
344
345 template <class T>
346 bool iMatrix<T>::SetCol(int col, std::vector<T> elements){
347     if (col < 0 || col >= m_nCols || static_cast<int>(elements.size()) != m_nRows)
348         return false;
349     for (int i = 0; i < m_nRows; ++i) {
350         m_matrixData[i * m_nCols + col] = elements[i];
351     }
352     return true;
353 }
354
355 template <class T>
356 bool iMatrix<T>::SetRow(int row, std::vector<T> elements){
357     int linearIndex = Sub2Ind(row, 0);
358     int aux = this->GetNumCols();
359     if (linearIndex >= 0){
360         for(unsigned int i=0; i<aux; i++){
361             (*this)(row, i) = elements[i];
362             //this->SetElement(row, i, elements[i]);
363             linearIndex=linearIndex+aux;
364         }
365         return true;
366     }
367     else{
368         return false;
369     }
370 }
371
372 template <class T>
373 std::vector<T> iMatrix<T>::GetCol(int col) const
374 {
375     std::vector<T> result(m_nRows);
376     for (int i = 0; i < m_nRows; ++i) {
377         result[i] = m_matrixData[i * m_nCols + col];
378     }
379     return result;
380 }
381
382 template <class T>
383 iMatrix<T> iMatrix<T>::GetSubMat(int rowIni, int rowEnd, int colIni, int colEnd) const
384 {
385     int numRows= rowEnd-rowIni+1;
386     int numCols= colEnd-colIni+1;
387
388     int aux = this->GetNumCols();
389     iMatrix<T> AuxMat(numRows, numCols);
390
391     for(unsigned int i=0; i< numRows; i++){
392         for(unsigned int j=0; j< numCols; j++){
393             AuxMat(i, j)=(*this)(i+rowIni, j+colIni);
394         }
395     }
396
397     return AuxMat;
398 }
399
400

```

```

401 }
402
403 /* *****
404 THE + OPERATOR
405 /* *****
406 // matrix + matrix.
407 template <class T>
408 iMatrix<T> operator+ (const iMatrix<T>& lhs, const iMatrix<T>& rhs)
409 {
410     int numRows = lhs.m_nRows;
411     int numCols = lhs.m_nCols;
412     int numElements = lhs.m_matrixData.size();
413     T *tempResult = new T[numElements];
414     for (int i=0; i<numElements; i++)
415         tempResult[i] = lhs.m_matrixData[i] + rhs.m_matrixData[i];
416
417     iMatrix<T> result(numRows, numCols, tempResult);
418     delete[] tempResult;
419     return result;
420 }
421
422 // scaler + matrix
423 template <class T>
424 iMatrix<T> operator+ (const T& lhs, const iMatrix<T>& rhs)
425 {
426     int numRows = rhs.m_nRows;
427     int numCols = rhs.m_nCols;
428     int numElements = rhs.m_matrixData.size();
429     T *tempResult = new T[numElements];
430     for (int i=0; i<numElements; ++i)
431         tempResult[i] = lhs + rhs.m_matrixData[i];
432
433     iMatrix<T> result(numRows, numCols, tempResult);
434     delete[] tempResult;
435     return result;
436 }
437
438 // matrix + scaler
439 template <class T>
440 iMatrix<T> operator+ (const iMatrix<T>& lhs, const T& rhs)
441 {
442     int numRows = lhs.m_nRows;
443     int numCols = lhs.m_nCols;
444     int numElements = lhs.m_matrixData.size();
445     T *tempResult = new T[numElements];
446     for (int i=0; i<numElements; ++i)
447         tempResult[i] = lhs.m_matrixData[i] + rhs;
448
449     iMatrix<T> result(numRows, numCols, tempResult);
450     delete[] tempResult;
451     return result;
452 }
453
454 /* *****
455 THE - OPERATOR
456 /* *****
457 // matrix - matrix
458 template <class T>
459 iMatrix<T> operator- (const iMatrix<T>& lhs, const iMatrix<T>& rhs)
460 {
461     int numRows = lhs.m_nRows;
462     int numCols = lhs.m_nCols;
463     int numElements = lhs.m_matrixData.size();
464     T *tempResult = new T[numElements];
465     for (int i=0; i<numElements; i++)
466         tempResult[i] = lhs.m_matrixData[i] - rhs.m_matrixData[i];
467
468     iMatrix result(numRows, numCols, tempResult);
469     delete[] tempResult;
470     return result;
471 }
472
473 // scaler - matrix
474 template <class T>
475 iMatrix<T> operator- (const T& lhs, const iMatrix<T>& rhs)
476 {
477     int numRows = rhs.m_nRows;
478     int numCols = rhs.m_nCols;
479     int numElements = rhs.m_matrixData.size();
480     T *tempResult = new T[numElements];
481     for (int i=0; i<numElements; ++i)
482         tempResult[i] = lhs - rhs.m_matrixData[i];
483
484     iMatrix<T> result(numRows, numCols, tempResult);
485     delete[] tempResult;
486     return result;
487 }
488
489 // matrix - scaler
490 template <class T>
491 iMatrix<T> operator- (const iMatrix<T>& lhs, const T& rhs)
492 {
493     int numRows = lhs.m_nRows;
494     int numCols = lhs.m_nCols;
495     int numElements = lhs.m_matrixData.size();
496     T *tempResult = new T[numElements];
497     for (int i=0; i<numElements; ++i)
498         tempResult[i] = lhs.m_matrixData[i] - rhs;
499
500     iMatrix<T> result(numRows, numCols, tempResult);
501     delete[] tempResult;
502     return result;
503 }

```

```

504
505 // scalar * matrix
506 template <class T>
507 iMatrix<T> operator* (const T& lhs, const iMatrix<T>& rhs)
508 {
509     iMatrix<T> result(rhs.m_nRows, rhs.m_nCols);
510     std::transform(
511         rhs.m_matrixData.begin(),
512         rhs.m_matrixData.end(),
513         result.m_matrixData.begin(),
514         [&lhs] (const T& element) { return lhs * element; }
515     );
516     return result;
517 }
518
519 /* Old product operator for the scalar
520
521 template <class T>
522 iMatrix<T> operator* (const T& lhs, const iMatrix<T>& rhs)
523 {
524     int numRows = rhs.m_nRows;
525     int numCols = rhs.m_nCols;
526     int numElements = rhs.m_matrixData.size();
527     T *tempResult = new T[numElements];
528     for (int i=0; i<numElements; ++i)
529         tempResult[i] = lhs * rhs.m_matrixData[i];
530
531     iMatrix<T> result(numRows, numCols, tempResult);
532     delete[] tempResult;
533     return result;
534 }
535
536
537 template <class T>
538 iMatrix<T> operator* (const iMatrix<T>& lhs, const T& rhs)
539 {
540     int numRows = lhs.m_nRows;
541     int numCols = lhs.m_nCols;
542     int numElements = lhs.m_matrixData.size();
543     T *tempResult = new T[numElements];
544     for (int i=0; i<numElements; ++i)
545         tempResult[i] = lhs.m_matrixData[i] * rhs;
546
547     iMatrix<T> result(numRows, numCols, tempResult);
548     delete[] tempResult;
549     return result;
550 }
551
552 */
553 // matrix * scalar
554 template <class T>
555 iMatrix<T> operator* (const iMatrix<T>& lhs, const T& rhs)
556 {
557     iMatrix<T> result(lhs.m_nRows, lhs.m_nCols);
558
559     std::transform(
560         lhs.m_matrixData.begin(),
561         lhs.m_matrixData.end(),
562         result.m_matrixData.begin(),
563         [&rhs] (const T& element) { return element * rhs; }
564     );
565
566     return result;
567 }
568
569 // matrix * matrix
570 template <class T>
571 iMatrix<T> operator* (const iMatrix<T>& lhs, const iMatrix<T>& rhs)
572 {
573     int r_numRows = rhs.m_nRows;
574     int r_numCols = rhs.m_nCols;
575     int l_numRows = lhs.m_nRows;
576     int l_numCols = lhs.m_nCols;
577
578     if (l_numCols == r_numRows)
579     {
580         iMatrix<T> rhs_T(r_numCols, r_numRows);
581         for (int i = 0; i < r_numRows; ++i) {
582             for (int j = 0; j < r_numCols; ++j) {
583                 rhs_T(j, i) = rhs(i, j);
584             }
585         }
586
587         T *tempResult = new T[lhs.m_nRows * rhs.m_nCols];
588
589         for (int lhsRow=0; lhsRow<l_numRows; lhsRow++)
590         {
591             for (int rhsCol=0; rhsCol<r_numCols; rhsCol++)
592             {
593                 T elementResult = static_cast<T>(0.0);
594                 int lhsLinearIndex = (lhsRow * l_numCols);
595                 int rhsTLinearIndex = (rhsCol * r_numRows);
596
597                 for (int k=0; k<l_numCols; k++)
598                 {
599                     elementResult += (lhs.m_matrixData[lhsLinearIndex + k] * rhs_T.m_matrixData[rhsTLinearIndex + k]);
600                 }
601                 int resultLinearIndex = (lhsRow * r_numCols) + rhsCol;
602                 tempResult[resultLinearIndex] = elementResult;
603             }
604         }
605     }
606 }

```

```

607
608         iMatrix<T> result(l_numRows, r_numCols, tempResult);
609         delete[] tempResult;
610         return result;
611     }
612     else
613     {
614         iMatrix<T> result(1, 1);
615         return result;
616     }
617 }
618
619 template <class T>
620 iMatrix<T> iMatrix<T>::operator= (const iMatrix<T> &rhs)
621 {
622     if (this != &rhs)
623     {
624         m_nRows = rhs.m_nRows;
625         m_nCols = rhs.m_nCols;
626         m_matrixData = rhs.m_matrixData;
627     }
628
629     return *this;
630 }
631
632
633 template <class T>
634 iMatrix<T> operator* (const std::vector<T>& v1, const std::vector<T>& v2)
635 {
636     int N = v1.size();
637     int M = v2.size();
638
639     iMatrix<T> result(N, M);
640     for (int i = 0; i < N; ++i)
641     {
642         T v1_i = v1[i];
643
644         for (int j = 0; j < M; ++j)
645         {
646             result(i, j) = v1_i * v2[j];
647         }
648     }
649
650     return result;
651 }
652
653
654 template <class T>
655 iMatrix<T>& iMatrix<T>::operator+= (const iMatrix<T>& rhs)
656 {
657     if (m_nRows != rhs.m_nRows || m_nCols != rhs.m_nCols)
658     {
659         throw std::invalid_argument("Error: Las matrices deben tener las mismas dimensiones para la suma in-place.");
660     }
661
662     int numElements = m_matrixData.size();
663     T* data_lhs = m_matrixData.data();
664     const T* data_rhs = rhs.m_matrixData.data();
665     for (int i = 0; i < numElements; ++i)
666     {
667         data_lhs[i] += data_rhs[i];
668     }
669     return *this;
670 }
671
672
673 template <class T>
674 iMatrix<T>& iMatrix<T>::operator*= (const T& rhs)
675 {
676     int numElements = m_matrixData.size();
677     T* data_lhs = m_matrixData.data();
678     for (int i = 0; i < numElements; ++i)
679     {
680         data_lhs[i] *= rhs;
681     }
682
683     return *this;
684 }
685
686
687 // A simple function to print a matrix to stdout.
688 template <class T>
689 void iMatrix<T>::PrintMatrix()
690 {
691     int nRows = this->GetNumRows();
692     int nCols = this->GetNumCols();
693     for (int row = 0; row < nRows; ++row)
694     {
695         for (int col = 0; col < nCols; ++col)
696         {
697             std::cout << std::fixed << std::setprecision(3) << (*this)(row, col) << " ";
698         }
699         std::cout << std::endl;
700     }
701 }
702
703 // A simple function to print a matrix to stdout, with specified precision.
704 template <class T>
705 void iMatrix<T>::PrintMatrix(int precision)
706 {
707     int nRows = this->GetNumRows();
708     int nCols = this->GetNumCols();
709     for (int row = 0; row < nRows; ++row)

```

```

710 {
711     for (int col = 0; col < nCols; ++col)
712     {
713         std::cout << std::fixed << std::setprecision(precision) << (*this)(row, col) << " ";
714     }
715     std::cout << std::endl;
716 }
717 }
718
719 // Function to test whether the matrix is square.
720 template <class T>
721 bool iMatrix<T>::IsSquare()
722 {
723     if (m_nCols == m_nRows)
724         return true;
725     else
726         return false;
727 }
728
729 template <class T>
730 T iMatrix<T>::Determinant() const
731 {
732     if (!this->IsSquare()){
733         return static_cast<T>(0.0);
734     }
735
736     int N = this->m_nRows;
737     if (N == 1){
738         return (*this)(0, 0);
739     }
740     iMatrix<T> tempMatrix = *this;
741
742     T det = static_cast<T>(1.0);
743     int sign = 1;
744
745     for (int k = 0; k < N; ++k){
746         int pivotRow = k;
747         T maxValue = std::abs(tempMatrix(k, k));
748
749         for (int i = k + 1; i < N; ++i)
750         {
751             if (std::abs(tempMatrix(i, k)) > maxValue)
752             {
753                 maxValue = std::abs(tempMatrix(i, k));
754                 pivotRow = i;
755             }
756         }
757
758         if (pivotRow != k){
759             for (int j = 0; j < N; ++j){
760                 std::swap(tempMatrix(k, j), tempMatrix(pivotRow, j));
761             }
762             sign = -sign;
763         }
764
765         if (std::abs(tempMatrix(k, k)) < static_cast<T>(1e-9)){
766             return static_cast<T>(0.0);
767         }
768
769         T pivotValue = tempMatrix(k, k);
770         for (int i = k + 1; i < N; ++i) {
771             T factor = tempMatrix(i, k) / pivotValue;
772
773             for (int j = k; j < N; ++j){
774                 tempMatrix(i, j) -= factor * tempMatrix(k, j);
775             }
776         }
777
778         det *= pivotValue;
779     }
780     return sign * det;
781 }
782
783 template <class T>
784 std::vector<T> iMatrix<T>::Vectorize() const
785 {
786     return m_matrixData;
787 }
788
789 template <class T>
790 iMatrix<T> iMatrix<T>::Transpose() const
791 {
792     iMatrix<T> result(m_nCols, m_nRows); // + filas y columnas intercambiadas
793
794     // Recorremos todos los elementos de la matriz original
795     for (int i = 0; i < m_nRows; ++i)
796     {
797         for (int j = 0; j < m_nCols; ++j)
798         {
799             // Elemento (i,j) en la original → posición (j,i) en la transpuesta
800             result(j, i) = (*this)(i, j);
801         }
802     }
803
804     return result;
805 }
806
807 template <class T>
808 T dot_product (const std::vector<T>& v1, const std::vector<T>& v2)
809 {
810
811 }

```

```

813     if (v1.size() != v2.size())
814     {
815         throw std::invalid_argument("Error: Los vectores deben tener la misma dimensión para el Producto Punto.");
816     }
817
818     int N = v1.size();
819     T resultValue = static_cast<T>(0.0);
820
821     for (int i = 0; i < N; ++i)
822     {
823         resultValue += v1[i] * v2[i];
824     }
825
826     return resultValue;
827 }
828 /* *****
829 PRIVATE FUNCTIONS
830 /* *****
831 // Function to return the linear index corresponding to the supplied row and column values.
832 template <class T>
833 int iMatrix<T>::Sub2Ind(int row, int col) const
834 {
835     if ((row < m_nRows) && (row >= 0) && (col < m_nCols) && (col >= 0))
836         return (row * m_nCols) + col;
837     else
838         return -1;
839 }
840
841
842
843 #endif

```

Fichero funciones.hpp

```

1  /******
2  *AUTHOR: Carlos Palomera Oliva
3  *DATE: MARCH 2025
4  *****/
5  #define _USE_MATH_DEFINES
6  #include <iostream>
7  #include <unordered_map>
8  #include <utility>
9  #include <cmath>
10 #include <iomanip>
11 #include <vector>
12 #include <algorithm>
13 #include <fstream>
14 #include <string>
15 #include <limits>
16 #include "iMatrix.cpp"
17 #include "C:\Librerias\eigen-3.4.0\Eigen\Sparse" //windows version
18 #include "C:\Librerias\eigen-3.4.0\Eigen\Dense" //windows version
19 // #include "/home/carlos/Librerias/eigen-3.4.0/Eigen/Dense"
20 // #include "/home/carlos/Librerias/eigen-3.4.0/Eigen/Sparse"
21 #ifndef FUNCIONES_HPP
22 #define FUNCIONES_HPP
23
24
25
26
27 /******
28 * Structure of the Data File for Matrix
29 * Nrow Ncol
30 * vector of size Ncol representing the first variable where elements are separated by space Ex(x1 x2 x3 ...)
31 * vector of size Ncol ...
32 * .
33 * . Nrow times in total
34 * .
35 *
36 *****/
37 /*
38 *Input: Direction of the data file
39 *Output: iMatrix filled with the control points in data file
40 */
41 iMatrix<double> ReadDataFile_Matrix(std::string direction);
42
43 /******
44 * Structure of the Data File for Tensors
45 * Ncol NSplits
46 * vector of size Ncol representing the first variable where elements are separated by space Ex(x1 x2 x3 ...)
47 * vector of size Ncol ...
48 * .
49 * . 3 times in total
50 * .
51 * -----
52 * The previous repeated NSplits totals
53 *****/
54 /*
55 *Input: Direction of the data file
56 *Output: vector of iMatrixes filled with the control points in data file
57 */
58 std::vector<iMatrix<double>> ReadDataFile_CtrlPts(std::string direction);
59
60
61 /*
62 *Input: Value of the point where the polinomial is evaluated t, and degree of the polinomial n
63 *Condition: t∈[0,1]
64 *Output: Values of all the Bernstein polinomial of degree n evaluated at t
65 */
66 std::vector<double> BernsteinPolinomial(double t, int n);

```

```

67
68
69 /*
70 *Input: iMatrix of control points and parameter of the curve t
71 *Conditions: t∈[0,1]
72 *Output: Value of the Bezier curve of such control points in the point t
73 */
74 std::vector<double> BezierCurve_Point_t(const iMatrix<double> &PtsControl, double t);
75
76
77 /*
78 *Input: iMatrix of control points and number of steps for the curve
79 *Conditions: Steps must be >= 1
80 *Output: Value of the Bezier curve at every point from 0 to 1 separated in N_Steps steps
81 */
82 iMatrix<double> BezierCurve(const iMatrix<double> &PtsControl, double N_Steps);
83
84 /*
85 *Input: Tensor of control points and the 2 parameters of the curve t1, t2
86 *Conditions: t1,t2∈[0,1]
87 *Output: Value of the Bezier surface at of such control points in the points t1 t2
88 */
89 std::vector<double> BezierSurface_Point_t1_t2(const std::vector<iMatrix<double>> &PtsControl, double t1, double t2);
90
91 /*
92 *Input: iMatrix of control points and number of steps for the curve in both parameters
93 *Conditions: N_Steps must be >= 1
94 *Output: Value of the Bezier surface at every point from 0 to 1 separated in N_Steps along both parameters
95 */
96 std::vector<iMatrix<double>> BezierSurface(const std::vector<iMatrix<double>> &PtsControl, double N_Steps);
97
98 /*
99 *Input: Bezier surface in form of a tensor
100 *Output: Writes a file of 3 matrices one for each coordinate
101 */
102 void Write_BezierSurface(const std::vector<iMatrix<double>> &Tensor, std::string name);
103
104 /*
105 *Input: CtrlPts in format described before
106 *Output: Writes a file of 3 matrices one for each coordinate
107 */
108 void Write_CtrlPts(const std::vector<iMatrix<double>> &CtrlPts, std::string name);
109
110
111
112 /*****
113 *
114 * Classic funtions from the Nurbs Book
115 *
116 *****/
117
118 /*
119 *Input: A KnotVector,t the point of evaluation, its polynomial degree, and n=the size of the Nurbs vector-p-2
120 *Output: the span index
121 */
122 int FindSpan(const std::vector<double> &KnotVector, double t, int p, int n);
123
124 /*
125 *Input: i span index, t eval point, p polinomial degree and KnotVector
126 *Output: vector of basis funcs
127 */
128 std::vector<double> BasisFuns(int i, double t, int p,const std::vector<double> &KnotVector);
129
130 /*
131 *Input: i span index, t eval point, p polinomial degree and KnotVector
132 *Output: iMatrix where the i-th row is the i-th degree basis fun
133 */
134 iMatrix<double> AllBasisFuns(int i, double t, int p,const std::vector<double> &KnotVector);
135
136 /*
137 *Input: i span index, t eval point, p polinomial degree, k derivate degree, KnotVector and an output vector ders;
138 *Output: value of the k-derivate at point t in the ders vector on the k-row of the matrix
139 */
140 iMatrix<double> DerBasisFuns(int span, double t, int p, int k,const std::vector<double> &KnotVector);
141
142 /*
143 *Input: matrix of BasisFunctions or its derivatives and the path where they will be written
144 *Output: Writes the Functions as a matrix on selected path
145 */
146 void Write_BasisFunctions(const iMatrix<double> &BasisFunctions, std::string name);
147
148 /*****
149 *
150 * Classic funtions for p spline curves/surfaces
151 *
152 *****/
153
154 /*
155 *Input: n=the size of the Nurbs vector-p-2, p polynomial degree, KnotVector, the CtrlPts ,t the point of evaluation
156 *Output: Evaluation of the curve at point t
157 */
158 std::vector<double> CurvePoint(int n, int p,const std::vector<double> &KnotVector,const iMatrix<double> &CtrlPts, double t);
159
160 /*
161 *Input: n=the size of the Nurbs vector-p-2, p polynomial degree, KnotVector, the CtrlPts ,
162 *      t the point of evaluation and d highest derivate order to compute
163 *Output: iMatrix where the i-th row is the evaluation of the i-th derivate
164 */
165 iMatrix<double> CurveDerivsAlg1(int n, int p,const std::vector<double> &KnotVector,const iMatrix<double> &CtrlPts, double t, int d);
166
167 /*
168 *Input: n=the size of the Nurbs vector-p-2, p polynomial degree, KnotVector, the CtrlPts ,t the point of evaluation,
169 *      d highest derivate order to compute, r1 and r2 specify the intervals of CtrlPoints

```

```

170 *Output: iMatrix where the i-th row is the vector of ctrl points
171 */
172 iMatrix<std::vector<double>> CurveDerivsCpts(int n, int p, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPts,
173 int d, int r1, int r2);
174
175 /*
176 *Input: n=the size of the Nurbs vector-p-2, p polynomial degree, KnotVector, the CtrlPts ,
177 * t the point of evaluation and d highest derivate order to compute
178 *Output: iMatrix where the i-th row is the evaluation of the i-th derivate
179 */
180 iMatrix<double> CurveDerivsAlg2(int n, int p, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPts, double t, int d);
181
182 /*
183 *Input: Curve and path
184 *Output: Writes the values of the Curve in the selected path
185 */
186 void Write_Curve(const iMatrix<double> &Curve, std::string filename);
187
188 /*
189 *Input: n_i=the size of the Nurbs vector_i-p-2, p_i polynomial degree, KnotVector_i, the CtrlPts ,t_i the point of evaluation
190 *Output: Evaluation of the surface at point (t1,t2)
191 */
192 std::vector<double> SurfacePoint(int n1, int p1, const std::vector<double> &KnotVector1, int n2, int p2, const std::vector<double> &KnotVector2,
193 const std::vector<iMatrix<double>> &CtrlPts, double t1, double t2);
194
195 /*
196 *Input: n_i=the size of the Nurbs vector_i-p-2, p_i polynomial degree, KnotVector_i, the CtrlPts ,
197 * t_i the point of evaluation and d maximum order of the derivatives
198 *Output: Evaluation of the derivatives up to degree d (maximum is p or q and would overwrite it if d>p,q) in (t1,t2) where Output[k_1][k_2] is been
199 derived k_i times respect to the ith variable
200 */
201 iMatrix<std::vector<double>> SurfaceDerivsAlg1(int n1, int p1, const std::vector<double> &KnotVector1, int n2, int p2,
202 const std::vector<double> &KnotVector2, const std::vector<iMatrix<double>> &CtrlPts, double t1, double t2, int d);
203
204
205 /*****
206 *
207 * Classic and support funtions for NURBS curves/surfaces
208 *
209 *****/
210
211
212 /*
213 *Input: Vector of CtrlPts and their weights at each point in a separated vector
214 *Output: iMatrix that each column is each of the coordinates of each CtrlPt except its last component that is it's respective weight
215 */
216 iMatrix<double> WeightCtrlPts(const iMatrix<double> &CtrlPts, const std::vector<double> &Weights);
217
218 /*
219 *Input: CtrlPts multiplied by their Weight with the respective weight at the end of each column
220 *Output: The original CtrlPts
221 */
222 iMatrix<double> UnWeightCtrlPts(const iMatrix<double> &CtrlPtsW);
223
224 /*
225 *Input: Vector of CtrlPts and their weights at each point in a separated vector
226 *Output: std::vector<iMatrix<double>> that keeps the format of the CtrlPts for Surface
227 operations with and extra 4th dimension on each coord thats's occupied by the weight of such pt
228 */
229 std::vector<iMatrix<double>> WeightCtrlPtsSurf(const std::vector<iMatrix<double>> &CtrlPts, const iMatrix<double> &Weights);
230
231 /*
232 *Input: CtrlPts multiplied by their Weight with the respective weight at the end of each column
233 *Output: The original CtrlPts
234 */
235 std::vector<iMatrix<double>> UnWeightCtrlPtsSurf(const std::vector<iMatrix<double>> &CtrlPtsW);
236
237 /*
238 *Input: n=the size of the Nurbs vector-p-2, p polynomial degree, KnotVector, the CtrlPts weighted with theris weight in the last component ,
239 * t the point of evaluation
240 *Output: Evaluation of the NURBS curve at point t
241 */
242 std::vector<double> CurvePointRational(int n, int p, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPtsWeighted, double t);
243
244 /*
245 *Input: binomial coeficients k, i
246 *Output: combinatorial number given by upper number k and lower number i
247 */
248 int Binomial(int k, int i);
249
250 /*
251 *Input: i span index, t eval point, p polinomial degree and KnotVector, weights vector
252 *Output: vector of rational basis funcs
253 */
254 std::vector<double> RationalBasisFuns(int i, double t, int p, const std::vector<double> &KnotVector, const std::vector<double> &WeightVector);
255
256 /*
257 *Input: i span index, t eval point, k derivate degree, p polinomial degree and KnotVector, weights vector
258 *Output: vector of rational basis funcs
259 */
260 iMatrix<double> RatDersBasisFuns(int i, double t, int p, int k, const std::vector<double> &KnotVector, const std::vector<double> &WeightVector);
261
262 /*
263 *Input: i span index, t eval point, k derivate degree, p polinomial degree and KnotVector, iMatrix of the derivatives of the non-rational
264 * basis funs and weights vector
265 *Output: vector of rational basis funcs
266 */
267 iMatrix<double> RationalizeDersBasisFuns(int i, double t, int p, int k, const std::vector<double> &KnotVector, const iMatrix<double> &ders ,
268 const std::vector<double> &WeightVector);
269
270 /*
271 *Input: Aders: derivatives of the CtrlPtsWeighted using CurvDersAlg1 evaluated at some t, wders: same as Aders but with the weighted vector,
272 * maximum degree of the derivate

```

```

273 *           -Aders = CurveDerivsAlgi(n, p,KnotVector, WeightedCtrlPts, t, d);
274 *           -wders = CurveDerivsAlgi(n, p,KnotVector, WeightVector, t, d);
275 *Output: iMatrix on which each row is the i-th derivate of the rational curve made of the weighted CtrlPts and the weight vector
276 */
277 iMatrix<double> RatCurveDerivs(const iMatrix<double> &Aders,const iMatrix<double> &wders, int d);
278
279 /*
280 *Input: n_i=the size of the Nurbs vector_i-p-2, p_i polynomial degree, KnotVector_i, the CtrlPtsWeighted ,t_i the point of evaluation
281 *Output: Evaluation of the surface at point (t1,t2)
282 */
283 std::vector<double> SurfacePointRational(int n1, int p1,const std::vector<double> &KnotVector1, int n2, int p2,
284     const std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPtsWeighted, double t1, double t2);
285
286 /*
287 *Input: derivate of the CtrlPtsWeighted using CurvDersAlgi evaluated at some t, wders: same as Aders but with the weighted vector,
288 *       maximum degree of the derivate
289 *Output: iMatrix on which each row is the i-th derivate of the rational surface made of the weighted CtrlPts and the weight vector
290 */
291 iMatrix<std::vector<double>> RatSurfaceDerivs(const iMatrix<std::vector<double>> &Aders,const iMatrix<double> &wders,int d);
292
293 /*****
294 *
295 * Fundamental Geometric Algorithms
296 *
297 *****/
298
299 /*
300 *Input: KnotVector and element you want to introduce
301 *Output: index i such that KnotVector[i-1] <= e < KnotVector[i]
302 *Note: Applying insertion with this index using KnotVector.insert(KnotVector.begin()+index+1, r, e) will yield the KnotVector
303 *       we want (r is number of insertions)
304 */
305 int findInsertionIndex(const std::vector<double> &KnotVector, double e);
306
307 /*
308 *Input: ordered vector and element you want to check it's number of occurrences
309 *Output: number of times e appears on the vector
310 */
311 int countOccurrences(const std::vector<double> &v, double e);
312
313 /*
314 *Input: A KnotVector,t the point of evaluation, its polynomial degree, and n=the size of the Nurbs vector-p-2
315 *Output: the span index at k, its multiplicity at s
316 */
317 void FindSpanMult(const std::vector<double> &KnotVector, double t, int p, int n,int &k, int &s);
318
319 /*
320 *Input: np number of initial CtrlPts-1, p polynomial degree, the initial KnotVector, the initial CtrlPts Weighted, u value of the inserted knot,
321 *       index such that KnotVector[k]<= u < KnotVector[k+1], s initial multiplicity of u in KnotVector, r number of insertions of u in Knotvector
322 *Output: New CtrlPts Weighted that preserves the initial curve
323 *Note: !! KnotVector will be updated !!
324 */
325 iMatrix<double> CurveKnotsIns(int np, int p, std::vector<double> &KnotVector,const iMatrix<double> &CtrlPtsW, double u, int index, int s, int r);
326
327 /*
328 *Input: n=the size of the Knot vector-p-2, p polynomial degree, KnotVector, CtrlPts Weighthed, u point of evaluation
329 *Output: Evaluation of the Curve given by the KnotVector and the ControlPts at the given point of evaluation u
330 */
331 std::vector<double> CurvePntByCornerCut(int n, int p, std::vector<double> &KnotVector, iMatrix<double> &CtrlPtsW, double u);
332
333 /*
334 *Input: npi size of KnotVectori-pi-2, pi degree of KnotVectori, KnotVectori for i=1,2, CtrlPts Weighthed, dir(true= knot inserted in Knotvector1,
335 *       false= knot inserted in Knotvector2, uv knot we want to insert, index such that KnotVectori[k]<= u < KnotVectori[k+1], s initial
336 *       multiplicity of u in KnotVectori, r number of insertions of u in Knotvectori
337 *Output: the KnotVectori will be updated and it will return new CtrlPtsW such that they generate the same surface.
338 */
339 std::vector<iMatrix<double>> SurfaceKnotIns(int np1, int p1, std::vector<double> &KnotVector1, int np2, int p2,
340     std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPtsW, bool dir,
341     double uv, int index, int r, int s);
342
343 /*
344 *Input: n=Knotvector.size()-2-p, p polynomial degree, KnotVector, CtrlPtsW, Insertions: vector of new elements to insert into KnotVector,
345 *       r=Insertion.size()-1.
346 *Output: Update of the Knotvector with the new inserions, CtrlPtsW that generate the same curve with the new KnotVector.
347 */
348 iMatrix<double> RefineKnotVectCurve(int n, int p, std::vector<double> &KnotVector,const iMatrix<double> &CtrlPtsW,
349     const std::vector<double> &Insertions, int r);
350
351 /*
352 *Input: npi size of KnotVectori-1, pi degree of KnotVectori, KnotVectori for i=1,2, CtrlPts Weighthed, dir(true= knot inserted in Knotvector1,
353 *       false= knot inserted in Knotvector2, Insertions: vector of new elements to insert into KnotVector, r=Insertion.size()-1.
354 *Output: the KnotVectori will be updated and it will return new CtrlPtsW such that they generate the same surface.
355 */
356 std::vector<iMatrix<double>> RefineKnotVectSurface(int np1, int p1, std::vector<double> &KnotVector1, int np2, int p2,
357     std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPtsW, bool dir,
358     const std::vector<double> &Insertions, int r);
359
360 /*****
361 *
362 * Functions for the Assembly of the Matrix
363 *
364 *****/
365
366 /*
367 *Input: n: n2 pts of integration
368 *Output: nodes will contain the roots of the legendre polynomial and weights its weights (all in double precision).
369 */
370 void legendre_pol(int n, Eigen::VectorXd &nodes, Eigen::VectorXd &weights);
371
372 /*
373 *Input: ordered vector
374 *Output: the original vector without duplicates
375 */

```

```

376 std::vector<double> removeDuplicates(const std::vector<double>& input);
377
378 /*
379 *Input: number of elements of evaluation
380 *Output: vector of nElements+1 uniformly distributed
381 */
382 std::vector<double> createSubIntervals(const double nElements, const double lower_limit, const double upper_limit);
383
384 /*
385 *Input: v1 vector where we remove the elements that appear in v2, v2 KnotVector
386 *Output: v1 without the elements in v2
387 */
388 std::vector<double> removeMatches(const std::vector<double>& v1, const std::vector<double>& v2);
389
390 /*
391 *Input: n=the size of the Knot vector-p-2, i span index, t eval point, p polinomial degree, number of points of evaluations in the integral,
392 *      KnotVector, upper and lower limits of the integration, weights vector, nodes of the legendre polynomial, CtrlPtsW the control
393 *      points weighted, the function f and the previous evalPoint.
394 *Output: BasisFunsEvals and DerBasisFunsEvals each row will store the evaluation of the value or derivate of the basis fun alongside each
395 *      evaluation point, same for the JacobianEvals, InverseJacobianEvals and funcEvals.
396 */
397 void Di_element_eval(int n, int i, int p, int nEvals, double lower_limit, double upper_limit, const std::vector<double> &KnotVector,
398                  const std::vector<double> &Weights, const Eigen::VectorXd &nodes, const iMatrix<double> &CtrlPtsW, std::function<double(double)> f,
399                  double &preEvalPoint, double &cumLenght, iMatrix<double> &BasisFunsEvals, iMatrix<double> &DerBasisFunsEvals, std::vector<double> &JacobianEvals,
400                  std::vector<double> &InverseJacobianEvals, std::vector<double> &funcEvals, double Lenght=1.0);
401
402 /*
403 *Input: p polinomial degree, upper and lower limits of the integration, number of points of evaluations in the integral,
404 *      evaluations of RationalDerivates, evaluation of the Inverse Jacobian and the legendre weights.
405 *Output: Matrix of all the permutations of evaluations where the elements i,j of the matrix correspond to
406 *      the BasisFuns(init+i)*BasisFuns(init+j) where init=spanIndex()
407 */
408 iMatrix<double> gauss_legendre_cuadrature_integral_bilinearForm(int p, double lower_limit, double upper_limit, int nEvals,
409                  const iMatrix<double> &DerBasisFunsEvals, const std::vector<double> &InverseJacobianEvals, const Eigen::VectorXd &weights);
410
411 /*
412 *Input: p polinomial degree, upper and lower limits of the integration, number of points of evaluations in the integral,
413 *      evaluations of RationalBasisFuns, evaluations of the Jacobian, evaluations of the function and the legendre weights.
414 *Output: Vector of evaluations which its components are l(index) where init=spanIndex()
415 */
416 std::vector<double> gauss_legendre_cuadrature_integral_linealForm(int p, double lower_limit, double upper_limit,
417                  int nEvals, const iMatrix<double> &BasisFunsEvals, const std::vector<double> &JacobianEvals, const std::vector<double> &funcEvals,
418                  const Eigen::VectorXd &weights);
419
420 /*
421 *Input: p polinomial degree, size=LinearForm.size()-1, SparseMatrix with the system already assambled,
422 *      same with the LinearForm, Modify0(1) true if we have dirichlet condition at the start(end) of the boundary,
423 *      vValueAt0(1) value set for the boundary condition, default is set at 0.
424 *Output: Modification of the sparse matrix and the linear form to keep the system as symetric as possible.
425 */
426 void impose_Dirichlet_Condition(int p, int size, Eigen::SparseMatrix<double> &global, Eigen::VectorXd &LinearForm,
427                  bool Modify0, bool Modify1, double ValueAt0=0.0, double ValueAt1=0.0);
428
429 /*
430 *Input: size=LinearForm.size()-1, LinearForm, Modify0(1) true if we have dirichlet condition at the start(end) of the boundary,
431 *      vValueAt0(1) value set for the boundary condition, default is set at 0.
432 *Output: Modification of the linear form applying the conditions to the borders.
433 */
434 void impose_Newmann_Condition(int size, Eigen::VectorXd &LinearForm, bool Modify0, bool Modify1, double ValueAt0=0.0, double ValueAt1=0.0);
435
436 /*
437 *Input: p polinomial degree, size=LinearForm.size()-1, SparseMatrix with the system already assambled, same with the LinearForm,
438 *      Modify0(1) true if we have dirichlet condition at the start(end) of the boundary,
439 *      vValueAt0(1) value set for the boundary condition, default is set at 0.
440 *      values alpha and beta come from the expresion alpha*u(x)+beta*u'(x)=ValueAtx.
441 *Output: Modification of the sparse matrix and the linear form to keep the system as symetric as possible.
442 */
443 void impose_Robin_Condition(int p, int size, Eigen::SparseMatrix<double> &global, Eigen::VectorXd &LinearForm, double alpha,
444                  double beta, bool Modify0, bool Modify1, double ValueAt0=0.0, double ValueAt1=0.0);
445
446 /*
447 *Input: vector of evaluations of the function, name of hte file where we write the evaluations
448 *Output: writes the evaluation of the function in a txt
449 */
450 void writeNumericFunction(Eigen::VectorXd funEvals, const std::vector<double> &KnotVector, const std::vector<double> &WeightVector,
451                  int n, int p, int nEvals, double lowerLimit, double upperLimit, std::string name,
452                  std::vector<double> Intervals, int nElements, double Lenght);
453
454 /*
455 *Input: the function we evaluate, lower and upper limits of evaluation and number of steps of the evaluation, name of the
456 *      file where we write the evaluations, for the evaluation of the curve point, CtrlPtsW, KnotVector, n = KnotVector.size()-p-2,
457 *      p polinomial degree.
458 *Output: writes the evaluation of the function in a txt
459 */
460 void writeAnalyticFunction(std::function<double(double)> f, std::function<double(double)> f_Ders, std::vector<double> Intervals, double nEvals,
461                  const iMatrix<double> &CtrlPtsW, const std::vector<double> &KnotVector, int n, int p, std::string name, double Lenght, double nElements);
462
463 /*
464 *Input: name of the file, p polinomial degree
465 *Output: vector of evaluations on each elements loaded
466 */
467 std::vector<std::vector<double>> leerArchivo(const std::string& name, int p);
468
469 /*
470 *Input: vector of evaluations of the function, name of hte file where we write the evaluations
471 *Output: writes the evaluation of the derivates of the function in a txt
472 */
473
474 /*
475 void writeFunctionDers(Eigen::VectorXd funEvals, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPtsW,
476                  const std::vector<double> &WeightVector, int n, int p, int nEvals, double lowerLimit, double upperLimit, std::string name);
477 */
478 /*
479 *Input: the function we evaluate, lower and upper limits of evaluation and number of steps of the evaluation, name of hte file

```

```

479         where we write the evaluations, for the evaluation of the curve point, CtrlPtsW, KnotVector, n = KnotVector.size()-p-2, p polynomial degree.
480 *Output: writes the evaluation of the function in a txt
481 */
482 /*
483 void writeFunctionDers(std::function<double(double)> f, double lowerLimit, double upperLimit, double nEvals,
484     const iMatrix<double> &CtrlPtsW, const std::vector<double> &KnotVector,
485     int n, int p, std::string name, double Lenght=1.0);
486 */
487 /*
488 *Input: lowerLimit of integration, upperLimit of integration, number of subintervals for the quadrature, CtrlPtsW, Knotvector, n, p
489 *Output: numerical value of the integral between lowerLimit and upperLimit of the norm of the derivate of the curve
490 */
491 double IntegrateNormDer(double lowerLimit, double upperLimit, int nEvals, const iMatrix<double> &CtrlPtsW,
492     const std::vector<double> &KnotVector, int n, int p);
493
494 /*
495 *Input: e fisical parameter, lowerLimit of integration, upperLimit of integration, number of subintervals for the quadrature,
496     CtrlPtsW, Knotvector, n, p
497 *Output: numerical value parametric space parameter such that the analytic solution of it its equal to the fisical solution at e
498 */
499 inline double Fisical_to_Parametric(double t2, double t1, double lowerLimit, double Lenght,
500     int nEvals, const iMatrix<double> &CtrlPtsW, const std::vector<double> &KnotVector, int n, int p){
501     return IntegrateNormDer(t2,t1,nEvals, CtrlPtsW, KnotVector, n, p)/Lenght;
502 }
503
504 /*
505 *Input: i span index, p polynomial degree, k maximum derivate order, KnotVector, Intervals, nodes and weights
506     of the gauss-legendre cuadrature, vector of storage with spanIndex(memory has to be reserved before execution).
507 *Output: vector that each element is a vector that evaluates the basis functions in each of the corresponding gauss-legendre
508     points of each interval, modification of the vector spanIndex.
509 */
510 std::vector<std::vector<iMatrix<double>>> PreBasis_and_Ders(int p, int k, std::vector<double> KnotVector, std::vector<double> Intervals,
511     Eigen::VectorXd nodes, std::vector<int> & spanIndex);
512
513 /*
514 *Input: span index, nEvals number of points of the gauss-legendre cuadrature alongside the corresponding variable, nodes,
515     vector of the Gauss-Legendre cuadrature, lower_limit, upper_limit of the interval, p polynomial degree, k maximum derivate order, KnotVector,
516     Basis_and_DersY vector of the precomputed Basis and its corresponding derivatives, nodes and weights of the gauss-legendre cuadrature
517     and Weights vector of the corresponding weights alongside the corresponding variables.
518 *Output: vector that each element is thee evaluations of the basis functions and its derivatives up to k-degree in each of the corresponding
519     gauss-legendre points of the interval.
520 */
521 std::vector<iMatrix<double>> RationalizeDersBasisFunsVec(int span, int nEvals, Eigen::VectorXd nodes, double lower_limit, double upper_limit,
522     int p, int k, int n, std::vector<double> KnotVector, std::vector<iMatrix<double>>> Basis_and_DersY, std::vector<double> Weights);
523
524 /*
525 *Input: vector of Intervals, and vector of the correlated indexes for each interval
526 *Output: unordered map such that given a span index returns its corresponding interval.
527 */
528 std::unordered_map<int, std::vector<std::pair<double, double>>> BuildSpanMap(const std::vector<double> & Intervals, const std::vector<int> & spanIndices);
529
530 /*
531 *Input: vector of Intervals, and vector of the correlated indexes for each interval
532 *Output: unordered map such that given a span index returns its corresponding interval.
533 */
534 iMatrix<iMatrix<double>> RationalizeDersBasisFuns2D(const iMatrix<double> &DersBasisX, const iMatrix<double> &DersBasisY,
535     const iMatrix<double> &activeWeights, unsigned int pX, unsigned int pY);
536
537 /*
538 *Input: SolEvals and Surface evaluations of the same points with the filename and the path
539 *Output: writes the corresponding .txt with the Surface and its evaluations.
540 */
541 void ExportToTxt(const iMatrix<double> & SolEvals, const iMatrix<std::vector<double>> & Surf, const std::string & filename);
542 #endif
543

```

Fichero funciones.cpp

```

1  /*****
2  *AUTHOR: Carlos Palomera Oliva
3  *DATE: MARCH 2025
4  *****/
5  #include "funciones.hpp"
6
7  /*****
8
9  *****/
10 iMatrix<double> ReadDataFile_Matrix(std::string direction){
11     double Nrow, Ncol;
12     std::string line;
13     std::ifstream DataSet;
14
15     DataSet.open(direction);
16     if(DataSet.is_open()){
17
18         if(std::getline(DataSet, line, ' ')){
19             Nrow = stod(line);
20         }
21         if(std::getline(DataSet, line, '\n')){
22             Ncol = stod(line);
23         }
24
25         iMatrix<double> matrix(Nrow, Ncol);
26
27         for(unsigned int i=0; i<Nrow; i++){
28             for(unsigned int j=0; j<Ncol-1; j++){
29                 if(std::getline(DataSet, line, ' ')){
30                     matrix(i, j)=stod(line);
31                 }
32             }
33         }
34     }
35 }

```

```

33         }
34         if (std::getline(DataSet, line, '\n')){
35             matrix(i, Ncol-1)=stod(line);
36         }
37     }
38     DataSet.close();
39     return matrix;
40 }
41 else{
42     throw std::invalid_argument("Cannot open Data File.");
43     iMatrix<double> m;
44     return m;
45 }
46
47
48 }
49 /***** */
50
51 /***** */
52 std::vector<iMatrix<double>> ReadDataFile_CtrlPts(std::string direction){
53     double Ncol, NSplits;
54     std::string line;
55     std::ifstream DataSet;
56
57     DataSet.open(direction);
58     if (DataSet.is_open()){
59
60         if (std::getline(DataSet, line, ' ')){
61             Ncol = stod(line);
62         }
63         if (std::getline(DataSet, line, '\n')){
64             NSplits = stod(line);
65         }
66
67         std::vector<iMatrix<double>> CtrlPts(NSplits);
68         for (unsigned int k=0; k<NSplits; k++){
69
70             CtrlPts[k].Resize(3, Ncol);
71
72             for (unsigned int i=0; i<3; i++){
73
74                 for (unsigned int j=0; j<Ncol-1; j++){
75
76                     if (std::getline(DataSet, line, ' ')){
77
78                         CtrlPts[k](i, j)=stod(line);
79                     }
80                 }
81                 if (std::getline(DataSet, line, '\n')){
82                     CtrlPts[k](i, Ncol-1)=stod(line);
83                 }
84             }
85             getline(DataSet, line, '\n');
86         }
87     }
88
89     DataSet.close();
90     return CtrlPts;
91 }
92
93 else{
94     throw std::invalid_argument("Cannot open Data File.");
95     std::vector<iMatrix<double>> T;
96     return T;
97 }
98 }
99
100
101 /***** */
102
103 /***** */
104 std::vector<double> BernsteinPolynomial(double t, int n){
105     std::vector<double> Bnew(n);
106     std::vector<double> Bold(n);
107     std::fill(Bold.begin(), Bold.end(), 0);
108     Bold[0]=1;
109     double t1=1-t;
110     for (unsigned int j=1; j<n; j++){
111
112         std::fill(Bnew.begin(), Bnew.end(), 0);
113
114         for (unsigned int i=0; i<j; i++){
115
116             Bnew[i]=Bnew[i]+t1*Bold[i];
117             Bnew[i+1]=Bnew[i+1]+t*Bold[i];
118         }
119
120         Bold=Bnew;
121     }
122     return Bnew;
123 }
124
125 }
126
127 /***** */
128
129 /***** */
130 std::vector<double> BezierCurve_Point_t(const iMatrix<double> &PtsControl, double t){
131     int dim = PtsControl.GetNumRows();
132     std::vector<double> Curve(dim);
133     int n=PtsControl.GetNumCols();
134     std::vector<double> B = BernsteinPolynomial(t, n);
135     for (unsigned int j=0; j<dim; j++){

```

```

136         for(unsigned int i=0; i<n;i++){
137             Curve[j]=Curve[j]+B[i]*PtosControl(j,i);
138         }
139     }
140     return Curve;
141 }
142
143 /***** */
144
145 /***** */
146 iMatrix<double> BezierCurve(const iMatrix<double> &PtosControl, double N_Steps){
147
148     iMatrix<double> Curve(PtosControl.GetNumRows(),N_Steps+1);
149     double Step=1/(N_Steps);
150     for(unsigned int i=0; i<N_Steps+1;i++){
151         Curve.SetCol(i,BezierCurve_Point_t(PtosControl,i*Step));
152     }
153     return Curve;
154 }
155
156 /***** */
157
158 /***** */
159 std::vector<double> BezierSurface_Point_t1_t2(const std::vector<iMatrix<double>> &PtosControl, double t1, double t2){
160     int dim = 3;
161     std::vector<double> Curve(dim);
162
163     int N_Splits = PtosControl.size();
164     int n=PtosControl[0].GetNumCols();
165     std::vector<double> B1 = BernsteinPolinomial(t1,N_Splits);
166     std::vector<double> B2 = BernsteinPolinomial(t2,n);
167
168     for(unsigned int k=0; k<N_Splits;k++){
169         for(unsigned int j=0; j<dim; j++){
170             for(unsigned int i=0; i<n;i++){
171                 Curve[j]=Curve[j]+B1[k]*B2[i]*PtosControl[k](j,i);
172             }
173         }
174     }
175
176     return Curve;
177 }
178
179 /***** */
180
181 /***** */
182 std::vector<iMatrix<double>> BezierSurface(const std::vector<iMatrix<double>> &PtosControl, double N_Steps){
183     std::vector<iMatrix<double>> Surface(N_Steps+1);
184     double Step=1/(N_Steps);
185
186     for(unsigned int k=0; k<N_Steps+1;k++){
187         Surface[k].Resize(3,N_Steps+1);
188         for(unsigned int i=0; i<N_Steps+1;i++){
189             Surface[k].SetCol(i,BezierSurface_Point_t1_t2(PtosControl,i*Step,k*Step));
190         }
191     }
192     return Surface;
193 }
194
195 /***** */
196
197 /***** */
198
199 void Write_BezierSurface(const std::vector<iMatrix<double>> &Tensor, std::string name){
200     std::ofstream fichero;
201     char coord[3]={'X','Y','Z'};
202     int iterj = Tensor[0].GetNumCols();
203     int iterk = Tensor.size();
204
205     for(unsigned int i=0; i<3;i++){
206         fichero.open(name+coord[i]+".txt");
207         for(unsigned int j=0; j<iterj;j++){
208             for(unsigned int k=0; k<iterk;k++){
209                 fichero<<Tensor[k](i,j)<<" ";
210             }
211             fichero<<std::endl;
212         }
213         fichero.close();
214     }
215 }
216
217 /***** */
218
219 /***** */
220
221 void Write_CtrlPts(const std::vector<iMatrix<double>> &CtrlPts, std::string name){
222     std::ofstream fichero;
223     char coord[3]={'X','Y','Z'};
224     int iterj = CtrlPts[0].GetNumCols();
225     int iterk = CtrlPts.size();
226
227     for(unsigned int i=0; i<3;i++){
228         fichero.open(name+coord[i]+".txt");
229         for(unsigned int j=0; j<iterj;j++){
230             for(unsigned int k=0; k<iterk;k++){
231                 fichero<<CtrlPts[k](i,j)<<" ";
232             }
233             fichero<<std::endl;
234         }
235         fichero.close();
236     }
237 }
238

```

```

239 }
240
241
242
243 /*****
244 *
245 * Classic funtions from the Nurbs Book
246 *
247 *****/
248
249 int FindSpan(const std::vector<double> &KnotVector, double t, int p, int n){
250
251     if (KnotVector[n+1]==t){
252         return n;
253     }
254
255     else{
256         int low =p;
257         int high=n+1;
258         int mid=(low+high)/2;
259         while (t<KnotVector[mid] || t>=KnotVector[mid+1]){
260             if (t<double(KnotVector[mid])){
261                 //std::cout<<"Condición t<U[mid]"<< "t="<<t<< " mid="<<mid<< " , U[mid]="<< KnotVector[mid]<<std::endl;
262                 high=mid;
263             }
264             else{
265                 //std::cout<<"Condición t>=U[mid]"<< "t="<<t<< " mid="<<mid<< " , U[mid]="<< KnotVector[mid]<<std::endl;
266                 low=mid;
267             }
268             mid=(low+high)/2;
269         }
270         return mid;
271     }
272 }
273
274 /***** */
275
276 /***** */
277 std::vector<double> BasisFuns(int i, double t, int p, const std::vector<double> &KnotVector){
278     std::vector<double> sol(p+1);
279     std::vector<double> left(p+1);
280     std::vector<double> right(p+1);
281     double temp,saved;
282     //std::cout<<"Entramos en basisFuns"<<std::endl;
283     sol[0]=1.0f;
284     for(unsigned int j=1; j<=p; j++){
285         //std::cout<< "j: " << j<< std::endl;
286         left[j]=t-KnotVector[i+1-j];
287         right[j]=KnotVector[i+j]-t;
288         saved = 0.0f;
289         for(unsigned int r=0; r<j; r++){
290             //std::cout<< "r: " << r<< std::endl;
291             temp= sol[r]/(right[r+1]+left[j-r]);
292             sol[r]=saved+right[r+1]*temp;
293             saved=left[j-r]*temp;
294         }
295         sol[j]=saved;
296     }
297     //std::cout<<"Salimos de basisFuns"<<std::endl;
298     return sol;
299 }
300
301 /***** */
302
303 /***** */
304
305
306
307 iMatrix<double> AllBasisFuns(int i, double t, int p, const std::vector<double> &KnotVector){
308
309
310     iMatrix<double> sol(p+1,p+1);
311     std::vector<double> left(p+1);
312     std::vector<double> right(p+1);
313     sol(0,0)=1.0f;
314     double temp,saved;
315
316     for(unsigned int j=1; j<=p; j++){
317
318         left[j]=t-KnotVector[i+1-j];
319         right[j]=KnotVector[i+j]-t;
320         saved = 0.0f;
321         for(unsigned int r=0; r<j; r++){
322
323             temp= sol(j-1,r)/(right[r+1]+left[j-r]);
324             sol(j,r)=saved+right[r+1]*temp;
325             saved=left[j-r]*temp;
326         }
327         sol(j,j)=saved;
328     }
329
330     return sol;
331 }
332
333
334 /***** */
335
336 /***** */
337 iMatrix<double> DerBasisFuns(int span, double t, int p, int k, const std::vector<double> &KnotVector){
338     iMatrix<double> ders(k+1,p+1);
339     iMatrix<double> ndu(p+1,p+1);
340     double temp,saved;
341     ndu(0,0)=1.0f;

```

```

342     std::vector<double> left(p + 1), right(p + 1);
343
344     for (int j = 1; j <= p; ++j) {
345         left[j] = t - KnotVector[span + 1 - j];
346         right[j] = KnotVector[span + j] - t;
347         double saved = 0.0;
348
349         for (int r = 0; r < j; ++r) {
350             ndu(j,r)= right[r + 1] + left[j - r];
351             double temp= ndu(r,j-1)/ndu(j,r);
352             ndu(r,j)=saved + right[r + 1] * temp ;
353             saved = left[j - r] * temp;
354         }
355         ndu(j,j)=saved;
356     }
357
358     for (int j = 0; j <= p; ++j) {
359         ders(0,j)=ndu(j,p);
360     }
361
362
363     for (int r = 0; r <= p; ++r) {
364         int s1 = 0, s2 = 1;
365         iMatrix<double> a(2,p+1);
366         a(0,0)=1.0f;
367
368         for (int i = 1; i <= k; ++i) {
369             double d = 0.0;
370             int rk = r - i, pk = p - i;
371             if (r >= i) {
372                 a(s2,0)= a(s1,0)/ndu(pk+1,rk);
373                 d= a(s2,0)*ndu(rk,pk);
374             }
375             int j1 = (rk >= -1) ? 1 : -rk;
376             int j2 = (r - 1 <= pk) ? i - 1 : p - r;
377
378             for (int j = j1; j <= j2; ++j) {
379                 a(s2,j)= (a(s1,j)-a(s1,j-1))/ndu(pk+1,rk+1) ;
380                 d += a(s2,j)*ndu(rk+j,pk);
381             }
382             if (r <= pk) {
383                 a(s2,i)= -a(s1,i-1)/ndu(pk+1,r) ;
384                 d += a(s2,i)*ndu(r,pk);
385             }
386             ders(i,r)=d;
387             std::swap(s1, s2);
388         }
389     }
390 }
391
392 // Ajustar multiplicadores
393 int r = p;
394 for (int i = 1; i <= k; ++i) {
395     for (int j = 0; j <= p; ++j) {
396         ders(i,j)= ders(i,j)*r;
397     }
398     r *= (p - i);
399 }
400
401 return ders;
402 }
403
404 /*****
405
406 *****/
407 void Write_BasisFunctions(const iMatrix<double> &BasisFunctions, std::string name){
408     std::ofstream fichero;
409     fichero.open(name+".txt");
410     for(unsigned int j=0; j<BasisFunctions.GetNumRows();j++){
411         for(unsigned int k=0; k<BasisFunctions.GetNumCols();k++){
412             fichero<<BasisFunctions(j,k)<<" ";
413         }
414         fichero<<std::endl;
415     }
416     fichero.close();
417 }
418
419 /*****
420 *
421 * Classic funtions for p spline curves/surfaces
422 *
423 *****/
424
425 std::vector<double> CurvePoint(int n, int p,const std::vector<double> &KnotVector,const iMatrix<double> &CtrlPts, double t){
426     int span =FindSpan(KnotVector, t, p, n);
427     std::vector<double> Funs = BasisFuns (span,t,p,KnotVector);
428
429     int itermax=CtrlPts.GetNumRows();
430     std::vector<double> C(itermax);
431     for(unsigned int i = 0; i<=p;i++){
432         double aux=Funs[i];
433
434         for(unsigned int j=0;j<itermax;j++){
435
436             C[j]=C[j] + aux*(CtrlPts(j,span-p+i));
437
438         }
439     }
440
441     return C;
442 }
443
444

```

```

445 /***** */
446
447 /***** */
448 iMatrix<double> CurveDerivsAlg1(int n, int p, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPts, double t, int d){
449     int du= std::min(d,p);
450     int span =FindSpan(KnotVector, t, p, n);
451     int itermax=CtrlPts.GetNumRows();
452     //std::cout << "du " << du<< std::endl;
453     iMatrix<double> CK(du+1, itermax); //inicializa a 0 no hace falta dar valores
454     iMatrix<double> nders= DerBasisFuns(span, t, p, du, KnotVector);
455     for(unsigned int k=0; k<=du;k++){
456         for(unsigned int i=0; i<=p;i++){
457             double aux =nders(k,i);
458
459             //CK[k] = CK[k] + nders[k] [j]*P[span-p+j]
460             for(unsigned int j=0;j<itermax;j++){
461
462                 CK(k,j)+=aux*(CtrlPts(j,span-p+i));
463
464             }
465         }
466     }
467     return CK;
468 }
469
470 /***** */
471
472 /***** */
473 iMatrix<std::vector<double>> CurveDerivsCpts(int n, int p, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPts, int d,
474 int r1, int r2){
475     int r= r2-r1;
476     iMatrix<std::vector<double>> sol(d+1,r+1);
477     for(unsigned int i=0;i<=r;i++){
478         sol(0,i)= CtrlPts.GetCol(r1+i);
479     }
480     for(unsigned int k=1;k<=d;k++){
481         int tmp = p-k+1;
482         for(unsigned int i=0;i<=r-k;i++){
483
484             int iter=CtrlPts.GetNumCols(); //dimension del espacio
485             std::vector<double> aux(iter);
486             double denominator=KnotVector[r1+i+p+1]-KnotVector[r1+i+k];
487
488             for(unsigned int j=0; j<iter; j++){
489                 aux[j]=tmp*(sol(k-1,i+1)[j]-sol(k-1,i)[j])/denominator;
490             }
491
492             sol(k,i)= aux ;
493         }
494     }
495     return sol;
496 }
497
498 /***** */
499
500 /***** */
501
502 iMatrix<double> CurveDerivsAlg2(int n, int p, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPts, double t, int d){
503     int du= std::min(d,p);
504     int span =FindSpan(KnotVector, t, p, n);
505     int itermax=CtrlPts.GetNumRows(); //dimension del espacio
506     iMatrix<double> CK(itermax, du); //inicializa a 0 no hace falta dar valores
507     iMatrix<double> nders= AllBasisFuns(span, t, p, KnotVector);
508     iMatrix<std::vector<double>> DervCpts=CurveDerivsCpts(n,p,KnotVector,CtrlPts,du,span-p,span);
509     for(unsigned int k=0; k<=du;k++){
510         for(unsigned int i=0; i<=p-k;i++){
511             double aux =nders(i,p-k);
512             for(unsigned int j=0;j<itermax;j++){
513
514                 CK(k,j)=CK(k,j)+aux*DervCpts(k,i)[j] ;
515
516             }
517         }
518     }
519     return CK;
520 }
521
522 /***** */
523
524 /***** */
525
526 void Write_Curve(const iMatrix<double> &Curve, std::string filename){
527     std::ofstream fichero;
528     int iterMaxi=Curve.GetNumCols();
529     int iterMaxj=Curve.GetNumRows();
530     fichero.open(filename);
531     for(unsigned int j=0; j<iterMaxj;j++){
532         for(unsigned int i=0; i<iterMaxi;i++){
533             fichero<<Curve(j,i)<<" ";
534         }
535         fichero<< std::endl;
536     }
537     fichero.close();
538 }
539
540 /***** */
541
542 /***** */
543
544 std::vector<double> SurfacePoint(int n1, int p1, const std::vector<double> &KnotVector1, int n2, int p2,
545 const std::vector<double> &KnotVector2, const std::vector<iMatrix<double>> &CtrlPts, double t1, double t2){
546     int span1=FindSpan(KnotVector1,t1,p1,n1);
547     int span2=FindSpan(KnotVector2,t2,p2,n2);

```

```

548     std::vector<double> Basis1=BasisFuns (span1,t1,p1,KnotVector1);
549     std::vector<double> Basis2=BasisFuns (span2,t2,p2,KnotVector2);
550     int ind1 = span1-p1;
551     std::vector<double> Pt= {0,0,0};
552     for(unsigned int l=0; l<=p2; l++){
553
554         std::vector<double> temp= {0,0,0};
555
556         int ind2= span2 - p2+1;
557         for(unsigned int k=0;k<=p1;k++){
558             double auxBasis1= Basis1[k];
559             //std::vector<double> auxCtrlPt = CtrlPts (ind1+k,ind2);
560             for(unsigned int i=0; i<3;i++){
561                 //std::cout<<"i="<<i<<" ,k="<<k<<" ,l="<<l<<std::endl;
562                 temp[i]=temp[i]+auxBasis1*CtrlPts [ind1+k] (i,ind2);
563             }
564         }
565
566         double auxBasis2=Basis2[l];
567         for(unsigned int i=0; i<3;i++){
568             Pt [i]=Pt [i]+auxBasis2*temp [i];
569         }
570     }
571     return Pt;
572 }
573
574 /***** */
575
576 /***** */
577 iMatrix<std::vector<double>> SurfaceDerivsAlg1(int n1, int p1,const std::vector<double> &KnotVector1, int n2, int p2,
578 const std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPts, double t1, double t2, int d){
579     unsigned int du=std::min(d,p1);
580     unsigned int dv=std::min(d,p2);
581
582     unsigned int dim = CtrlPts[0].GetNumRows();
583     //std::cout<<"dim: "<<dim<<std::endl;
584     iMatrix<std::vector<double>> Sol(du+1,dv+1);
585     const std::vector<double> zero(dim);
586     for(unsigned int k=0; k<=du; k++){
587         for(unsigned int j=0; j<=dv;j++){
588             Sol(k,j)= zero ;
589         }
590     }
591     //std::cout<<"2" <<std::endl;
592     int span1=FindSpan(KnotVector1,t1,p1,n1);
593     int span2=FindSpan(KnotVector2,t2,p2,n2);
594
595     iMatrix<double> Ders1 = DerBasisFuns (span1, t1, p1, du, KnotVector1);
596     iMatrix<double> Ders2 = DerBasisFuns (span2, t2, p2, dv, KnotVector2);
597     iMatrix<double> temp(dim,p2+1);
598     //std::cout<<"3" <<std::endl;
599     for(unsigned int k=0; k<=du; k++){
600         //std::cout<<"4" <<std::endl;
601
602         for(unsigned int s=0;s<=p2;s++){
603             //std::cout <<"s: "<<s <<std::endl;
604             temp.SetCol(s,zero);
605             for(unsigned int r=0; r<=p1; r++){
606                 // std::cout<<"6" <<std::endl;
607                 //temp[s] = temp[s] + Nu[k] [r]*P[uspan-p+r] [vspan-q+s];
608                 double aux= Ders1(k,r);
609                 for(unsigned int j=0; j<dim;j++){
610                     // std::cout<<"7" <<std::endl;
611                     //std::cout<< CtrlPts[span1-p1+r] (j,span2 -p2+s) <<"|";
612                     temp(j,s)= temp(j,s)+aux*CtrlPts [span1-p1+r] (j,span2 -p2+s);
613                 }
614             }
615         }
616         //std::cout <<"-----" <<std::endl;
617     }
618     //std::cout <<"8" <<std::endl;
619     int dd=std::min(d-k,dv);
620     for(unsigned int l=0; l<= dd; l++){
621         Sol(k,l)=zero;
622         for(unsigned int s=0;s<=p2;s++){
623             double aux = Ders2(l,s);
624             std::vector<double> auxSol= Sol(k,l);
625             std::vector<double> aux2(dim);
626             for(unsigned int j=0; j<dim; j++){
627                 aux2[j]=auxSol[j]+aux*temp(j,s);
628             }
629             Sol(k,l)= aux2;
630         }
631     }
632 }
633 }
634 }
635
636 return Sol;
637 }
638
639 /*****
640 *
641 * Classic and support funtions for NURBS curves/surfaces
642 *
643 *****/
644 iMatrix<double> WeightCtrlPts(const iMatrix<double> &CtrlPts,const std::vector<double> &Weights){
645     int dim= CtrlPts.GetNumRows()+1;
646     int size = CtrlPts.GetNumCols();
647     iMatrix<double> CtrlPtsWeighted(dim,size);
648     for(unsigned int i=0; i<size;i++){
649         for(unsigned int j=0; j<dim-1; j++){
650             CtrlPtsWeighted(j,i)=CtrlPts (j,i)*Weights [i];

```

```

651         }
652         CtrlPtsWeighted(dim-1,i)=Weights[i];
653     }
654     return CtrlPtsWeighted;
655 }
656
657 /***** */
658 /***** */
659
660 iMatrix<double> UnWeightCtrlPts(const iMatrix<double> &CtrlPtsW){
661     int NumPts = CtrlPtsW.GetNumCols();
662     int dim = CtrlPtsW.GetNumRows()-1;
663     iMatrix<double> CtrlPts(dim,NumPts);
664     for(unsigned int j=0; j< NumPts; j++){
665         for(unsigned int i=0; i<dim; i++){
666             if (CtrlPtsW(dim,j)==0){
667                 //nothing, the declaracion automatically sets it to 0
668             }
669             else{
670                 CtrlPts(i,j)=CtrlPtsW(i,j)/CtrlPtsW(dim,j);
671             }
672         }
673     }
674     return CtrlPts;
675 }
676
677 /***** */
678 /***** */
679
680 std::vector<iMatrix<double>> WeightCtrlPtsSurf(const std::vector<iMatrix<double>> &CtrlPts,const iMatrix<double> &Weights){
681     int dim = CtrlPts.size();
682     std::vector<iMatrix<double>> WeightedCtrlPts(dim);
683     for(unsigned int i = 0; i< dim; i++){
684         WeightedCtrlPts[i]=WeightCtrlPts(CtrlPts[i], Weights.GetRow(i));
685     }
686     return WeightedCtrlPts;
687 }
688
689 /***** */
690 /***** */
691
692 std::vector<iMatrix<double>> UnWeightCtrlPtsSurf(const std::vector<iMatrix<double>> &CtrlPtsW){
693     int NumPts2= CtrlPtsW.size();
694     int NumPts1 = CtrlPtsW[0].GetNumCols();
695     int dim = CtrlPtsW[0].GetNumRows()-1;
696
697     std::vector<iMatrix<double>> CtrlPts(NumPts2);
698
699     for(unsigned int k=0; k<NumPts2; k++){
700         CtrlPts[k]=iMatrix<double>(dim, NumPts1);
701         //iMatrix<double> AuxMat (dim,NumPts1);
702
703         for(unsigned int j=0; j< NumPts1; j++){
704             for(unsigned int i=0; i<dim; i++){
705                 if (CtrlPtsW[k](dim,j)==0){
706                     //nothing, the declaracion automatically sets it to 0
707                 }
708                 else{
709                     CtrlPts[k](i,j)=CtrlPtsW[k](i,j)/CtrlPtsW[k](dim,j);
710                 }
711             }
712         }
713     }
714
715     return CtrlPts;
716 }
717
718 /***** */
719 /***** */
720
721 std::vector<double> CurvePointRational(int n, int p,const std::vector<double> &KnotVector,
722 const iMatrix<double> &CtrlPtsWeighted, double t){
723     int span =FindSpan(KnotVector, t, p, n);
724     std::vector<double> Funs = BasisFuns(span,t,p,KnotVector);
725     int dim= CtrlPtsWeighted.GetNumRows();
726     std::vector<double> Cw(dim, 0.0f);
727     for(unsigned int j=0; j<=p; j++){
728         //Cw=Cw + Funs[j]*CtrlPtsWeighted[span-p+j]
729         for(unsigned int i=0; i<dim; i++){
730             Cw[i]=Cw[i]+Funs[j]*CtrlPtsWeighted(i,span-p+j);
731         }
732     }
733     std::vector<double> C(dim-1);
734     double w = Cw[dim-1];
735
736     for(unsigned int i=0; i<dim-1; i++){
737         C[i]=Cw[i]/w;
738     }
739     return C;
740 }
741
742 /***** */
743 /***** */
744
745

```

```

754 int Binomial(int k, int i){
755     if (i < 0 || i > k) return 0;
756     if (i == 0 || i == k) return 1;
757     if (i > k - i) i = k - i; // simetria
758
759     long long res = 1;
760     for (int j = 1; j <= i; j++) {
761         res *= k - (i - j);
762         res /= j;
763     }
764     return (int)res;
765 }
766
767 /***** */
768
769 /***** */
770 std::vector<double> RationalBasisFuns(int i, double t, int p, const std::vector<double> &KnotVector,
771     const std::vector<double> &WeightVector){
772     std::vector<double> Funs = BasisFuns(i, t, p, KnotVector);
773     int itermax=Funs.size();
774     double coef=0.0f;
775     //std::cout<<itermax<<std::endl;
776     for(int j=0; j<itermax;j++){
777         //std::cout<<"a"<<std::endl;
778         coef=coef+Funs[j]*WeightVector[i-p+j];
779     }
780     for(int j=0; j<itermax;j++){
781         //std::cout<<"b"<<std::endl;
782         Funs[j]=Funs[j]*WeightVector[i-p+j]/coef;
783     }
784
785     return Funs;
786 }
787
788 /***** */
789
790 /***** */
791 iMatrix<double> RatDersBasisFuns(int i, double t, int p, int k, const std::vector<double> &KnotVector,
792     const std::vector<double> &WeightVector){
793     iMatrix<double> R_ders(k+1, p+1);
794     iMatrix<double> ders = DerBasisFuns(i, t, p, k, KnotVector);
795
796     std::vector<double> Wk(k+1, 0.0);
797     for (int j = 0; j <= k; ++j){
798         for (int iter = 0; iter <= p; iter++){
799             Wk[j] += WeightVector[i-p+iter] * ders(j, iter);
800             //std::cout << "Weight1: " << WeightVector[i-p+iter] << " ";
801         }
802     }
803     //std::cout << std::endl;
804
805     for (int a = 0; a < p+1; ++a) {
806         R_ders(0, a) = ders(0, a)*WeightVector[i-p+a]/Wk[0]; // (D(0, a) * weights[a]) / Wk[0];
807         for (int j = 1; j <= k; ++j) {
808             double sum = 0.0;
809             for (int m = 1; m <= j; ++m){
810                 sum += Binomial(j, m) * Wk[m] * R_ders(j - m, a);
811             }
812             double num = ders(j, a) * WeightVector[i-p+a] - sum;
813             R_ders(j, a) = num / Wk[0];
814         }
815     }
816
817     return R_ders;
818 }
819
820 }
821
822 /***** */
823
824 /***** */
825 iMatrix<double> RationalizeDersBasisFuns(int i, double t, int p, int k, const std::vector<double> &KnotVector,
826     const iMatrix<double> &ders, const std::vector<double> &WeightVector){
827     iMatrix<double> R_ders(k+1, p+1);
828
829
830     std::vector<double> Wk(k+1, 0.0);
831     for (int j = 0; j <= k; ++j){
832         for (int iter = 0; iter <= p; iter++){
833             Wk[j] += WeightVector[i-p+iter] * ders(j, iter);
834         }
835     }
836
837     for (int a = 0; a < p+1; ++a) {
838         R_ders(0, a) = ders(0, a)*WeightVector[i-p+a]/Wk[0]; // (D(0, a) * weights[a]) / Wk[0];
839         for (int j = 1; j <= k; ++j) {
840             double sum = 0.0;
841             for (int m = 1; m <= j; ++m){
842                 sum += Binomial(j, m) * Wk[m] * R_ders(j - m, a);
843             }
844             double num = ders(j, a) * WeightVector[i-p+a] - sum;
845             R_ders(j, a) = num / Wk[0];
846         }
847     }
848
849     return R_ders;
850 }
851
852 }
853
854 /***** */
855
856 /***** */

```

```

857 iMatrix<double> RatCurveDerivs(const iMatrix<double> &Aders,const iMatrix<double> &wders, int d){
858     int NumCols=Aders.GetNumCols();
859     iMatrix<double> CK(d+1,NumCols);
860     //CK[0]=Aders[0]/wders[0]
861     //std::vector<double> init(NumCols);
862     for(unsigned int k=0; k<NumCols; k++){
863         CK(0,k)=Aders(0,k)/wders(0,0);
864     }
865
866
867     for(unsigned int k =1; k<=d; k++){
868         std::vector<double> v = Aders.GetRow(k);
869         for(int i=1;i<=k;i++){
870             int Bin=Binomial(k,i);
871             //std::cout << "k: " << k <<std::endl;
872             //std::cout << "i: " << i <<std::endl;
873             //std::cout << "binomial: " << Bin <<std::endl;
874             //v = v - Binomial(k, i) * wders[i] * CK[k - i];
875             for(unsigned int j=0; j<NumCols; j++){
876                 v[j]-=Bin*wders(i,0)*CK(k-i,j);
877             }
878         }
879         //CK[k] = v / wders[0];
880         for(unsigned int j=0; j<NumCols; j++){
881             v[j] /= wders(0,0);
882         }
883         CK.SetRow(k,v);
884     }
885
886     return CK;
887 }
888
889 /***** */
890
891 /***** */
892
893 std::vector<double> SurfacePointRational(int n1, int p1,const std::vector<double> &KnotVector1, int n2, int p2,
894     const std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPtsWeighted, double t1, double t2){
895     int span1=FindSpan(KnotVector1,t1,p1,n1);
896     int span2=FindSpan(KnotVector2,t2,p2,n2);
897     std::vector<double> Basis1=BasisFuns(span1,t1,p1,KnotVector1);
898     std::vector<double> Basis2=BasisFuns(span2,t2,p2,KnotVector2);
899     int ind1 = span1-p1;
900     int dim = CtrlPtsWeighted[0].GetNumRows();
901     std::vector<double> Pt(dim);
902     iMatrix<double> auxtemp(p2+1,dim);
903     for(unsigned int l=0; l<=p2; l++){
904
905         std::vector<double> temp(dim);
906
907         int ind2= span2 - p2+1;
908         for(unsigned int k=0;k<=p1;k++){
909             double auxBasis1= Basis1[k];
910             //std::vector<double> auxCtrlPt = CtrlPts(ind1+k,ind2);
911             for(unsigned int i=0; i<dim;i++){
912                 //std::cout<<"i="<<i<<","k="<<k<<","l="<<l<<std::endl;
913                 temp[i]=temp[i]+auxBasis1*CtrlPtsWeighted[ind1+k](i,ind2);
914             }
915         }
916         auxtemp.SetRow(l, temp);
917
918     }
919
920
921     for(unsigned int l=0; l<=p2;l++){
922         double auxBasis2=Basis2[l];
923         for(unsigned int i=0; i<dim;i++){
924             Pt[i]=Pt[i]+auxBasis2*auxtemp(l,i);
925         }
926     }
927
928     std::vector<double> C(dim-1);
929     double w = Pt[dim-1];
930
931     for(unsigned int i=0; i<dim-1; i++){
932         C[i]=Pt[i]/w;
933     }
934     return C;
935 }
936
937 /***** */
938
939 /***** */
940 iMatrix<std::vector<double>> RatSurfaceDerivs(const iMatrix<std::vector<double>> &Aders,const iMatrix<double> &wders,int d){
941
942     int dim=Aders(0,0).size();
943     iMatrix<std::vector<double>> SKL(d+1,d+1);
944
945     for(unsigned int k=0; k<=d; k++){
946         for(unsigned int l=0;l<=d-k;l++){
947             //v = Aders [k] [l] ;
948             std::vector<double> v = Aders(k,l);
949
950             for(unsigned int j=1; j<=l;j++){
951                 double binaux=Binomial(l,j);
952                 double Wdersaux=wders(0,j);
953
954
955
956                 for(unsigned int iter=0; iter < dim; iter++){
957
958                     //v = v - Bin[l] [j]*wders[0] [j]*SKL[k] [l-j];
959                     v[iter]=v[iter]-binaux*Wdersaux*SKL(k,l-j)[iter];

```

```

960         }
961     }
962
963     }
964
965     for(unsigned int i=1; i<=k; i++){
966         double binaux=Binomial(k,i);
967         double Wdersaux=wders(i,0);
968
969         for(unsigned int iter=0; iter < dim; iter++){
970             //v = v - Bin[k] [i]*wders[i] [0]*SKL[k-i] [1];
971             v[iter]=v[iter]-binaux*Wdersaux*SKL(k-i,1)[iter];
972         }
973
974         //v2 = 0.0;
975         std::vector<double> v2(Aders(0,0).size());
976         for(unsigned int j=1; j<=l;j++){
977             binaux=Binomial(l,j);
978             Wdersaux=wders(i,j);
979
980             for(unsigned int iter=0; iter < dim; iter++){
981                 //v2 = v2 + Bin[l] [j]*wders[i] [j]*SKL[k-i] [1-j];
982                 v2[iter]=v2[iter]+binaux*Wdersaux*SKL(k-i,1-j)[iter];
983             }
984
985             binaux=Binomial(k,i);
986             for(unsigned int iter=0; iter < dim; iter++){
987                 //v = v - Bin[k] [i]*v2;
988                 v[iter]=v[iter]-binaux*v2[iter];
989             }
990
991             }
992
993             //SKL[k] [1] = v/wders[0] [0];
994             std::vector<double> auxvec(dim);
995             for(unsigned int iter=0; iter < dim; iter++){
996                 auxvec[iter]=v[iter]/wders(0,0);
997             }
998
999             SKL(k,1)=auxvec;
1000         }
1001     }
1002
1003     }
1004
1005     return SKL;
1006 }
1007
1008 /*****
1009 *
1010 * Fundamental Geometric Algorithms and support functions
1011 *
1012 *****/
1013 int findInsertionIndex(const std::vector<double>& v, double e){
1014     auto it = std::upper_bound(v.begin(), v.end(), e);
1015     return it - v.begin()-1;
1016 }
1017
1018 /***** */
1019
1020 /***** */
1021
1022 int countOccurrences(const std::vector<double>& v, double e) {
1023     double eps = 1e-6f;
1024     int left = 0;
1025     int right = v.size() - 1;
1026     int index = -1;
1027
1028     //binary search
1029     while (left <= right) {
1030         int mid = left + (right - left) / 2;
1031         if (std::fabs(v[mid] - e) < eps) {
1032             index = mid;
1033             right = mid - 1;
1034         } else if (v[mid] < e) {
1035             left = mid + 1;
1036         } else {
1037             right = mid - 1;
1038         }
1039     }
1040
1041     if (index == -1) return 0; // Not found case
1042
1043     //linear count
1044     int count = 0;
1045     while (index + count < v.size() && std::fabs(v[index + count] - e) < eps) {
1046         count++;
1047     }
1048
1049     return count;
1050 }
1051
1052 /***** */
1053
1054 /***** */
1055 void FindSpanMult(const std::vector<double> &KnotVector, double t, int p, int n,int &k, int &s){
1056     double eps = 1e-6f;
1057     k=FindSpan(KnotVector,t,p,n);
1058     s=0;
1059     for (int i = k; i >= 0 && std::fabs(KnotVector[i] - t) < eps; --i) s++;
1060     for (int i = k + 1; i <= n + p + 1 && std::fabs(KnotVector[i] - t) < eps; ++i) s++;
1061 }
1062

```

```

1063  /***** */
1064
1065  /***** */
1066  iMatrix<double> CurveKnotsIns(int mp, int p, std::vector<double> &KnotVector, const iMatrix<double> &CtrlPtsW,
1067  double u, int index, int s, int r){
1068  //int mp = np+p+1;
1069  int dim=CtrlPtsW.GetNumRows();
1070  int L;
1071  double alpha;
1072
1073
1074  //New CtrlPts
1075  iMatrix<double> NewCtrlPtsW(dim, CtrlPtsW.GetNumCols()+r);
1076
1077  for(unsigned int i=0; i<=index-p; i++){
1078      NewCtrlPtsW.SetCol(i, CtrlPtsW.GetCol(i));
1079  }
1080
1081  for(unsigned int i=index-s; i<=mp; i++){
1082      NewCtrlPtsW.SetCol(i+r, CtrlPtsW.GetCol(i));
1083  }
1084
1085  iMatrix<double> AuxCtrlPts(dim, p+1);
1086
1087  for(unsigned int i=0; i<=p-s; i++){
1088      AuxCtrlPts.SetCol(i, CtrlPtsW.GetCol(index-p+i));
1089  }
1090
1091  for(unsigned int j=1; j<=r; j++){
1092
1093      L=index-p+j;
1094
1095      for(int i=0; i<=p-j-s; i++){
1096          alpha= (u-KnotVector[L+i])/(KnotVector[i+index+1]-KnotVector[L+i]);
1097
1098          for(unsigned int iter=0; iter<dim; iter++){
1099
1100              AuxCtrlPts(iter, i)=alpha*AuxCtrlPts(iter, i+1)+(1.0f-alpha)*AuxCtrlPts(iter, i);
1101          }
1102      }
1103
1104      NewCtrlPtsW.SetCol(L, AuxCtrlPts.GetCol(0));
1105
1106      NewCtrlPtsW.SetCol(index+r-j-s, AuxCtrlPts.GetCol(p-j-s));
1107
1108  }
1109  for(unsigned int i=L+1; i<index-s; i++){
1110
1111      NewCtrlPtsW.SetCol(i, AuxCtrlPts.GetCol(i-L));
1112  }
1113
1114  //Update KnotVector
1115  KnotVector.insert(KnotVector.begin()+index+1, r, u);
1116
1117  return NewCtrlPtsW;
1118 }
1119
1120  /***** */
1121
1122  /***** */
1123  std::vector<double> CurvePntByCornerCut(int n, int p, std::vector<double> &KnotVector, iMatrix<double> &CtrlPtsW, double u){
1124  int dim = CtrlPtsW.GetNumRows()-1;
1125  std::vector<double> aux(dim+1);
1126
1127  if(u==KnotVector[0]){
1128      std::vector<double> aux(dim);
1129      for(unsigned int i=0; i< dim; i++){
1130          aux[i]=CtrlPtsW(i,0)/CtrlPtsW(dim,0);
1131      }
1132      return aux;
1133  }
1134  if(u==KnotVector[n+p+1]){
1135      std::vector<double> aux(dim);
1136      for(unsigned int i=0; i< dim; i++){
1137          aux[i]=CtrlPtsW(i,n)/CtrlPtsW(dim,n);
1138      }
1139      return aux;
1140  }
1141  int k,s;
1142  FindSpanMult(KnotVector, u, p, n, k, s);
1143  int r = p-s;
1144  //std::cout<< "r: " << r << std::endl;
1145  iMatrix<double> AuxCtrlPts(dim+1, r+1);
1146  for(unsigned int i=0; i<=r; i++){
1147      AuxCtrlPts.SetCol(i, CtrlPtsW.GetCol(k-p+i));
1148  }
1149
1150
1151
1152  for(unsigned int j=1; j<=r; j++){
1153      for(unsigned int i=0; i<=r-j; i++){
1154          double alpha= (u-KnotVector[k-p+j+i])/(KnotVector[i+k+1]-KnotVector[k-p+j+i]);
1155
1156
1157          for(unsigned int iter=0; iter< dim+1; iter++){
1158              aux[iter]=alpha*AuxCtrlPts(iter, i+1)+(1.0f-alpha)*AuxCtrlPts(iter, i);
1159          }
1160          AuxCtrlPts.SetCol(i, aux);
1161          //AuxCtrlPts.PrintMatrix();
1162      }
1163  }
1164
1165  std::vector<double> sol(dim);

```

```

1166
1167     for(unsigned int i=0; i<dim; i++){
1168         sol[i]=AuxCtrlPts(i,0)/AuxCtrlPts(dim,0);
1169     }
1170     return sol;
1171 }
1172
1173 /***** */
1174
1175 /***** */
1176 std::vector<iMatrix<double>> SurfaceKnotIns(int np1, int p1, std::vector<double> &KnotVector1, int np2, int p2,
1177     std::vector<double> &KnotVector2, const std::vector<iMatrix<double>> &CtrlPtsW, bool dir,
1178         double uv, int index, int r, int s){
1179     if(dir==true){//KnotVector1 will be updated
1180         int dim=CtrlPtsW[0].GetNumRows();
1181         iMatrix<double> Aux(dim,p1-s+1);
1182         int numCtrlPts1=CtrlPtsW.size()+1;
1183         int L;
1184         iMatrix<double> alpha(p1-s+1,r+1);
1185         std::vector<iMatrix<double>> NewCtrlPts(numCtrlPts1);
1186         for(unsigned int i=0; i<numCtrlPts1; i++){
1187             NewCtrlPts[i]=iMatrix<double>(dim, CtrlPtsW[0].GetNumCols());
1188         }
1189
1190         for(unsigned int j=1; j<=p1-j-s; j++){
1191             L=index-p1+j;
1192             for(unsigned int i=0; i<=p1-j-s; i++){
1193                 alpha(i,j) = (uv-KnotVector1[L+i])/(KnotVector1[i+index+1]-KnotVector1[L+i]);
1194             }
1195         }
1196
1197         for(unsigned int row=0; row<= np2; row++){
1198             for(unsigned int i=0; i<=index-p1; i++){
1199                 NewCtrlPts[i].SetCol(row, CtrlPtsW[i].GetCol(row));
1200             }
1201             for(unsigned int i=index-s; i<=np1; i++){
1202                 NewCtrlPts[i+r].SetCol(row, CtrlPtsW[i].GetCol(row));
1203             }
1204             for(unsigned int i=0; i<=p1-s; i++){
1205                 Aux.SetCol(i, CtrlPtsW[index-p1+i].GetCol(row));
1206             }
1207
1208             for(unsigned int j=1; j<=r; j++){
1209                 L=index-p1+j;
1210                 for(unsigned int i=0; i<= p1-j-s; i++){
1211                     std::vector<double> aux(dim);
1212                     for(unsigned int iter=0; iter<dim; iter++){
1213                         aux[iter]= alpha(i,j)*Aux(iter,i+1)+(1.0f-alpha(i,j))*Aux(iter,i);
1214                     }
1215                     Aux.SetCol(i, aux);
1216                 }
1217                 NewCtrlPts[L].SetCol(row, Aux.GetCol(0));
1218                 NewCtrlPts[index+r-j-s].SetCol(row, Aux.GetCol(p1-j-s));
1219             }
1220             for(unsigned int i=L+1; i<index-s; i++){
1221                 NewCtrlPts[i].SetCol(row, Aux.GetCol(i-L));
1222             }
1223         }
1224         KnotVector1.insert(KnotVector1.begin()+index+1, r, uv);
1225         return NewCtrlPts;
1226     }
1227     else{//KnotVector2 will be updated
1228         int dim = CtrlPtsW[0].GetNumRows();
1229         iMatrix<double> Aux(dim, p2 - s + 1);
1230         int numCtrlPts2 = CtrlPtsW[0].GetNumCols() + 1;
1231         int L;
1232         iMatrix<double> alpha(p2 - s + 1, r + 1);
1233         std::vector<iMatrix<double>> NewCtrlPts(CtrlPtsW.size());
1234
1235         for (unsigned int i = 0; i < CtrlPtsW.size(); i++) {
1236             NewCtrlPts[i] = iMatrix<double>(dim, numCtrlPts2);
1237         }
1238
1239         for (unsigned int j = 1; j <= p2 - j - s; j++) {
1240             L = index - p2 + j;
1241             for (unsigned int i = 0; i <= p2 - j - s; i++) {
1242                 alpha(i, j) = (uv - KnotVector2[L + i]) / (KnotVector2[i + index + 1] - KnotVector2[L + i]);
1243             }
1244         }
1245
1246         for (unsigned int col = 0; col <= np1; col++) {
1247             for (unsigned int i = 0; i <= index - p2; i++) {
1248                 NewCtrlPts[col].SetCol(i, CtrlPtsW[col].GetCol(i));
1249             }
1250             for (unsigned int i = index - s; i <= np2; i++) {
1251                 NewCtrlPts[col].SetCol(i + r, CtrlPtsW[col].GetCol(i));
1252             }
1253
1254             for (unsigned int i = 0; i <= p2 - s; i++) {
1255                 Aux.SetCol(i, CtrlPtsW[col].GetCol(index - p2 + i));
1256             }
1257
1258             for (unsigned int j = 1; j <= r; j++) {
1259                 L = index - p2 + j;
1260                 for (unsigned int i = 0; i <= p2 - j - s; i++) {
1261                     std::vector<double> aux(dim);
1262                     for (unsigned int d = 0; d < dim; d++) {
1263                         aux[d] = alpha(i, j) * Aux(d, i + 1) +
1264                             (1.0f - alpha(i, j)) * Aux(d, i);
1265                     }
1266                     Aux.SetCol(i, aux);
1267                 }
1268             }
1269         }
1270     }

```

```

1269         NewCtrlPts[col].SetCol(L, Aux.GetCol(0));
1270         NewCtrlPts[col].SetCol(index + r - j - s, Aux.GetCol(p2 - j - s));
1271     }
1272
1273     for (unsigned int i = L + 1; i < index - s; i++) {
1274         NewCtrlPts[col].SetCol(i, Aux.GetCol(i - L));
1275     }
1276 }
1277
1278 KnotVector2.insert(KnotVector2.begin() + index + 1, r, uv);
1279
1280 return NewCtrlPts;
1281
1282 }
1283
1284 }
1285
1286
1287
1288 /***** */
1289
1290 /***** */
1291 iMatrix<double> RefineKnotVectCurve(int n, int p, std::vector<double> &KnotVector,
1292     const iMatrix<double> &CtrlPtsW, const std::vector<double> &Insertions, int r){
1293     int m=n+p+1;
1294     int dim=CtrlPtsW.GetNumRows();
1295     std::vector<double> NewKnotVector(m+r+2);
1296     //int numCtrlPts=CtrlPtsW.GetNumCols();
1297     iMatrix<double> NewCtrlPts(dim,n+r+2);
1298     int a=FindSpan(KnotVector,Insertions[0],p,n);
1299     int b=FindSpan(KnotVector,Insertions[r],p,n)+1;
1300
1301     for(unsigned int j=0; j<=a-p; j++){
1302         NewCtrlPts.SetCol(j, CtrlPtsW.GetCol(j));
1303     }
1304     for(unsigned int j=b-1; j<=n; j++){
1305         NewCtrlPts.SetCol(j+r+1, CtrlPtsW.GetCol(j));
1306     }
1307     for(unsigned int j=0; j<=a; j++){
1308         NewKnotVector[j]=KnotVector[j];
1309     }
1310     for(unsigned int j=b+p; j<=m; j++){
1311         NewKnotVector[j+r+1]=KnotVector[j];
1312     }
1313
1314     int i=b+p-1;
1315     int k=b+p+r;
1316     for(int j=r; j>=0; j--){
1317         while(Insertions[j]<=KnotVector[i]&&i>a){
1318             NewCtrlPts.SetCol(k-p-1, CtrlPtsW.GetCol(i-p-1));
1319             NewKnotVector[k]=KnotVector[i];
1320             k--;
1321             i--;
1322         }
1323         NewCtrlPts.SetCol(k-p-1, NewCtrlPts.GetCol(k-p));
1324         for(unsigned int l=1; l<=p; l++){
1325             int ind = k-p+1;
1326             double alpha=NewKnotVector[k+1]-Insertions[j];
1327             if (alpha==0.0){
1328                 NewCtrlPts.SetCol(ind-1, NewCtrlPts.GetCol(ind));
1329             }
1330             else{
1331                 alpha=alpha/(NewKnotVector[k+1]-KnotVector[i-p+1]);
1332                 std::vector<double> aux(dim);
1333                 for(unsigned int iter=0; iter<dim; iter++){
1334                     aux[iter]=alpha*NewCtrlPts(iter, ind-1)+(1.0f-alpha)*NewCtrlPts(iter, ind);
1335                 }
1336                 NewCtrlPts.SetCol(ind-1, aux);
1337             }
1338         }
1339     }
1340     NewKnotVector[k]=Insertions[j];
1341     k--;
1342 }
1343
1344 KnotVector=NewKnotVector;
1345 return NewCtrlPts;
1346 }
1347
1348 /***** */
1349
1350 /***** */
1351 std::vector<iMatrix<double>> RefineKnotVectSurface(int np1, int p1, std::vector<double> &KnotVector1,
1352     int np2, int p2, std::vector<double> &KnotVector2, const std::vector<iMatrix<double>> &CtrlPtsW, bool dir,
1353     const std::vector<double> Insertions, int r){
1354     int dim=CtrlPtsW[0].GetNumRows();
1355     if(dir==true){
1356         //int L;
1357         int numCtrlPts1=CtrlPtsW.size()+r+1;
1358         int numCtrlPts2=CtrlPtsW[0].GetNumCols();
1359         int a=FindSpan(KnotVector1,Insertions[0],p1,np1);
1360         int b=FindSpan(KnotVector1,Insertions[r],p1,np1)+1;
1361         int m=np1+p1+1;
1362         std::vector<double> NewKnotVector(m+r+2);
1363         std::vector<iMatrix<double>> NewCtrlPts(numCtrlPts1);
1364         for(unsigned int i=0; i<numCtrlPts1; i++){
1365             NewCtrlPts[i]=iMatrix<double>(dim, numCtrlPts2);
1366         }
1367
1368         for(unsigned int row=0; row<=m; row++){
1369             for(unsigned int k=0; k<=a-p1; k++){
1370                 NewCtrlPts[k].SetCol(row, CtrlPtsW[k].GetCol(row));
1371             }

```

```

1372     for(unsigned int k=b-1; k<=np1; k++){
1373         NewCtrlPts[k+r+1].SetCol(row, CtrlPtsW[k].GetCol(row));
1374     }
1375
1376 }
1377
1378 for(unsigned int j=0; j<=a; j++){
1379     NewKnotVector[j]=KnotVector1[j];
1380 }
1381 for(unsigned int j=b+p1; j<=m; j++){
1382     NewKnotVector[j+r+1]=KnotVector1[j];
1383 }
1384
1385 int i=b+p1-1;
1386 int k=b+p1+r;
1387
1388 for(int j=r; j>=0; j--){
1389     while(Insertions[j] <= KnotVector1[i] && i>a){
1390         for(unsigned int row=0; row<=m; row++){
1391             NewCtrlPts[k-p1-1].SetCol(row, CtrlPtsW[i-p1-1].GetCol(row));
1392         }
1393         NewKnotVector[k]=KnotVector1[i];
1394         k--;
1395         i--;
1396     }
1397
1398     for(unsigned int row=0; row<=m; row++){
1399         NewCtrlPts[k-p1-1].SetCol(row, NewCtrlPts[k-p1].GetCol(row));
1400     }
1401
1402     for(unsigned int l=1; l<=p1; l++){
1403         int ind=k-p1+l;
1404         double alpha=NewKnotVector[k+l]-Insertions[j];
1405         if(alpha==0.0){
1406             //NewCtrlPts.SetCol(ind-1, NewCtrlPts.GetCol(ind));
1407             for(unsigned int row=0; row<=m; row++){
1408                 NewCtrlPts[ind-1].SetCol(row, NewCtrlPts[ind].GetCol(row));
1409             }
1410         }
1411         else{
1412             alpha=alpha/(NewKnotVector[k+l]-KnotVector1[i-p1+l]);
1413             for(unsigned int row=0; row<=m; row++){
1414                 std::vector<double> aux(dim);
1415                 for(unsigned int iter=0; iter<dim; iter++){
1416                     aux[iter]=alpha*NewCtrlPts[ind-1](iter, row)+(1.0f-alpha)*NewCtrlPts[ind](iter, row);
1417                 }
1418                 NewCtrlPts[ind-1].SetCol(row, aux);
1419             }
1420         }
1421     }
1422 }
1423
1424 NewKnotVector[k]=Insertions[j];
1425 k--;
1426 }
1427
1428 KnotVector1=NewKnotVector;
1429 return NewCtrlPts;
1430 }
1431 else{
1432     int numCtrlPts1 = CtrlPtsW.size();
1433     int numCtrlPts2 = CtrlPtsW[0].GetNumCols() + r + 1;
1434     int a = FindSpan(KnotVector2, Insertions[0], p2, np2);
1435     int b = FindSpan(KnotVector2, Insertions[r], p2, np2) + 1;
1436     int m = np2 + p2 + 1;
1437
1438     std::vector<double> NewKnotVector(m + r + 2);
1439     std::vector<iMatrix<double>> NewCtrlPts(numCtrlPts1);
1440
1441     for (int i = 0; i < numCtrlPts1; i++) {
1442         NewCtrlPts[i] = iMatrix<double>(dim, numCtrlPts2);
1443     }
1444
1445     for (int col = 0; col <= a - p2; col++) {
1446         for (int i = 0; i < numCtrlPts1; i++) {
1447             NewCtrlPts[i].SetCol(col, CtrlPtsW[i].GetCol(col));
1448         }
1449     }
1450
1451     for (int col = b - 1; col <= np2; col++) {
1452         for (int i = 0; i < numCtrlPts1; i++) {
1453             NewCtrlPts[i].SetCol(col + r + 1, CtrlPtsW[i].GetCol(col));
1454         }
1455     }
1456
1457     for (int j = 0; j <= a; j++) {
1458         NewKnotVector[j] = KnotVector2[j];
1459     }
1460
1461     for (int j = b + p2; j <= m; j++) {
1462         NewKnotVector[j + r + 1] = KnotVector2[j];
1463     }
1464
1465     int i = b + p2 - 1;
1466     int k = b + p2 + r;
1467
1468     for (int j = r; j >= 0; j--) {
1469         while (Insertions[j] <= KnotVector2[i] && i > a) {
1470             for (int row = 0; row < numCtrlPts1; ++row) {
1471                 NewCtrlPts[row].SetCol(k - p2 - 1, CtrlPtsW[row].GetCol(i - p2 - 1));
1472             }
1473             NewKnotVector[k] = KnotVector2[i];
1474             k--;

```

```

1475         i--;
1476     }
1477
1478     for (int row = 0; row < numCtrlPts1; row++) {
1479         NewCtrlPts[row].SetCol(k - p2 - 1, NewCtrlPts[row].GetCol(k - p2));
1480     }
1481
1482     for (int l = 1; l <= p2; l++) {
1483         int ind = k - p2 + 1;
1484         double alpha = NewKnotVector[k + 1] - Insertions[j];
1485         if (alpha == 0.0f) {
1486             for (int row = 0; row < numCtrlPts1; row++) {
1487                 NewCtrlPts[row].SetCol(ind - 1, NewCtrlPts[row].GetCol(ind));
1488             }
1489         } else {
1490             alpha = alpha / (NewKnotVector[k + 1] - KnotVector2[i - p2 + 1]);
1491             for (int row = 0; row < numCtrlPts1; row++) {
1492                 std::vector<double> aux(dim);
1493                 for (int iter = 0; iter < dim; iter++) {
1494                     aux[iter] = alpha * NewCtrlPts[row](iter, ind - 1) + (1.0f - alpha) * NewCtrlPts[row](iter, ind);
1495                 }
1496                 NewCtrlPts[row].SetCol(ind - 1, aux);
1497             }
1498         }
1499     }
1500
1501     NewKnotVector[k] = Insertions[j];
1502     k--;
1503 }
1504
1505 KnotVector2 = NewKnotVector;
1506 return NewCtrlPts;
1507 }
1508 }
1509
1510 }
1511
1512 /*****
1513 *
1514 * Functions for the Assembly of the Matrix
1515 *
1516 *****/
1517 void legendre_pol(int n, Eigen::VectorXd &nodes, Eigen::VectorXd &weights) {
1518
1519     Eigen::VectorXd beta(n-1);
1520     for (int i = 0; i < n-1; ++i) {
1521         double k = i + 1;
1522         beta(i) = k / std::sqrt(4.0 * k * k - 1);
1523     }
1524
1525     Eigen::MatrixXd J = Eigen::MatrixXd::Zero(n, n);
1526     for (int i = 0; i < n-1; ++i) {
1527         J(i, i+1) = beta(i);
1528         J(i+1, i) = beta(i);
1529     }
1530
1531     Eigen::SelfAdjointEigenSolver<Eigen::MatrixXd> solver(J);
1532     nodes = solver.eigenvalues();
1533
1534     Eigen::MatrixXd V = solver.eigenvectors();
1535     weights.resize(n);
1536     for (int i = 0; i < n; ++i) {
1537         weights(i) = 2.0 * V(0, i) * V(0, i);
1538     }
1539 }
1540
1541
1542 /***** */
1543
1544 /***** */
1545 std::vector<double> removeDuplicates(const std::vector<double>& input) {
1546     std::vector<double> result = input;
1547     auto last = std::unique(result.begin(), result.end());
1548     result.erase(last, result.end());
1549     return result;
1550 }
1551
1552 /***** */
1553
1554 /***** */
1555 std::vector<double> createSubIntervals(const double nElements, const double lower_limit, const double upper_limit){
1556     std::vector<double> sol(nElements+1);
1557     double Lenght = upper_limit - lower_limit;
1558     double Step=Lenght/nElements;
1559     for(unsigned int i=0; i<=nElements; i++){
1560         sol[i]=lower_limit + i*Step;
1561     }
1562     return sol;
1563 }
1564
1565 /***** */
1566
1567 /***** */
1568 std::vector<double> removeMatches(const std::vector<double>& v1, const std::vector<double>& v2 ){
1569     if (v1.empty()) return {};
1570
1571     double start = v1.front();
1572     double step = v1.size() > 1 ? v1[1] - v1[0] : 1;
1573
1574     std::vector<char> toRemove(v1.size(), 0);
1575
1576     for (size_t i = 0; i < v2.size(); ++i) {
1577         double val = v2[i];

```

```

1578         if (val < start) continue;
1579
1580         double diff = val - start;
1581         double idxReal = diff / step;
1582         long long idx = llround(idxReal);
1583
1584         if (std::fabs(idxReal - idx) > 1e-9) continue;
1585
1586         if (idx >= 0 && static_cast<size_t>(idx) < v1.size()) {
1587             toRemove[idx] = 1;
1588         }
1589     }
1590
1591     std::vector<double> result;
1592     result.reserve(v1.size());
1593     for (size_t i = 0; i < v1.size(); ++i) {
1594         if (!toRemove[i]) result.push_back(v1[i]);
1595     }
1596
1597     return result;
1598 }
1599
1600 /***** */
1601
1602 /***** */
1603
1604 void D1_element_eval(int n, int i, int p, int nEvals, double lower_limit, double upper_limit, const std::vector<double> &KnotVector,
1605     const std::vector<double> &Weights, const Eigen::VectorXd &nodes, const iMatrix<double> &CtrlPtsW, std::function<double(double)> f,
1606     double &preEvalPoint, double &cumLenght, iMatrix<double> &BasisFunsEvals, iMatrix<double> &DerBasisFunsEvals,
1607     std::vector<double> &JacobianEvals, std::vector<double> &InverseJacobianEvals, std::vector<double> &funcEvals, double Lenght){
1608
1609     BasisFunsEvals.Resize(p+1,nEvals);
1610     DerBasisFunsEvals.Resize(p+1,nEvals);
1611     JacobianEvals.resize(nEvals);
1612     InverseJacobianEvals.resize(nEvals);
1613     funcEvals.resize(nEvals);
1614     double auxeval1= (upper_limit - lower_limit)/2;
1615     double auxeval2= (upper_limit + lower_limit)/2;
1616
1617     for(unsigned int k=0; k<nEvals; k++){
1618         //std::cout << k << std::endl;
1619         double EvalPoint = auxeval1*nodes(k)+auxeval2;
1620         //std::cout << "EvalPoint: " << EvalPoint << std::endl;
1621         iMatrix<double> AuxMat1 = RatDersBasisFuns(i, EvalPoint, p, 1, KnotVector, Weights);
1622         BasisFunsEvals.SetCol(k, AuxMat1.GetRow(0));
1623         DerBasisFunsEvals.SetCol(k, AuxMat1.GetRow(1));
1624         //std::cout << "i1" << std::endl;
1625         iMatrix<double> Aders, wders;
1626         iMatrix<double> Allders = CurveDerivsAlg1(n, p, KnotVector, CtrlPtsW, EvalPoint, 1);
1627         //std::cout << "i2" << std::endl;
1628         int auxInt=Allders.GetNumCols()-1;
1629         Aders = Allders.GetSubMat(0,1,0, auxInt-1);
1630         wders = Allders.GetSubMat(0,1,auxInt, auxInt);
1631         iMatrix<double> AuxMat2 = RatCurveDerivs(Aders, wders, 1);
1632         //std::cout << "i3" << std::endl;
1633
1634         std::vector<double> AuxVec= AuxMat2.GetRow(1);
1635         double AuxVal=0;
1636         for(unsigned int iter=0; iter< AuxVec.size(); iter++){
1637             AuxVal+= AuxVec[iter]*AuxVec[iter];
1638         }
1639         AuxVal=sqrt(AuxVal);
1640         //std::cout << "i4" << std::endl;
1641         JacobianEvals[k]=AuxVal;
1642         InverseJacobianEvals[k]=1/AuxVal;
1643         double FisicalP=Fisical_to_Parametric(preEvalPoint, EvalPoint, 0, Lenght, 20, CtrlPtsW, KnotVector, n, p);
1644         cumLenght += FisicalP;
1645         preEvalPoint=EvalPoint;
1646         //std::cout << std::setprecision(7) << "EvalPoint= " << cumLenght << std::endl;
1647         //funcEvals[k]=f(EvalPoint)/(M_PI *M_PI ); //Case Jacobian is constant at 1
1648         //funcEvals[k] = 10*10*M_PI*M_PI*sin(10*M_PI*CurvePointRational(n, p, KnotVector, CtrlPtsW, EvalPoint)[0]);
1649         //funcEvals[k] = f(CurvePointRational(n, p, KnotVector, CtrlPtsW, EvalPoint)[0]); //Case unidimensional curve
1650         funcEvals[k]= f(cumLenght); //Generic case (optimization needed)
1651     }
1652 }
1653
1654 /***** */
1655
1656 /***** */
1657
1658 iMatrix<double> gauss_legendre_cuadrature_integral_bilinealForm(int p, double lower_limit, double upper_limit, int nEvals,
1659     const iMatrix<double> &DerBasisFunsEvals, const std::vector<double> &InverseJacobianEvals, const Eigen::VectorXd &weights){
1660     iMatrix<double> sol(p+1,p+1);
1661     double AuxConstant = (upper_limit-lower_limit)/2;
1662
1663     for(unsigned int i=0; i<p; i++){
1664         for(unsigned int j=0; j<p; j++){
1665             double AuxVal=0;
1666             for(unsigned int iter=0; iter<nEvals; iter++){
1667                 AuxVal+=weights[iter]*DerBasisFunsEvals(i, iter)*DerBasisFunsEvals(j, iter)*InverseJacobianEvals[iter];
1668             }
1669             AuxVal=AuxConstant*AuxVal;
1670
1671             sol(i,j)=AuxVal;
1672             //sol(j,i,AuxVal);
1673         }
1674     }
1675     return sol;
1676 }
1677
1678 /***** */
1679
1680 /***** */

```

```

1681 std::vector<double> gauss_legendre_cuadrature_integral_linealForm(int p, double lower_limit, double upper_limit,
1682 int nEvals, const iMatrix<double> &BasisFunsEvals, const std::vector<double>& JacobianEvals,
1683 const std::vector<double>& funcEvals, const Eigen::VectorXd &weights){
1684 std::vector<double> sol(p+1);
1685 double AuxConstant = (upper_limit-lower_limit)/2;
1686
1687 for(unsigned int i=0; i<=p; i++){
1688     double AuxVal=0;
1689     for(unsigned int iter=0; iter<nEvals; iter++){
1690         AuxVal+=weights[iter]*BasisFunsEvals(i, iter)*JacobianEvals[iter]*funcEvals[iter];
1691     }
1692     sol[i]=AuxConstant*AuxVal;
1693 }
1694 return sol;
1695 }
1696
1697
1698 /***** */
1699
1700 /***** */
1701 void impose_Dirichlet_Condition(int p, int size, Eigen::SparseMatrix<double> &global, Eigen::VectorXd &LinearForm,
1702 bool Modify0, bool Modify1, double ValueAt0, double ValueAt1){
1703     if (Modify0){
1704
1705         global.coeffRef(0, 0) = 1.0;
1706         LinearForm(0)=ValueAt0;
1707         for(unsigned int i=1; i<=p; i++){
1708             global.coeffRef(i, 0) = 0.0;
1709             LinearForm(i)-=ValueAt0*global.coeffRef(0, i);
1710             global.coeffRef(0, i) = 0.0;
1711         }
1712     }
1713
1714     if (Modify1){
1715
1716         global.coeffRef(size, size) = 1.0;
1717         LinearForm(size)=ValueAt1;
1718         for(int i=size-1; i>=size-p; i--){
1719             global.coeffRef(size, i) = 0.0;
1720             LinearForm(i)-=ValueAt1*global.coeffRef(i, size);
1721             global.coeffRef(i, size) = 0.0;
1722         }
1723     }
1724 }
1725
1726 /***** */
1727
1728 /***** */
1729 void impose_Neumann_Condition(int size, Eigen::VectorXd &LinearForm, bool Modify0, bool Modify1, double ValueAt0, double ValueAt1){
1730     if (Modify0){
1731         LinearForm(0)-=ValueAt0;
1732     }
1733     if (Modify1){
1734         LinearForm(size)+=ValueAt1;
1735     }
1736 }
1737
1738 /***** */
1739
1740 /***** */
1741 void impose_Robin_Condition(int p, int size, Eigen::SparseMatrix<double> &global, Eigen::VectorXd &LinearForm,
1742 double alpha, double beta, bool Modify0, bool Modify1, double ValueAt0, double ValueAt1){
1743
1744     double BetaInverse=1/beta;
1745
1746     if (Modify0){
1747
1748         global.coeffRef(0, 0) -= alpha/beta;
1749         LinearForm(0)-=ValueAt0/beta;
1750     }
1751
1752     if (Modify1){
1753
1754         global.coeffRef(size, size) += alpha/beta;
1755         LinearForm(0)+=ValueAt1/beta;
1756     }
1757 }
1758
1759 /***** */
1760
1761 /***** */
1762 void writeNumericFunction(Eigen::VectorXd funEvals, const std::vector<double> &KnotVector, const std::vector<double> &WeightVector, int n,
1763 int p, int nEvals, double lowerLimit, double upperLimit, std::string name, std::vector<double> Intervals, int nElements, double Lenght){
1764
1765     std::ofstream fichero1, fichero2;
1766     fichero1.open(name+"_h=" + std::to_string(static_cast<int>(nElements)) + "_p=" + std::to_string(p)+"_testCircle.txt");
1767     fichero2.open(name+"_Ders_h=" + std::to_string(static_cast<int>(nElements)) + "_p=" + std::to_string(p)+"_testCircle.txt");
1768
1769     // Configurar alta precisión
1770     fichero1 << std::scientific << std::setprecision(15);
1771     fichero2 << std::scientific << std::setprecision(15);
1772     Eigen::VectorXd nodes, weights;
1773     legendre_pol(nEvals, nodes, weights);
1774
1775     for(unsigned int iter=0; iter<Intervals.size()-1; iter++){
1776
1777         double auxeval1= (Intervals[iter+1] - Intervals[iter])/2;
1778         double auxeval2= (Intervals[iter+1] + Intervals[iter])/2;
1779         //double preVal=Intervals[iter];
1780
1781
1782
1783

```

```

1784
1785
1786
1787
1788     for(unsigned int i=0; i<nEvals; i++){
1789         double t=0.5*(auxeval1)*nodes(i)+auxeval2;
1790
1791         int span= FindSpan(KnotVector, t, p, n);
1792         //std::cout << "span: " << span << std::endl;
1793         double AuxVal1=0;
1794         double AuxVal2=0;
1795         //std::vector<double> eval= RatDersBasisFuns(span, t, p, 0, KnotVector, WeightVector).GetRow(0);
1796         iMatrix<double> evalMat= RatDersBasisFuns(span, t, p, 1, KnotVector, WeightVector);
1797         std::vector<double> eval1=evalMat.GetRow(0);
1798         std::vector<double> eval2=evalMat.GetRow(1);
1799         //std::cout << "t= " << t << std::endl;
1800         for(int j=0; j<p; j++){
1801             //std::cout << "funEvals[" << span-p+j<< "] = " << funEvals[span-p+j]<< std::endl;
1802             //std::cout << "eval[" << j << "] = " << eval[j]<< std::endl;
1803             AuxVal1+=funEvals[span-p+j]*eval1[j];
1804             AuxVal2+=funEvals[span-p+j]*eval2[j];
1805         }
1806         fichero1 << AuxVal1 << " ";
1807         fichero2 << AuxVal2 << " ";
1808     }
1809 }
1810 fichero1.close();
1811 fichero2.close();
1812 }
1813
1814
1815
1816 /***** */
1817
1818 /***** */
1819 void writeAnalyticFunction(std::function<double(double)> f, std::function<double(double)> f_Ders, std::vector<double> Intervals,
1820 double nEvals, const iMatrix<double> &CtrlPtsW, const std::vector<double> &KnotVector, int n, int p,
1821 std::string name, double Lenght, double nElements){
1822
1823     std::ofstream fichero1, fichero2;
1824     fichero1.open(name+ "_h=" + std::to_string(static_cast<int>(nElements)) + "_p=" + std::to_string(p) + "_Analytic_testCircle.txt");
1825     fichero2.open(name+ "_h=" + std::to_string(static_cast<int>(nElements)) + "_p=" + std::to_string(p) + "_Ders_Analytic_testCircle.txt");
1826
1827     // Configurar alta precisión
1828     fichero1 << std::scientific << std::setprecision(15);
1829     fichero2 << std::scientific << std::setprecision(15);
1830     double preVal=Intervals[0];
1831     double cumLenght=0;
1832     Eigen::VectorXd nodes, weights;
1833     legendre_pol(nEvals, nodes, weights);
1834
1835     for(unsigned int iter=0; iter<Intervals.size()-1; iter++){
1836
1837
1838         double auxeval1= (Intervals[iter+1] - Intervals[iter])/2;
1839         double auxeval2= (Intervals[iter+1] + Intervals[iter])/2;
1840         //double preVal=Intervals[iter];
1841
1842
1843         for(unsigned int i=0; i<nEvals; i++){
1844             iMatrix<double> Aders, wders;
1845             double t=0.5*(auxeval1)*nodes(i)+auxeval2;
1846             iMatrix<double> Allders = CurveDerivsAlg1(n, p, KnotVector, CtrlPtsW, t, 1);
1847             //std::cout << "i2" << std::endl;
1848             int auxInt=Allders.GetNumCols()-1;
1849             Aders = Allders.GetSubMat(0,1,0, auxInt-1);
1850             wders = Allders.GetSubMat(0,1,auxInt, auxInt);
1851             iMatrix<double> AuxMat2 = RatCurveDerivs(Aders, wders, 1);
1852             //std::cout << "i3" << std::endl;
1853
1854             std::vector<double> AuxVec= AuxMat2.GetRow(1);
1855             double Jacobian=0;
1856             for(unsigned int i=0; i< AuxVec.size(); i++){
1857                 Jacobian+= AuxVec[i]*AuxVec[i];
1858             }
1859             Jacobian=sqrt(Jacobian);
1860             double FisicalP=Fisical_to_Parametric(preVal, t, 0, Lenght, 20, CtrlPtsW, KnotVector, n, p);
1861             cumLenght+=FisicalP;
1862             //double auxval = CurvePointRational(n, p, KnotVector, CtrlPtsW, lowerLimit + i*Step)[0];
1863             //std::cout << "point is: " << lowerLimit + (auxeval1*nodes(i)+auxeval2) << std::endl;
1864             //fichero << f(auxeval1*nodes(i)+auxeval2) << " "; //case Jacobian is constant at value 1
1865             //fichero << f(CurvePointRational(n, p, KnotVector, CtrlPtsW, auxeval1*nodes(i)+auxeval2)[0])
1866             //<< " "; //case jacobian is not constant but f only takes 1 dimensional parameters
1867             fichero1 << f(cumLenght)<< " "; //optimization needed, not necessary to
1868             //compute the lagranje pol pts in each iteration
1869             fichero2 << f_Ders(cumLenght)*Jacobian<< " ";
1870             preVal=t;
1871             //fichero1 << f(auxeval1*nodes(i)+auxeval2) << " ";
1872             //optimization needed, not necessary to compute the lagranje pol pts in each iteration
1873             //fichero2 << f_Ders(auxeval1*nodes(i)+auxeval2)*Jacobian<< " ";
1874             //std::cout << CurvePointRational(n, p, KnotVector, CtrlPtsW, time[i])[0] << std::endl;
1875             //fichero1 << f(CurvePointRational(n, p, KnotVector, CtrlPtsW, time[i])[0]) << " ";
1876             //fichero2 << f_Ders(CurvePointRational(n, p, KnotVector, CtrlPtsW, time[i])[0])*Jacobian << " ";
1877         }
1878     }
1879 }
1880 fichero1.close();
1881 fichero2.close();
1882 std::cout << "Longitud total arco recorrida: " << cumLenght << std::endl;
1883
1884 }
1885
1886 /***** */

```

```

1887
1888
1889 //***** */
1890 std::vector<std::vector<double>> leerArchivo(const std::string& name, int p) {
1891     std::vector<std::vector<double>> resultado;
1892     std::ifstream archivo(name);
1893     if (!archivo.is_open()) {
1894         std::cerr << "Error: No se pudo abrir el archivo " << name << std::endl;
1895         return resultado;
1896     }
1897
1898     std::string linea;
1899     std::vector<double> filaActual;
1900
1901     std::stringstream buffer;
1902     buffer << archivo.rdbuf();
1903     std::string contenido = buffer.str();
1904
1905     std::istringstream ss(contenido);
1906     double numero;
1907
1908     while (ss >> numero) {
1909         filaActual.push_back(numero);
1910
1911         if (filaActual.size() == p) {
1912             resultado.push_back(filaActual);
1913             filaActual.clear();
1914         }
1915     }
1916
1917     if (!filaActual.empty()) {
1918         resultado.push_back(filaActual);
1919     }
1920
1921     archivo.close();
1922     return resultado;
1923 }
1924 //***** */
1925
1926 //***** */
1927 /*
1928 void writeFunctionDers(Eigen::VectorXd funEvals, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPtsW,
1929                      const std::vector<double> &WeightVector, int n, int p, int nEvals, double lowerLimit, double upperLimit, std::string name){
1930     std::ofstream fichero;
1931     fichero.open(name);
1932     Eigen::VectorXd nodes, weights;
1933     legendre_pol(nEvals, nodes, weights);
1934     double auxeval1= (upperLimit - lowerLimit)/2;
1935     double auxeval2= (upperLimit + lowerLimit)/2;
1936
1937
1938     //double Step=(upperLimit-lowerLimit)/(funEvals.size()-p);
1939     for(unsigned int i=0; i<nEvals; i++){
1940         double t=auxeval1*nodes(i)+auxeval2;
1941         //std::cout << "i4" << std::endl;
1942
1943         int span= FindSpan(KnotVector, t, p, n);
1944         double AuxVal=0;
1945         std::vector<double> eval= RatDersBasisFuns(span, t, p, 1, KnotVector, WeightVector).GetRow(1);
1946         for(int j=0; j<p; j++){
1947             AuxVal+=funEvals[span-p+j]*eval[j];
1948         }
1949
1950         fichero << AuxVal << " ";
1951     }
1952     fichero.close();
1953 }
1954 */
1955 //***** */
1956
1957 //***** */
1958 /*
1959 void writeFunctionDers(std::function<double(double)> f, double lowerLimit, double upperLimit, double nEvals, const iMatrix<double> &CtrlPtsW,
1960                      const std::vector<double> &KnotVector, int n, int p, std::string name, double Lenght){
1961     std::ofstream fichero;
1962     fichero.open(name);
1963     Eigen::VectorXd nodes, weights;
1964     legendre_pol(nEvals, nodes, weights);
1965     double auxeval1= (upperLimit - lowerLimit)/2;
1966     double auxeval2= (upperLimit + lowerLimit)/2;
1967     for(unsigned int i=0; i<nEvals; i++){
1968         double t= auxeval1*nodes(i)+auxeval2;
1969         iMatrix<double> Aders, wders;
1970         iMatrix<double> Allders = CurveDerivsAlg1(n, p, KnotVector, CtrlPtsW, t, 1);
1971         //std::cout << "i2" << std::endl;
1972         int auxInt=Allders.GetNumCols()-1;
1973         Aders = Allders.GetSubMat(0,1,0, auxInt-1);
1974         wders = Allders.GetSubMat(0,1,auxInt, auxInt);
1975         iMatrix<double> AuxMat2 = RatCurveDerivs(Aders, wders, 1);
1976         //std::cout << "i3" << std::endl;
1977
1978         std::vector<double> AuxVec= AuxMat2.GetRow(1);
1979         double Jacobian=0;
1980         for(unsigned int i=0; i< AuxVec.size(); i++){
1981             Jacobian+= AuxVec[i]*AuxVec[i];
1982         }
1983         Jacobian=sqrt(Jacobian);
1984         //double auxVal = f(CurvePointRational(n, p, KnotVector, CtrlPtsW, t)[0])*Jacobian; //Case curve is 1 dimensional
1985         //std::cout << "tders = " << Fisical_to_Parametric(t, 0, Lenght, p+2, CtrlPtsW, KnotVector, n, p) << std::endl;
1986         double auxVal = f(Fisical_to_Parametric(t, 0, Lenght, 20, CtrlPtsW, KnotVector, n, p))*Jacobian; //Generic case
1987
1988         //double auxval = CurvePointRational(n, p, KnotVector, CtrlPtsW, lowerLimit + i*Step)[0];
1989         //std::cout << "point is: " << lowerLimit + (auxeval1*nodes(i)+auxeval2) << std::endl;

```

```

1990
1991 //fichero << f(auxeval1*nodes(i)+auxeval2) << " "; //case Jacobian is constant at value 1
1992 fichero << auxVal << " "; //case jacobian is not constant but f only takes 1 dimensional parameters
1993 }
1994
1995 fichero.close();
1996 }
1997 */
1998
1999 /***** */
2000
2001 /***** */
2002 double IntegrateNormDer(double lowerLimit, double upperLimit, int nEvals, const iMatrix<double> &CtrlPtsW,
2003 const std::vector<double> &KnotVector, int n, int p){
2004 Eigen::VectorXd nodes, weights;
2005 legendre_pol(nEvals, nodes, weights);
2006 double auxeval1= (upperLimit - lowerLimit)/2;
2007 double auxeval2= (upperLimit + lowerLimit)/2;
2008 double sol=0;
2009
2010 for(unsigned int i=0; i< nEvals; i++){
2011
2012     double t= auxeval1*nodes(i)+auxeval2;
2013     //std::cout << "t= " << t << std::endl;
2014     iMatrix<double> Aders, wders;
2015     //std::cout << "i0" << std::endl;
2016     iMatrix<double> Allders = CurveDerivsAlg1(n, p, KnotVector, CtrlPtsW, t, 1);
2017     //std::cout << "i1" << std::endl;
2018     //Allders.PrintMatrix();
2019     //std::cout << "i2" << std::endl;
2020     int auxInt=Allders.GetNumCols()-1;
2021     Aders = Allders.GetSubMat(0,1,0, auxInt-1);
2022     wders = Allders.GetSubMat(0,1,auxInt, auxInt);
2023     iMatrix<double> AuxMat2 = RatCurveDerivs(Aders, wders, 1);
2024     //std::cout << "i3" << std::endl;
2025
2026     std::vector<double> AuxVec= AuxMat2.GetRow(1);
2027     double Jacobian=0;
2028     for(unsigned int iter=0; iter< AuxVec.size(); iter++){
2029         Jacobian+= AuxVec[iter]*AuxVec[iter];
2030     }
2031     Jacobian=sqrt(Jacobian);
2032     sol+=Jacobian*weights[i];
2033
2034     //double auxval = CurvePointRational(n, p, KnotVector, CtrlPtsW, lowerLimit + i*Step)[0];
2035     //std::cout << "point is: " << lowerLimit + (auxeval1*nodes(i)+auxeval2) << std::endl;
2036     //fichero << f(auxeval1*nodes(i)+auxeval2) << " "; //case Jacobian is constant at value 1
2037
2038 }
2039
2040 return sol*auxeval1;
2041 }
2042
2043 /***** */
2044
2045 /***** */
2046 std::vector<std::vector<iMatrix<double>>> PreBasis_and_Ders(int p, int k, std::vector<double> KnotVector,
2047 std::vector<double> Intervals, Eigen::VectorXd nodes, std::vector<int> & spanIndex){
2048     int nIntervals =Intervals.size()-1;
2049     int nEvals=nodes.size();
2050     int n= KnotVector.size()-p-2;
2051     std::vector<std::vector<iMatrix<double>>> sol(nIntervals);
2052     for(unsigned int i=0; i< nIntervals; i++){
2053         double lower_limit=Intervals[i];
2054         double upper_limit=Intervals[i+1];
2055         double auxeval1= (upper_limit - lower_limit)/2;
2056         double auxeval2= (upper_limit + lower_limit)/2;
2057         int span= FindSpan(KnotVector, auxeval2, p, n);
2058         spanIndex[i]=span;
2059         std::vector<iMatrix<double>>> auxVec(nEvals);
2060         for(unsigned int j=0; j<nEvals; j++){
2061             double EvalPoint = auxeval1*nodes(j)+auxeval2;
2062
2063             //std::cout <<"EvalPoint: " << EvalPoint<< std::endl;
2064             auxVec[j] = DerBasisFuns(span,EvalPoint, p, k, KnotVector);
2065         }
2066         sol[i]=auxVec;
2067     }
2068     return sol;
2069 }
2070
2071 /***** */
2072
2073 /***** */
2074 std::vector<iMatrix<double>> RationalizeDersBasisFunsVec(int span, int nEvals, Eigen::VectorXd nodes,
2075 double lower_limit, double upper_limit, int p, int k, int n, std::vector<double> KnotVector,
2076 std::vector<iMatrix<double>>> Basis_and_DersY, std::vector<double> Weights){
2077     std::vector<iMatrix<double>>> sol(nEvals);
2078     double auxeval1= (upper_limit - lower_limit)/2;
2079     double auxeval2= (upper_limit + lower_limit)/2;
2080     for(unsigned int iter=0; iter<nEvals; iter++){
2081         double EvalPoint = auxeval1*nodes(iter)+auxeval2;
2082         sol[iter]=RationalizeDersBasisFuns(span, auxeval1*nodes(iter)+auxeval2, p, k, KnotVector, Basis_and_DersY[iter], Weights);
2083     }
2084     return sol;
2085 }
2086
2087 /***** */
2088
2089 /***** */
2090 std::unordered_map<int, std::vector<std::pair<double,double>>> BuildSpanMap(const std::vector<double> & Intervals, const std::vector<int> & spanIndices){
2091     std::unordered_map<int, std::vector<std::pair<double,double>>> map;
2092     size_t nIntervals = spanIndices.size(); // debe ser nodes.size() - 1

```

```

2093
2094     for (size_t i = 0; i < nIntervals; ++i) {
2095         int span = spanIndices[i];
2096         std::pair<double, double> interval = { Intervals[i], Intervals[i+1] }; // extremos reales
2097         map[span].push_back(interval);
2098     }
2099
2100     return map;
2101 }
2102
2103 /***** */
2104
2105 /***** */
2106 iMatrix<iMatrix<double>> RationalizeDersBasisFuns2D(const iMatrix<double> &DersBasisX, const iMatrix<double> &DersBasisY,
2107 const iMatrix<double> &activeWeights, unsigned int pX, unsigned int pY) {
2108     // Matriz de salida 2x2 de matrices
2109     iMatrix<iMatrix<double>> Rationalized(2, 2);
2110
2111     // Inicializamos las submatrices (pY+1)x(pX+1)
2112     for (int i = 0; i < 2; ++i){
2113         for (int j = 0; j < 2; ++j){
2114             Rationalized(i, j).Resize(pY + 1, pX + 1);
2115         }
2116     }
2117     // Calcular la superficie de pesos y sus derivadas
2118     double W = 0.0;
2119     double Wx = 0.0;
2120     double Wy = 0.0;
2121
2122     for (unsigned int j = 0; j <= pY; ++j)
2123     {
2124         for (unsigned int i = 0; i <= pX; ++i)
2125         {
2126             double w = activeWeights(j, i);
2127             double Nx = DersBasisX(0, i);
2128             double Ny = DersBasisY(0, j);
2129             double Nx_x = DersBasisX(1, i);
2130             double Ny_y = DersBasisY(1, j);
2131
2132             W += Nx * Ny * w;
2133             Wx += Nx_x * Ny * w;
2134             Wy += Nx * Ny_y * w;
2135         }
2136     }
2137
2138     // Racionalización de funciones base y derivadas
2139     for (unsigned int j = 0; j <= pY; ++j)
2140     {
2141         for (unsigned int i = 0; i <= pX; ++i)
2142         {
2143             double w = activeWeights(j, i);
2144             double Nx = DersBasisX(0, i);
2145             double Ny = DersBasisY(0, j);
2146             double Nx_x = DersBasisX(1, i);
2147             double Ny_y = DersBasisY(1, j);
2148
2149             double N = Nx * Ny * w;
2150
2151             // Función base racionalizada
2152             Rationalized(0, 0)(j, i) = N / W;
2153
2154             // Derivada respecto a X
2155             Rationalized(1, 0)(j, i) = w * (Nx_x * Ny * W - Nx * Ny * Wx) / (W * W);
2156
2157             // Derivada respecto a Y
2158             Rationalized(0, 1)(j, i) = w * (Nx * Ny_y * W - Nx * Ny * Wy) / (W * W);
2159         }
2160     }
2161
2162     return Rationalized;
2163 }
2164
2165 /***** */
2166
2167 /***** */
2168 void ExportToTxt(const iMatrix<double>& SolEvals, const iMatrix<std::vector<double>>& Surf, const std::string& filename){
2169     std::ofstream file(filename);
2170     if (!file.is_open()) {
2171         std::cerr << "Error abriendo archivo para escritura: " << filename << std::endl;
2172         return;
2173     }
2174
2175     size_t nx = SolEvals.GetNumRows();
2176     size_t ny = SolEvals.GetNumCols();
2177     //std::cout << "1";
2178     for (size_t i = 0; i < nx; ++i) {
2179         //std::cout << "2";
2180         for (size_t j = 0; j < ny; ++j) {
2181
2182             const auto& xyz = Surf(i, j);
2183             //std::cout << xyz.size();
2184             double val = SolEvals(i, j);
2185             file<< std::setprecision(std::numeric_limits<double>::max_digits10) << xyz[0] << " " << xyz[1] << " " << xyz[2] << " " << val << "\n";
2186         }
2187     }
2188
2189     file.close();
2190     std::cout << "Datos exportados correctamente a " << filename << std::endl;
2191 }
2192

```

A.4.4. Curvas de NURBS

Código del dibujo de las curvas de NURBS

```

1  RUTA_ARCHIVO_CURVA = PATH + 'EjCurva3d.txt';
2  NOMBRE_ARCHIVO_SVG = 'CurvaNurbs_3D_Plot.svg';
3
4  PC = [
5      10, 0, 0;
6      15, 10, 5;
7      5, 15, 20;
8      0, 5, 10;
9      10, 0, 0
10 ];
11
12 try
13     D = load(RUTA_ARCHIVO_CURVA);
14 catch
15     error('ERROR: No se pudo cargar el archivo. Comprueba que la ruta sea correcta: %s',
16         ↪ RUTA_ARCHIVO_CURVA);
17 end
18
19 [M, N] = size(D);
20
21 C = D';
22
23 figure;
24 hold on;
25 grid on;
26
27 plot3(C(:,1), C(:,2), C(:,3), 'b-', 'LineWidth', 2);
28 plot3(PC(:,1), PC(:,2), PC(:,3), 'r--', 'LineWidth', 1, 'DisplayName', 'Polígono de Control');
29 plot3(PC(:,1), PC(:,2), PC(:,3), 'ro', 'MarkerSize', 4, 'MarkerFaceColor', 'r', 'DisplayName',
30     ↪ 'Puntos de Control');
31
32 min_coords = min([C; PC]);
33 max_coords = max([C; PC]);
34 margen = 2;
35 axis([min_coords(1)-margen max_coords(1)+margen ...
36     min_coords(2)-margen max_coords(2)+margen ...
37     min_coords(3)-margen max_coords(3)+margen]);
38
39 view(3);
40 set(gca, 'Box', 'on');
41 print(NOMBRE_ARCHIVO_SVG, '-dsvg');
42 disp(['Gráfico final guardado en: ', fullfile(pwd, NOMBRE_ARCHIVO_SVG)]);

```

Código de las derivadas de las funciones base de NURBS

```

1  int p=3;
2  std::vector<double> Nurbs={0,0,0,0,0.5,1,1,1,1};
3  std::vector<double> weights={1,1,5,1,1};
4
5  double N_Steps = 1000;
6  std::vector<double> eval(Nurbs.size()-p);
7  iMatrix<double> ders(p+1,p+1);
8  iMatrix<double> sol(Nurbs.size()-p,N_Steps+1);
9  //std::vector<double> Nurb(NurbsVector.size());
10 double Step=1/(N_Steps);
11 auto start = std::chrono::high_resolution_clock::now();
12 for(unsigned int i = 0; i<N_Steps+1; i++){
13

```

```

14     double t=Step*i;
15     //std::cout << "Iteración iniciada " << i<< std::endl;
16     int span = FindSpan(Nurbs, t, p, Nurbs.size()-2-p);
17     ders = RatDersBasisFuns(span,t,p,1,Nurbs, weights);
18     ders.PrintMatrix();
19     std::cout<<"-----"<<std::endl;
20     for(int j = span-p; j<=span;j++){
21         if(j>=0) {
22             sol(j,i)=ders(1,j-span+p);
23         }
24         else{
25             //Caso error.
26         }
27     }
28 }
29 }
30
31 Write_BasisFunctions(sol, "path+name");
32 auto end = std::chrono::high_resolution_clock::now();
33 auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>(end - start);
34
35 std::cout << "Tiempo tomado: " << duration.count() << " nanosegundos\n";

```

Código de las funciones base de NURBS

```

1  int p=3;
2  double N_Steps = 1000;
3  std::vector<double> KnotVector={0,0,0,0.5,1,1,1,1};
4  std::vector<double> WeightVector={1,1,5,1,1};
5  iMatrix<double> sol(KnotVector.size()-p-1,N_Steps+1);
6
7  double Step=1/(N_Steps);
8  for(unsigned int i = 0; i<N_Steps+1; i++){
9      double t=Step*i;
10     int span = FindSpan(KnotVector, t, p, KnotVector.size()-2-p);
11     std::vector<double> eval= RationalBasisFuns(span, t , p, KnotVector, WeightVector);
12     for(int j = span-p; j<=span;j++){
13         if(j>=0) {
14             sol(j,i)=eval[j-span+p];
15         }
16         else{
17             //Caso error.
18         }
19     }
20 }
21 Write_BasisFunctions(sol, "path+name");

```

A.4.5. Superficies de NURBS

Código del dibujo de las superficies de NURBS

```

1  dataX = dlmread(PATH + 'SuperficieRationalEj2X.txt', ' ');
2  dataY = dlmread(PATH + 'SuperficieRationalEj2Y.txt', ' ');
3  dataZ = dlmread(PATH + 'SuperficieRationalEj2Z.txt', ' ');
4
5  hold on
6  surf(dataX, dataY, dataZ);
7
8  dataCtrlX = dlmread(PATH + 'Ej2X.txt', ' ');
9  dataCtrlY = dlmread(PATH + 'Ej2Y.txt', ' ');
10 dataCtrlZ = dlmread(PATH + 'Ej2Z.txt', ' ');

```

```

11
12 [n1 n2] = size(dataCtrlX);
13
14 for i=1:n1
15     plot3(dataCtrlX(i,:),dataCtrlY(i,:),dataCtrlZ(i,:), color='r');
16 endfor
17
18 for j=1:n2
19     plot3(dataCtrlX(:,j),dataCtrlY(:,j),dataCtrlZ(:,j), color='r');
20 endfor
21
22 scatter3(dataCtrlX(:), dataCtrlY(:), dataCtrlZ(:), 30, 'k', 'filled', 'MarkerEdgeColor', 'w');
23 view(30, 45);
24 grid on;
25
26 print -dsvg 'Superficie_NURBS_Ej1.svg';
27 hold off;

```

Código del cálculo de las superficies de NURBS

```

1  std::vector<iMatrix<double>> CtrlPts = ReadDataFile_CtrlPts("path+nameCtrlPts");
2  iMatrix<double> Weights =ReadDataFile_Matrix("path+nameWeights");
3  std::vector<iMatrix<double>> CtrlPtsW=WeightCtrlPtsSurf(CtrlPts,Weights);
4  std::vector<double> V1={0,0,0,0.5,0.5,1,1,1};
5  std::vector<double> V2={0,0,0,0,1,1,1,1};
6  int p1=2;
7  int p2=3;
8  double N_Steps_1 = 25;
9  double N_Steps_2 = 25;
10
11 double Step_1=1/N_Steps_1;
12 double Step_2=1/N_Steps_2;
13
14 std::vector<iMatrix<double>> eval(N_Steps_1+1);
15 for(unsigned int i=0; i<N_Steps_1+1;i++){
16
17     iMatrix<double> auxMatrix(3,N_Steps_2+1);
18     for(unsigned int j=0; j< N_Steps_2+1;j++){
19
20         std::vector<double> aux = SurfacePointRational(V1.size()-2-p1,p1,V1,
21                                                         V2.size()-2-p2,p2,V2,CtrlPtsW,i*Step_1,j*Step_2);
22         for(unsigned int k=0; k<3;k++){
23             auxMatrix(k,j)=aux[k];
24         }
25     }
26     eval[i]=auxMatrix;
27 }
28
29 Write_BezierSurface(eval,"path+nameSurf" );
30 Write_CtrlPts(CtrlPts,"path+nameCtrlPts");

```

A.4.6. Implementaciones de IGA

Caso 1D, Solución

```

1
2
3 bool PrintMatixes=false;
4 bool PrintIntermediateValues=false;
5 bool PrintPrevalues=false;
6
7
8 int p=2;
9 double nElements = 512;
10 //variables que no cambian en cada ejecución del bucle o que se declaran fuera para ir actualizandolas

```

```

11  iMatrix<double> CtrlPts = ReadDataFile_Matrix(PATH + "EjCirculo.txt");
12  if(PrintPrevalues){
13      std::cout<< "-----CtrlPts-----" << std::endl;
14      CtrlPts.PrintMatrix();
15  }
16  std::vector<double> KnotVector={0,0,0,0.5,0.5,1,1,1};
17  double lower_limit=KnotVector[0];
18  double upper_limit=KnotVector[KnotVector.size()-1];
19  std::vector<double> Weights={1,sqrt(2)/2,1,sqrt(2)/2,1};
20
21
22  iMatrix<double> WeightedCtrlPts=WeightCtrlPts(CtrlPts,Weights);
23
24  if(PrintPrevalues){
25      std::cout<< "-----WeightedCtrlPts-----" << std::endl;
26      WeightedCtrlPts.PrintMatrix();
27  }
28  int n= KnotVector.size()-p-2;
29  if(PrintPrevalues){
30      std::cout<< "-----KnotVectorOld-----" << std::endl;
31      for(unsigned int i=0; i<KnotVector.size(); i++){
32          std::cout << std::setprecision(7) << KnotVector[i] << " ";
33      }
34      std::cout << std::endl;
35      std::cout << "n value:" << n << std::endl;
36  }
37  std::vector<double> Insertions=createSubIntervals(nElements, lower_limit, upper_limit);
38  Insertions=removeMatches(Insertions, KnotVector);
39  iMatrix<double> NewCtrlPts=RefineKnotVectCurve(n,p,KnotVector, WeightedCtrlPts, Insertions, Insertions.size()-1);
40  if(PrintPrevalues){
41      std::cout<< "-----NewCtrlPtsW-----" << std::endl;
42      NewCtrlPts.PrintMatrix(7);
43  }
44  int size = KnotVector.size()-p-1;
45  std::function<double(double)> f = [](double x) {
46      return 100 std::sin( 10*M_PI * x);
47  };
48
49  auto analyticSol = [](double x){
50      //double pi= std::numbers::pi;
51      return sin(10*M_PI*x);
52  };
53
54  auto analyticSolDers = [](double x){
55      //double pi= std::numbers::pi;
56      return 10*cos(10*M_PI*x);
57  };
58
59  n = KnotVector.size()-p-2;
60
61
62  if(PrintPrevalues){
63      std::cout<< "-----KnotVectorNew-----" << std::endl;
64      for(unsigned int i=0; i<KnotVector.size(); i++){
65          std::cout <<std::setprecision(7) << KnotVector[i] << " ";
66      }
67  }
68  //double Lenght=IntegrateNormDer(lower_limit,upper_limit, 5000, NewCtrlPts, KnotVector, n, p);
69  double Lenght = M_PI;
70  std::cout << std::endl;
71  std::cout << "n value:" << n << std::endl;
72
73  Weights=NewCtrlPts.GetRow(NewCtrlPts.GetNumRows()-1);
74  if(PrintPrevalues){
75      std::cout<< "-----Weights-----" << std::endl;
76      for(unsigned int i=0; i<Weights.size(); i++){
77          std::cout <<std::setprecision(7) << Weights[i] << " ";
78      }
79      std::cout << std::endl;
80  }
81  int dim=NewCtrlPts.GetNumRows()-1;
82  int numElements= NewCtrlPts.GetNumCols();
83
84  int nEvals=p+1;
85
86
87  Eigen::SparseMatrix<double> global(size, size);
88  Eigen::VectorXd LinearForm(size);
89  LinearForm.setZero(); // Inicializa en cero
90
91  std::vector<double> Intervals=removeDuplicates(KnotVector);
92
93
94  if(PrintPrevalues){
95      std::cout<< "-----Intervals-----" << std::endl;
96      for(unsigned int i=0; i<Intervals.size(); i++){
97          std::cout << Intervals[i] << " ";
98      }
99      std::cout << std::endl;
100  }
101  Eigen::VectorXd nodes, weights;
102  legendre_pol(nEvals, nodes, weights);
103  if(PrintPrevalues){
104      std::cout << "Nodos (raices):\n" << nodes.transpose() << "\n";
105      std::cout << "Pesos:\n" << weights.transpose() << "\n";
106  }
107  typedef Eigen::Triplet<double> T;
108  std::vector<Eigen::Triplet<double>> tripletList;
109  double preEvalPoint=lower_limit;
110  double cumLenght=0;
111
112  //iteraciones
113  auto start = std::chrono::high_resolution_clock::now();

```

```

114 for(unsigned int index=0; index<Intervals.size()-1; index++){
115     double lowerLimit=Intervals[index], upperLimit=Intervals[index+1];
116     int span = FindSpan(KnotVector, lowerLimit,p,n);
117     std::vector<int> global_indices(p+1);
118     for(unsigned int w=0; w<=p; w++){
119         global_indices[w]=span-p+w;
120     }
121     if(PrintIntermediateValues){
122
123         std::cout << "global_indices: (" ;
124         for(unsigned int w=0; w<=p; w++){
125             std::cout << global_indices[w] << ", ";
126         }
127         std::cout << global_indices[p] << ")" << std::endl;
128         std::cout << "Evaluation Interval: (" << Intervals[index] << ", " << Intervals[index+1] << ")" << std::endl;
129         std::cout << "-----" << std::endl;
130     }
131
132     iMatrix<double> BasisFunsEvals;
133     iMatrix<double> DerBasisFunsEvals;
134     std::vector<double> JacobianEvals;
135     std::vector<double> InverseJacobianEvals;
136     std::vector<double> funcEvals;
137
138     D1_element_eval(n, span,p,nEvals, lowerLimit, upperLimit, KnotVector, Weights, nodes, NewCtrlPts,
139         f, preEvalPoint, cumLenght, BasisFunsEvals, DerBasisFunsEvals, JacobianEvals, InverseJacobianEvals, funcEvals, Lenght);
140     if(PrintIntermediateValues){
141         std::cout << "-----BasisFunsEvals-----" << std::endl;
142         BasisFunsEvals.PrintMatrix(7);
143         std::cout << "-----DerBasisFunsEvals-----" << std::endl;
144         DerBasisFunsEvals.PrintMatrix(7);
145         std::cout << "-----JacobianEvals-----" << std::endl;
146         for(unsigned int i=0; i<JacobianEvals.size(); i++){
147             std::cout << std::setprecision(7) << JacobianEvals[i] << ", ";
148         }
149         std::cout << std::endl;
150         std::cout << "-----InverseJacobianEvals-----" << std::endl;
151         for(unsigned int i=0; i<InverseJacobianEvals.size(); i++){
152             std::cout << std::setprecision(7) << InverseJacobianEvals[i] << ", ";
153         }
154         std::cout << std::endl;
155         std::cout << "-----funcEvals-----" << std::endl;
156         for(unsigned int i=0; i<funcEvals.size(); i++){
157             std::cout << std::setprecision(7) << funcEvals[i] << ", ";
158         }
159         std::cout << std::endl;
160         std::cout << "-----Element-----" << std::endl;
161     }
162
163     iMatrix<double> Element=gauss_legendre_cuadrature_integral_bilinearForm(p,lowerLimit,upperLimit,nEvals, DerBasisFunsEvals, InverseJacobianEvals, weights);
164
165     if(PrintIntermediateValues){
166         Element.PrintMatrix(7);
167         std::cout << "-----" << std::endl;
168     }
169
170     for (int i = 0; i <= p; ++i){
171         for (int j = 0; j <= p; ++j){
172             tripletList.push_back(T(global_indices[i], global_indices[j], Element(i, j)));
173         }
174     }
175
176     std::vector<double> LinearElement=gauss_legendre_cuadrature_integral_linealForm(p,lowerLimit, upperLimit, nEvals,
177         BasisFunsEvals, JacobianEvals, funcEvals, weights);
178
179     if(PrintIntermediateValues){
180         std::cout << "-----LinearElement-----" << std::endl;
181         for(unsigned int i=0; i<LinearElement.size(); i++){
182             std::cout << std::setprecision(7) << LinearElement[i] << ", ";
183         }
184         std::cout << std::endl;
185     }
186     for(unsigned int i=0; i<=p; i++){
187         LinearForm(global_indices[i])+=LinearElement[i];
188     }
189 }
190
191
192 global.setFromTriplets(tripletList.begin(), tripletList.end());
193
194 if(PrintMatixes){
195     std::cout << "-----GlobalMat-----" << std::endl;
196     for (int k = 0; k < global.outerSize(); ++k){
197         for (Eigen::SparseMatrix<double>::InnerIterator it(global, k); it; ++it){
198             std::cout << "(" << it.row() << ", " << it.col() << "): " << it.value() << "\n";
199         }
200     }
201     //std::cout<<Eigen::MatrixXd(global) <<std::endl;
202     std::cout << "-----GlobalLinearForm-----" << std::endl;
203     for (int k = 0; k < LinearForm.size(); ++k){
204         std::cout << LinearForm[k] << ", ";
205     }
206     std::cout << std::endl;
207 }
208
209 //Compute of the Inverse jacobians for boundary conditions on dirhcllet/Robin conditions
210 iMatrix<double> Aders, wders;
211 iMatrix<double> Allders0 = CurveDerivsAlg1(n, p, KnotVector, NewCtrlPts, 0, 1);
212 int auxInt=Allders0.GetNumCols()-1;
213 Aders = Allders0.GetSubMat(0,1,0, auxInt-1);
214 wders = Allders0.GetSubMat(0,1,auxInt, auxInt);
215 iMatrix<double> AuxMat2 = RatCurveDerivs(Aders, wders, 1);
216 std::vector<double> AuxVec= AuxMat2.GetRow(1);

```

```

217 double JacobianEvals0=0;
218 for(unsigned int i=0; i< AuxVec.size(); i++){
219     JacobianEvals0+= AuxVec[i]*AuxVec[i];
220 }
221
222
223 double InverseJacobianEvals0=1/JacobianEvals0;
224
225
226 iMatrix<double> Allders1 = CurveDerivsAlg1(n, p, KnotVector, NewCtrlPts, 1, 1);
227 Aders = Allders1.GetSubMat(0,1,0, auxInt-1);
228 wders = Allders1.GetSubMat(0,1,auxInt, auxInt);
229 AuxMat2 = RatCurveDerivs(Aders, wders, 1);
230 AuxVec= AuxMat2.GetRow(1);
231 double JacobianEvals1=0;
232 for(unsigned int i=0; i< AuxVec.size(); i++){
233     JacobianEvals1+= AuxVec[i]*AuxVec[i];
234 }
235 double InverseJacobianEvals1=1/JacobianEvals1;
236
237 //apply boundary conditions
238 impose_Dirichlet_Condition(p, size-1, global, LinearForm, true, true);
239 //impose_Neumann_Condition(size-1, LinearForm, true, true);
240 //double alpha = 3;
241 //double beta= 5;
242 //impose_Robin_Condition(p, size-1, global, LinearForm, alpha, beta, true, false);
243
244 if(PrintMatixes){
245     std::cout<< "-----GlobalMatPostCond-----" << std::endl;
246     for (int k = 0; k < global.outerSize(); ++k){
247         for (Eigen::SparseMatrix<double>::InnerIterator it(global, k); it; ++it){
248             std::cout << setprecision(std::numeric_limits<double>::max_digits10) << "(" << it.row() << ", " << it.col() << "): " << it.value() << "\n";
249         }
250     }
251 }
252
253 if(PrintMatixes){
254     std::cout<< "-----GlobalLinearFormPostCond-----" << std::endl;
255     for (int k = 0; k < LinearForm.size(); ++k){
256         std::cout << setprecision(std::numeric_limits<double>::max_digits10) << LinearForm[k] << ", ";
257     }
258     std::cout << std::endl;
259 }
260
261
262 Eigen::SparseLU<Eigen::SparseMatrix<double>> solver;
263 solver.compute(global);
264 if(solver.info() != Eigen::Success) {
265     std::cout << "Error 1" << std::endl;
266 }
267
268 Eigen::VectorXd u = solver.solve(LinearForm);
269 if(solver.info() != Eigen::Success) {
270     std::cout << "Error 2" << std::endl;
271 }
272
273 auto end = std::chrono::high_resolution_clock::now();
274 auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>(end - start);
275 std::cout << "Tiempo tomado: " << duration.count() << " nanosegundos\n";
276
277 //std::cout<<"Longitud de la curva= "<< setprecision(std::numeric_limits<double>::max_digits10) << Lenght <<std::endl;
278 std::string Text=PATH + "EjSinNonConst"; //_h=" + to_string(static_cast<int>(nElements)) + "_p=" + to_string(p)+"_test2.txt"
279
280 writeNumericFunction(u, KnotVector, Weights, n, p, nEvals, lower_limit, upper_limit, Text,Intervals, nElements, M_PI);
281 //std::cout<<"Longitud de la curva= "<< setprecision(std::numeric_limits<double>::max_digits10) << Lenght <<std::endl;
282 writeAnalyticFunction(analyticSol, analyticSolDers, Intervals, nEvals, NewCtrlPts, KnotVector, n, p, Text, Lenght, nElements);
283
284 //std::cout<<"Longitud de la curva= "<< setprecision(std::numeric_limits<double>::max_digits10) << cumLenght <<std::endl;
285
286 iMatrix<double> sol(dim,nEvals*nElements);
287 std::vector<double> eval(dim);
288
289 for(unsigned int iter = 0; iter<nElements; iter++){
290     double auxeval1= (Intervals[iter+1] - Intervals[iter])/2;
291     double auxeval2= (Intervals[iter+1] + Intervals[iter])/2;
292
293     for(unsigned int j=0; j<nEvals; j++){
294         double t=0.5*(auxeval1)*nodes(j)+auxeval2;
295         //std::cout << i << std::endl;
296         eval=CurvePointRational(KnotVector.size()-2-p,p,KnotVector,NewCtrlPts,t);
297         //std::cout << eval[0] << "|" << eval[1] << std::endl;
298         sol.SetCol(iter*nEvals + j,eval);
299     }
300 }
301
302 Write_Curve(sol,PATH + "CurvaEjCircle_h="+ std::to_string(static_cast<int>(nElements)) + "_p=" + to_string(p) + ".txt");
303

```

Caso1D, Cálculo de Normas

```

1
2 int h_val = 32768;
3 int p_val = 2;
4 std::string nameAnalytic=PATH + "EjSinNonConst_h="+ to_string(h_val) + "_p=" + to_string(p_val) + "_Analytic_testCircle.txt";
5 std::string nameNumeric=PATH + "EjSinNonConst_h="+ to_string(h_val) + "_p=" + to_string(p_val) + "_testCircle.txt";
6 std::string nameAnalyticDers=PATH + "EjSinNonConst_h="+ to_string(h_val) + "_p=" + to_string(p_val) + "_Ders_Analytic_testCircle.txt";
7 std::string nameNumericDers=PATH + "EjSinNonConstDers_h="+ to_string(h_val) + "_p=" + to_string(p_val) + "_testCircle.txt";
8 std::vector<std::vector<double>> AnalyticSol = leerArchivo(nameAnalytic, p_val+1);
9 std::vector<std::vector<double>> NumericSol = leerArchivo(nameNumeric, p_val+1);

```

```

10  std::vector<std::vector<double>> AnalyticSolDers = leerArchivo(nameAnalyticDers, p_val+1);
11  std::vector<std::vector<double>> NumericSolDers = leerArchivo(nameNumericDers, p_val+1);
12
13  Eigen::VectorXd nodes, weights;
14  legendre_pol(p_val+1, nodes, weights);
15
16  std::vector<double> Intervals = createSubIntervals(h_val, 0, 1);
17
18  double L2sqr=0;
19  double H1sqr=0;
20  double cumSum=0;
21  double cumSumDers=0;
22
23
24  for(unsigned int iter=0; iter<h_val; iter++){
25      double a = Intervals[iter];
26      double b = Intervals[iter+1];
27      double h_elemento = (b - a)/2.0;
28
29      cumSum=0;
30      cumSumDers=0;
31
32      for(unsigned int i=0; i<=p_val; i++){
33          cumSum+=(AnalyticSol[iter][i]-NumericSol[iter][i])*(AnalyticSol[iter][i]-NumericSol[iter][i])*weights(i)* h_elemento;
34          cumSumDers+=(AnalyticSolDers[iter][i]-NumericSolDers[iter][i])*(AnalyticSolDers[iter][i]-NumericSolDers[iter][i])*weights(i)* h_elemento;
35      }
36
37      L2sqr+=cumSum;
38      H1sqr+=cumSumDers;
39  }
40
41  double L2norm=std::sqrt(L2sqr);
42  double H1norm=std::sqrt(L2sqr+H1sqr);
43  std::cout <<"Norma L2=" << L2norm << std::endl;
44  std::cout <<"Norma H1=" << H1norm << std::endl;
45  std::cout<< "fin 29 " << std::endl;
46

```

Caso 2D

```

1  bool PrintMatrixes=false;
2  bool PrintMatrixes2=false;
3  bool ShowValues=false;
4  bool ShowIterations=false;
5  bool ShowIterations2=false;
6  bool ShowConditions=false;
7  bool ShowSolutions=false;
8
9  //Initial definitions of variables
10 int pY= 2;
11 int pX= 2;
12 int hY= 512;
13 int hX= 512;
14 int nEvalsY=pY+1;
15 int nEvalsX=pX+1;
16 int subMatSize=nEvalsX*nEvalsY;
17 int nElementsY=hY*pY;
18 int nElementsX=hX*pX;
19 int size = nElementsY*nElementsX;
20
21 Eigen::SparseMatrix<double> global(size, size);
22 Eigen::VectorXd LinearForm(size);
23 std::vector<double> KnotVectorY = {0,0,0,1,1,1};
24 std::vector<double> KnotVectorX = {0,0,0,1,1,1};
25
26 double lower_limitY=KnotVectorY[0];
27 double upper_limitY=KnotVectorY[KnotVectorY.size()-1];
28 double lower_limitX=KnotVectorX[0];
29 double upper_limitX=KnotVectorX[KnotVectorX.size()-1];
30
31
32
33 auto analyticSol = [](double x, double y){
34     //double pi=3.141592653589793;
35     return (1-x*x-y*y)/4;
36 };
37
38 auto analyticSol_dx = [](double x, double y) {
39     //double pi = 3.141592653589793;
40     return -2*x/4;
41 };
42
43 auto analyticSol_dy = [](double x, double y) {
44     //double pi = 3.141592653589793;
45     return -2*y/4;
46 };
47
48 auto f = [](double x, double y){
49     //double pi=3.141592653589793;
50     return 1;
51 };
52
53 std::vector<double> data={1,std::sqrt(2)/2,1,
54                        std::sqrt(2)/2,1,std::sqrt(2)/2,
55                        1,std::sqrt(2)/2,1,
56
57                        };
58
59 iMatrix<double> Weights(3,3, data.data());

```

```

60 std::vector<iMatrix<double>> CtrlPts = ReadDataFile_CtrlPts(PATH + "EjSuperficieCircle.txt");
61 std::vector<iMatrix<double>> CtrlPtsW=WeightCtrlPtsSurf(CtrlPts,Weights);
62 Eigen::VectorXd nodesY, nodesX, weightsY, weightsX;
63 legendre_pol(nEvalsY, nodesY, weightsY);
64 legendre_pol(nEvalsX, nodesX, weightsX);
65
66
67 if(ShowValues){
68     std::cout<< "-----KnotVectorY-----" << std::endl;
69     for(unsigned int i=0; i<KnotVectorY.size(); i++){
70         std::cout << KnotVectorY[i] << " ";
71     }
72     std::cout << std::endl;
73
74     std::cout<< "-----KnotVectorX-----" << std::endl;
75     for(unsigned int i=0; i<KnotVectorX.size(); i++){
76         std::cout << KnotVectorX[i] << " ";
77     }
78     std::cout << std::endl;
79
80
81     std::cout << "lower_limitY= " << lower_limitY << std::endl;
82     std::cout << "upper_limitY= " << upper_limitY << std::endl;
83     std::cout << "lower_limit2= " << lower_limitX << std::endl;
84     std::cout << "upper_limit2= " << upper_limitX << std::endl;
85
86     std::cout<<"----CtrlPts----" << std::endl;
87     for(unsigned int i=0; i<CtrlPts.size();i++){
88         CtrlPts[i].PrintMatrix();
89         std::cout<<"-----" << std::endl;
90     }
91     std::cout<<"---Matriz_Pesos---" << std::endl;
92     Weights.PrintMatrix();
93     std::cout<<"-----CtrlPtsW-----" << std::endl;
94
95     for(unsigned int i=0; i<CtrlPtsW.size();i++){
96         CtrlPtsW[i].PrintMatrix();
97         std::cout<<"-----" << std::endl;
98     }
99 }
100
101 std::vector<double> InsertionsY=createSubIntervals(hY, lower_limitY, upper_limitY);
102 InsertionsY=removeMatches(InsertionsY, KnotVectorY);
103
104
105 std::vector<double> InsertionsX=createSubIntervals(hX, lower_limitX, upper_limitX);
106 InsertionsX=removeMatches(InsertionsX, KnotVectorX);
107
108
109
110 std::vector<iMatrix<double>> NewCtrlPtsWintermediate= RefineKnotVectSurface(KnotVectorY.size()-2-pY,pY,KnotVectorY,KnotVectorX.size()-2-pX,pX,
111                                     KnotVectorX, CtrlPtsW, true, InsertionsY, InsertionsY.size()-1);
112 std::vector<iMatrix<double>> NewCtrlPtsW= RefineKnotVectSurface(KnotVectorY.size()-2-pY,pY,KnotVectorY,KnotVectorX.size()-2-pX,pX,
113                                     KnotVectorX, NewCtrlPtsWintermediate, false, InsertionsX, InsertionsX.size()-1);
114
115
116 NewCtrlPtsWintermediate.clear();
117
118 iMatrix<double> NewWeights(NewCtrlPtsW.size(),NewCtrlPtsW[0].GetNumCols());
119 for(unsigned int i=0; i<NewWeights.GetNumRows(); i++){
120     for(unsigned int j=0; j<NewWeights.GetNumCols(); j++){
121         NewWeights(i,j)=NewCtrlPtsW[i](3,j);
122     }
123 }
124 std::vector<double> IntervalsY=removeDuplicates(KnotVectorY);
125 std::vector<double> IntervalsX=removeDuplicates(KnotVectorX);
126
127
128
129
130
131 if(ShowValues){
132
133     std::cout<< "-----InsertionsY-----" << std::endl;
134     for(unsigned int i=0; i<InsertionsY.size(); i++){
135         std::cout << InsertionsY[i] << " ";
136     }
137     std::cout << std::endl;
138
139     std::cout<< "-----InsertionsX-----" << std::endl;
140     for(unsigned int i=0; i<InsertionsX.size(); i++){
141         std::cout << InsertionsX[i] << " ";
142     }
143     std::cout << std::endl;
144
145
146     std::cout<<"----NewCtrlPtsW----" << std::endl;
147     for(unsigned int i=0; i<NewCtrlPtsW.size();i++){
148         NewCtrlPtsW[i].PrintMatrix();
149         std::cout<<"-----" << std::endl;
150     }
151
152     std::cout<<"----NewWeights----" << std::endl;
153     NewWeights.PrintMatrix();
154
155     std::cout<< "-----NewKnotVectorY-----" << std::endl;
156     for(unsigned int i=0; i<KnotVectorY.size(); i++){
157         std::cout << KnotVectorY[i] << " ";
158     }
159     std::cout << std::endl;
160
161     std::cout<< "-----NewKnotVectorX-----" << std::endl;
162     for(unsigned int i=0; i<KnotVectorX.size(); i++){

```

```

163         std::cout << KnotVectorX[i] << " ";
164     }
165     std::cout << std::endl;
166
167     std::cout<< "-----IntervalsY-----" << std::endl;
168
169     for(unsigned int i=0; i<IntervalsY.size(); i++){
170         std::cout << IntervalsY[i] << " ";
171     }
172     std::cout << std::endl;
173
174     std::cout<< "-----IntervalsX-----" << std::endl;
175
176     for(unsigned int i=0; i<IntervalsX.size(); i++){
177         std::cout << IntervalsX[i] << " ";
178     }
179     std::cout << std::endl;
180 }
181
182
183 //int nBasisX=NewCtrlPtsW.size();
184 //int nBasisY=NewCtrlPtsW[0].GetNumCols();
185 int nX= KnotVectorX.size()-pX-2;
186 int nY= KnotVectorY.size()-pY-2;
187 int nLoc=(pX+1)*(pY+1);
188 std::vector<Eigen::Triplet<double>> tripletList;
189 tripletList.reserve(hX * hY * nLoc * nLoc);
190
191 //for paralelitation
192 int nthreads = omp_get_max_threads();
193 std::vector<std::vector<Eigen::Triplet<double>>> threadTriplets(nthreads);
194 std::vector<std::vector<double>> threadLinearForms(nthreads, std::vector<double>(size, 0.0));
195
196 // Heuristica de reserva por hilo para evitar muchas realocaciones.
197 // Ajusta segun memoria / experiencia.
198 for (int t = 0; t < nthreads; ++t) {
199     threadTriplets[t].reserve((hX * hY * nLoc * nLoc) / nthreads / 4 + 1000);
200 }
201
202 Eigen::MatrixXd GaussWeights = weightsX * weightsY.transpose();
203 /*
204 cout << "weightsY: " <<weightsY << std::endl;
205 cout << "weightsX: " << weightsX << std::endl;
206 cout << "GaussWeights: " << std::endl << GaussWeights << std::endl;
207 */
208 auto start = std::chrono::high_resolution_clock::now();
209 //Precomputation of BasisFuns and its derivatives.
210 std::vector<int> SpanIndexY(IntervalsY.size()-1);
211 std::vector<int> SpanIndexX(IntervalsX.size()-1);
212
213 std::vector<std::vector<Matrix<double>>> Basis_and_DersY=PreBasis_and_Ders(pY, 1, KnotVectorY, IntervalsY, nodesY, SpanIndexY);
214 std::vector<std::vector<Matrix<double>>> Basis_and_DersX=PreBasis_and_Ders(pX, 1, KnotVectorX, IntervalsX, nodesX, SpanIndexX);
215
216 if(ShowValues){
217     std::cout<< "-----Basis_and_DersY-----" << std::endl;
218     for(unsigned int i=0; i< Basis_and_DersY.size(); i++){
219         std::cout<< " - - - - - " << std::endl;
220         for(unsigned int j=0; j< nEvalsY; j++){
221             Basis_and_DersY[i][j].PrintMatrix();
222             std::cout<< " - - - - - " << std::endl;
223         }
224         std::cout<< "-+-+--+--+--+--+--+--+--+--+-" << std::endl;
225     }
226     std::cout<< "-----Basis_and_DersX-----" << std::endl;
227     for(unsigned int i=0; i< Basis_and_DersX.size(); i++){
228         std::cout<< " - - - - - " << std::endl;
229         for(unsigned int j=0; j< nEvalsX; j++){
230             Basis_and_DersX[i][j].PrintMatrix();
231             std::cout<< " - - - - - " << std::endl;
232         }
233         std::cout<< "-+-+--+--+--+--+--+--+--+--+-" << std::endl;
234     }
235 }
236 }
237 //std::unordered_map<int, std::vector<std::pair<double,double>>> spanMapY=BuildSpanMap(IntervalsY, SpanIndexY);
238 //std::unordered_map<int, std::vector<std::pair<double,double>>> spanMapX=BuildSpanMap(IntervalsX, SpanIndexX);
239
240 // Double index matrix for easier and faster assembly
241 std::vector<double> detg_vals;
242 #pragma omp parallel for collapse(2) schedule(static)
243 // Assembly of the matrix
244 for (unsigned int i = 0; i < hX; i++) {
245     for (unsigned int j = 0; j < hY; j++) {
246
247         int tid = omp_get_thread_num();
248         // DEBUG: vector local por hilo
249         std::vector<double> local_detg_vals;
250
251
252         // Local buffers referidos por tid
253         auto &localTriplets = threadTriplets[tid];
254         auto &localLinear = threadLinearForms[tid];
255
256         iMatrix<double> K_e(subMatSize, subMatSize);
257         std::vector<double> F_e((pX+1)*(pY+1));
258         if (ShowIterations) {
259             std::cout<<"-----Active Intervals-----" << std::endl;
260             std::cout<< "IntervalX: (" << IntervalsX[i] << " " << IntervalsX[i+1]<<")" << std::endl;
261             std::cout<< "IntervalY: (" << IntervalsY[j] << " " << IntervalsY[j+1]<<")" << std::endl;
262             std::cout<< "SpanIndexX: (" << SpanIndexX[i] <<")" << std::endl;
263             std::cout<< "SpanIndexY: (" << SpanIndexY[j] <<")" << std::endl;
264         }
265         double auxevaliX= (IntervalsX[i+1] - IntervalsX[i])/2;

```

[illegible]

```

369         //std::cout << "I_global= " << I_global << std::endl;
370
371         //LinearForm[I_global] += F_e[i_local];
372
373         for(int c = 0; c <= pX; ++c){
374             for(int d = 0; d <= pY; ++d){
375
376                 int j_local = d*(pX+1) + c;
377                 int J_global = (spanU - pX + c) + (spanV - pY + d)*nElementsX;
378
379                 //std::cout << "J_global= " << J_global << std::endl;
380                 double val = K_e(i_local, j_local);
381                 if(val != 0.0){
382                     //localTriplets.emplace_back(I_global, J_global, val);
383                     localTriplets.emplace_back(I_global, J_global, val);
384                 }
385             }
386         }
387     }
388 }
389
390 #pragma omp critical
391 {
392     detg_vals.insert(detg_vals.end(),
393                     local_detg_vals.begin(),
394                     local_detg_vals.end());
395 }
396 }
397 }
398
399 for (int t = 0; t < nthreads; ++t) {
400     // mover triplets al tripletList (efficient push_back many)
401     tripletList.insert(tripletList.end(), threadTriplets[t].begin(), threadTriplets[t].end());
402
403     // sumar fuerza local en LinearForm
404     for (int idx = 0; idx < size; ++idx) {
405         LinearForm[idx] += threadLinearForms[t][idx];
406     }
407 }
408 global.setFromTriplets(tripletList.begin(), tripletList.end());
409
410 auto end = std::chrono::high_resolution_clock::now();
411 auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>(end - start);
412 std::cout << "Tiempo tomado: " << duration.count() << " nanosegundos\n";
413
414 if(PrintMatrixes){
415     std::cout<< "-----GlobalMat-----" << std::endl;
416     for (int k = 0; k < global.outerSize(); ++k){
417         for (Eigen::SparseMatrix<double>::InnerIterator it(global, k); it; ++it){
418             std::cout << "(" << it.row() << ", " << it.col() << ") : " << it.value() << "\n";
419         }
420     }
421     //std::cout<<Eigen::MatrixXd(global) <<std::endl;
422 }
423
424 if(PrintMatrixes){
425     std::cout<< "-----GlobalLinearForm-----" << std::endl;
426     for (int k = 0; k < LinearForm.size(); ++k){
427         std::cout << LinearForm[k] << ", ";
428     }
429     std::cout << std::endl;
430 }
431
432 // ----- Imposicion de Dirichlet homogeneo (u = 0) -----
433 std::vector<char> isDirichlet(size, false);
434 std::vector<int> dirichletNodes;
435 dirichletNodes.reserve(2 * (nElementsX + nElementsY));
436
437 for (int j = 0; j < nElementsY; ++j) {
438     for (int i = 0; i < nElementsX; ++i) {
439         int idx = i + j * nElementsX;
440         if (i == 0 || i == nElementsX - 1 || j == 0 || j == nElementsY - 1) {
441             isDirichlet[idx] = 1;
442             dirichletNodes.push_back(idx);
443         }
444     }
445 }
446
447 if(ShowConditions){
448     std::cout << "Numero de nodos Dirichlet detectados: " << dirichletNodes.size() << std::endl;
449 }
450
451 for (int col = 0; col < global.outerSize(); ++col) {
452     for (Eigen::SparseMatrix<double>::InnerIterator it(global, col); it; ++it) {
453         int row = it.row();
454         int c = it.col();
455         if (isDirichlet[row] && row != c) {
456             it.valueRef() = 0.0;
457         }
458     }
459 }
460
461 for (int k = 0; k < (int)dirichletNodes.size(); ++k) {
462     int colIdx = dirichletNodes[k];
463     for (Eigen::SparseMatrix<double>::InnerIterator it(global, colIdx); it; ++it) {
464         int row = it.row();
465         if (row != colIdx) {
466             it.valueRef() = 0.0;
467         }
468     }
469 }
470
471 for (int k = 0; k < (int)dirichletNodes.size(); ++k) {
472     int idx = dirichletNodes[k];

```

```

472     global.coeffRef(idx, idx) = 1.0;
473     LinearForm[idx] = 0.0;
474 }
475
476 if(PrintMatrixes2){
477
478     std::cout<< "-----GlobalMat-----" << std::endl;
479     for (int k = 0; k < global.outerSize(); ++k){
480         for (Eigen::SparseMatrix<double>::InnerIterator it(global, k); it; ++it){
481             std::cout << "(" << it.row() << ", " << it.col() << "): " << it.value() << "\n";
482         }
483     }
484     std::cout<< "-----GlobalLinearForm-----" << std::endl;
485     for (int k = 0; k < LinearForm.size(); ++k){
486         std::cout << LinearForm[k] << ", ";
487     }
488     std::cout << std::endl;
489 }
490
491 global.makeCompressed();
492
493 if(ShowSolutions){
494     std::cout << "Resolviendo sistema (K u = F) con Conjugate Gradient..." << std::endl;
495 }
496
497 int maxIter = std::max(100, std::min(5 * size, 500000));
498 auto tsolve_start = std::chrono::high_resolution_clock::now();
499 Eigen::VectorXd solution;
500 // IncompleteCholesky como preconditionador (SPD)
501 try {
502     using Precond = Eigen::IncompleteCholesky<double>;
503     Eigen::ConjugateGradient<Eigen::SparseMatrix<double>, Eigen::Lower|Eigen::Upper, Precond> solver;
504     solver.compute(global);
505     if (solver.info() != Eigen::Success) {
506         throw std::runtime_error("Precond compute fallo (IC).");
507     }
508     solver.setTolerance(1e-8);
509     solver.setMaxIterations(maxIter);
510     solution = solver.solve(LinearForm);
511     auto tsolve_end = std::chrono::high_resolution_clock::now();
512
513     double elapsed_s = std::chrono::duration_cast<std::chrono::duration<double>>(tsolve_end - tsolve_start).count();
514     std::cout << "Tiempo solucion: " << elapsed_s << " s\n";
515     if(ShowSolutions){
516         std::cout << "Solver info (IC precondition): " << solver.info() << "\n";
517         std::cout << "Iterations: " << solver.iterations() << ", Estimated error: " << solver.error() << "\n";
518         std::cout << "Solucion: " << solution.transpose() << std::endl;
519     }
520 }
521
522 } catch (const std::exception &e) {
523     // Fallback: usar preconditionador diagonal (Jacobi) si IC falla
524     if(ShowSolutions){
525         std::cerr << "Fallo preconditionador IncompleteCholesky o excepcion: " << e.what() << "\n";
526         std::cerr << "Usando ConjugateGradient + DiagonalPreconditioner (fallback)." << std::endl;
527     }
528     Eigen::ConjugateGradient<Eigen::SparseMatrix<double>, Eigen::Lower|Eigen::Upper, Eigen::DiagonalPreconditioner<double>> solver2;
529     solver2.compute(global);
530     solver2.setTolerance(1e-8);
531     solver2.setMaxIterations(2*maxIter);
532     solution = solver2.solve(LinearForm);
533
534     auto tsolve_end2 = std::chrono::high_resolution_clock::now();
535     double elapsed_s2 = std::chrono::duration_cast<std::chrono::duration<double>>(tsolve_end2 - tsolve_start).count();
536     std::cout << "Tiempo solucion: " << elapsed_s2 << " s\n";
537     if(ShowSolutions){
538         std::cout << "Solver info (Diag precondition): " << solver2.info() << "\n";
539         std::cout << "Iterations: " << solver2.iterations() << ", Estimated error: " << solver2.error() << "\n";
540         std::cout << "Solucion: " << solution.transpose() << std::endl;
541     }
542 }
543
544 // ----- Evaluacion en los elementos de la Solucion -----
545
546
547
548
549
550 iMatrix<double> MatSol(nElementsX, nElementsY, solution);
551 //iMatrix<double> MatSol=MatSol1.Transpose();
552 iMatrix<double> SolEvals(nEvalsX*hX, nEvalsY*hY);
553 iMatrix<double> SolEvalsX(nEvalsX*hX, nEvalsY*hY);
554 iMatrix<double> SolEvalsY(nEvalsX*hX, nEvalsY*hY);
555 iMatrix<double> SolAnalytic(nEvalsX*hX, nEvalsY*hY);
556 iMatrix<double> SolAnalyticX(nEvalsX*hX, nEvalsY*hY);
557 iMatrix<double> SolAnalyticY(nEvalsX*hX, nEvalsY*hY);
558 iMatrix<std::vector<double>> Surf(nEvalsX*hX, nEvalsY*hY);
559 for (unsigned int i = 0; i < hX; i++) {
560     for (unsigned int j = 0; j < hY; j++) {
561
562         double auxeval1X= (IntervalsX[i+1] - IntervalsX[i])/2;
563         double auxeval2X= (IntervalsX[i+1] + IntervalsX[i])/2;
564         double auxeval1Y= (IntervalsY[j+1] - IntervalsY[j])/2;
565         double auxeval2Y= (IntervalsY[j+1] + IntervalsY[j])/2;
566
567
568         int spanU = SpanIndexX[i];
569         int spanV = SpanIndexY[j];
570         iMatrix<double> activeWeights=NewWeights.GetSubMat(spanU-pX,spanU, spanV-pY, spanV).Transpose();
571         auto &DersBasisX_vec = Basis_and_DersX[i]; // vector<iMatrix<double>> size nEvalsX
572         auto &DersBasisY_vec = Basis_and_DersY[j]; // vector<iMatrix<double>> size nEvalsY
573         for(int a = 0; a < nEvalsX; ++a){
574

```

```

575     double EvalPointX=auxeval1X*nodesX(a)+auxeval2X;
576     //std::cout << "EvalPointX: " << EvalPointX << ". ";
577     for(int b = 0; b < nEvalsY; ++b){
578         double EvalPointY=auxeval1Y*nodesY(b)+auxeval2Y;
579         iMatrix<iMatrix<double>> RationalizedBasis2d=RationalizeDersBasisFuns2D(DersBasisX_vec[a], DersBasisY_vec[b],activeWeights,pX,pY);
580
581         iMatrix<double> Rbasis = RationalizedBasis2d(0, 0);
582         iMatrix<double> RbasisX = RationalizedBasis2d(1, 0);
583         iMatrix<double> RbasisY = RationalizedBasis2d(0, 1);
584         iMatrix<double> LocalSol = MatSol.GetSubMat(spanU - pX, spanU, spanV - pY, spanV);
585         double u_h = 0.0;
586         double u_hX= 0.0;
587         double u_hY= 0.0;
588         for (int ii = 0; ii < Rbasis.GetNumRows(); ++ii) {
589             for (int jj = 0; jj < Rbasis.GetNumCols(); ++jj) {
590                 u_h += Rbasis(ii, jj) * LocalSol(jj, ii);
591                 u_hX+= RbasisX(ii,jj) * LocalSol(jj, ii);
592                 u_hY+= RbasisY(ii,jj) * LocalSol(jj, ii);
593             }
594         }
595         SolEvals(i * nEvalsX + a, j * nEvalsY + b) = u_h;
596         SolEvalsX(i * nEvalsX + a, j * nEvalsY + b) = u_hX;
597         SolEvalsY(i * nEvalsX + a, j * nEvalsY + b) = u_hY;
598         std::vector<double> SurfPoint=SurfacePointRational(nY, pY, KnotVectorY, nX, pX, KnotVectorX, NewCtrlPtsW, EvalPointY, EvalPointX);
599         Surf(i*nEvalsX + a, j*nEvalsY + b)=SurfPoint;
600         SolAnalytic(i*nEvalsX+a, j*nEvalsY +b) = analyticSol(SurfPoint[0], SurfPoint[1]);
601         SolAnalyticX(i*nEvalsX+a, j*nEvalsY +b) = analyticSol_dx(SurfPoint[0], SurfPoint[1]);
602         SolAnalyticY(i*nEvalsX+a, j*nEvalsY +b) = analyticSol_dy(SurfPoint[0], SurfPoint[1]);
603         /*
604         std::cout<<"-----"<<std::endl;
605         std::cout<< "SurfPoint[0]=" << SurfPoint[0]<< ", EvalPointX="<< EvalPointX <<std::endl;
606         std::cout<< "SurfPoint[1]=" << SurfPoint[1]<< ", EvalPointY="<< EvalPointY <<std::endl;
607         std::cout << "Analytic= " << SolAnalytic(i*nEvalsX+a, j*nEvalsY +b) << ", Numeric= " << SolEvals(i*nEvalsX + a, j*nEvalsY + b) << std::endl;
608         */
609     }
610 }
611 }
612 }
613 //MatSol.PrintMatrix();
614
615 //ExportToTxt(SolEvals, Surf, PATH + "surfaceCircle3d_hX="+ std::to_string(static_cast<int>(hX)) + "_pX=" + to_string(pX) + "_hY="+ std::to_string(static_cast<int>(hY)) + ".txt");
616 // "p=" + to_string(pY) + " data.txt");
617
618 //-----L2 norm calculations-----
619
620 double L2_error_sq = 0.0;
621 double H1_semi_error_sq = 0.0;
622
623 for (unsigned int i = 0; i < hX; i++) {
624     double auxeval1X= (IntervalsX[i+1] - IntervalsX[i])/2;
625     double auxeval2X= (IntervalsX[i+1] + IntervalsX[i])/2;
626     for (unsigned int j = 0; j < hY; j++) {
627         double auxeval1Y= (IntervalsY[j+1] - IntervalsY[j])/2;
628         double auxeval2Y= (IntervalsY[j+1] + IntervalsY[j])/2;
629         double h_elementos = auxeval1X*auxeval1Y;
630         double cumSumElements=0.0;
631         double cumSumH1 = 0.0;
632         for (int a = 0; a < nEvalsX; ++a) {
633             double EvalPointX=auxeval1X*nodesX(a)+auxeval2X;
634             for (int b = 0; b < nEvalsY; ++b) {
635                 // --- Diferencial parametrico ---
636
637
638
639
640                 double EvalPointY=auxeval1Y*nodesY(b)+auxeval2Y;
641                 iMatrix<std::vector<double>> Allders = SurfaceDerivsAlg1(KnotVectorY.size()-2-pY,pY,KnotVectorY,KnotVectorX.size()-2-pX,pX,
642                                     KnotVectorX,NewCtrlPtsW, EvalPointY, EvalPointX, 1);
643
644
645                 iMatrix<std::vector<double>> Aders(2, 2);
646                 iMatrix<double> Wders(2, 2);
647                 for(unsigned int iteri=0; iteri<=1; iteri++){
648                     for(unsigned int iterj=0; iterj<=1; iterj++){
649                         Aders(iteri,iterj)= std::vector<double>(Allders(iteri,iterj).begin(), Allders(iteri,iterj).begin()+3);
650                         Wders(iteri,iterj)=Allders(iteri,iterj)[3];
651                     }
652                 }
653                 iMatrix<std::vector<double>> SurfDers= RatSurfaceDerivs(Aders, Wders, 1);
654
655                 // Punto fisico (x, y, z) de la superficie S
656                 std::vector<double> SurfPoint = SurfDers(0, 0);
657                 double x_phys = SurfPoint[0];
658                 double y_phys = SurfPoint[1];
659
660                 // Gradiente fisico de la solucion analitica (grad x u)
661                 double du_dx_analytic = -x_phys / 2.0;
662                 double du_dy_analytic = -y_phys / 2.0;
663
664                 // Gradiente parametrico de la solucion analitica (grad xi u) usando la Regla de la Cadena
665                 // dS/dxi = SurfDers(0, 1) = (dx/dxi, dy/dxi, dz/dxi)
666                 std::vector<double> S_xi = SurfDers(0, 1);
667                 double du_dxi_analytic = du_dx_analytic * S_xi[0] + du_dy_analytic * S_xi[1];
668                 // (El termino dz/dxi * du/dz es 0 ya que u no depende de z)
669
670                 // dS/deta = SurfDers(1, 0) = (dx/deta, dy/deta, dz/deta)
671                 std::vector<double> S_eta = SurfDers(1, 0);
672                 double du_deta_analytic = du_dx_analytic * S_eta[0] + du_dy_analytic * S_eta[1];
673
674
675                 double g11=dot_product(SurfDers(1,0),SurfDers(1,0));
676                 double g12=dot_product(SurfDers(1,0),SurfDers(0,1));
677                 double g22=dot_product(SurfDers(0,1),SurfDers(0,1));

```

```

678
679         double detg=g11*g22-g12*g12;
680         double J=std::sqrt(detg);
681         double Jinv=1/J;
682
683         double g11c_J = g22 * Jinv; // g-11 * J
684         double g12c_J = -g12 * Jinv; // g-12 * J
685         double g22c_J = g11 * Jinv; // g-22 * J
686         //double dXiEta = auxevaliX * auxevaliY * GaussWeights(a,b);
687         double diff = SolEvals(i*nEvalsX + a, j*nEvalsY + b) - SolAnalytic(i*nEvalsX + a, j*nEvalsY + b);
688         double diff_dx = SolEvalsX(i*nEvalsX + a, j*nEvalsY + b) - du_dxi_analytic; // d/dxi
689         double diff_dy = SolEvalsY(i*nEvalsX + a, j*nEvalsY + b) - du_deta_analytic; // d/deta
690
691
692         // --- Aportacion al error L2 ---
693         cumSumElements += diff * diff * GaussWeights(a,b)*J;
694         cumSumH1 += (diff_dx * diff_dx * g22c_J + 2.0*g12c_J*diff_dx*diff_dy + diff_dy * diff_dy*g11c_J) * GaussWeights(a,b);
695     }
696 }
697 L2_error_sq+=cumSumElements*h_elementos;
698 H1_semi_error_sq += cumSumH1 * h_elementos;
699 }
700 }
701
702 double L2_error = std::sqrt(L2_error_sq);
703 double H1_error = sqrt(L2_error_sq + H1_semi_error_sq);
704 std::cout << "Norma L2 = " << L2_error << std::endl;
705 std::cout << "Norma H1 = " << H1_error << std::endl;

```
